# 07 Python SAST Scanner

07-python-sast-scanner

Parses Python into a syntax tree (never runs it), flags 8 CWE-mapped risky patterns, and writes JSON, SARIF 2.1.0 and HTML reports with suppression support.

**Verified here: 86 passed; 0 failed; 0 skipped**

## Measured results

- 23/23 planted issues detected across all 8 rules, 0 false positives on benign look-alike code (own corpus, not an independent benchmark)
- psf/requests v2.32.3: 36 files / 11,209 lines scanned in 0.157 s, 14 findings, 0 exploitable after manual triage
- pypa/sampleproject @ 621e4974ca25: 0 findings
- SARIF output validates against the official OASIS 2.1.0 JSON schema
- 86 pytest tests pass, incl. a test proving scanned code is never executed

## Data provenance and caveats

- samples/vulnerable and samples/benign: written for this project; all credentials are fake strings
- pypa/sampleproject commit 621e4974ca25ce531773def586ba3ed8e736b3fc via https://github.com/pypa/sampleproject (zip archive)
- psf/requests tag v2.32.3 (commit 0e322af87745eff34caffe4df68456ebc20d9068) via https://github.com/psf/requests (zip archive)
- SARIF 2.1.0 schema: https://raw.githubusercontent.com/oasis-tcs/sarif-spec/main/sarif-2.1/schema/sarif-schema-2.1.0.json
- Detection is syntactic only; no data-flow/taint proof that untrusted input reaches a sink.
- The 100% recall / 0 FP figure is on a corpus written together with the rules, not an independent benchmark.
- The requests triage (0 exploitable of 14) is a manual judgment recorded in samples/allowlists/requests-triage.yml.
- Known misses are pinned by tests: SQL built in a helper function; secrets under innocuous names.
- JavaScript scanning (optional in the original idea) was not built.

## Project contents

README.md, requirements.txt, cli.py, app.py, src/__init__.py, src/analyzer.py, src/evaluate.py, src/fetch.py, src/findings.py, src/reporters.py, src/rules.py, src/scanner.py, tests/conftest.py, tests/test_app.py, tests/test_corpus_and_cli.py, tests/test_network.py, tests/test_reports.py, tests/test_rules.py, tests/test_safety.py, tests/test_suppression.py, .gitignore, .sast-allowlist.example.yml, data/README.md, pytest.ini, samples/README.md, samples/allowlists/requests-triage.yml, samples/benign/safe_app.py, samples/vulnerable/app/__init__.py, samples/vulnerable/app/config.py, samples/vulnerable/app/db.py, samples/vulnerable/app/utils.py, schemas/sarif-schema-2.1.0.json, scripts/make_charts.py, scripts/make_results.py, scripts/run_demo.sh

# pysast: AST-based Python SAST scanner

A small static application security testing tool that parses Python code into a syntax
tree, flags
eight classes of risky code, and writes JSON, SARIF 2.1.0 and self-contained HTML reports.
 It never
runs the code it scans.

## Problem

Insecure patterns such as hard-coded passwords, SQL built with f-strings, `pickle.loads`
on untrusted
data, `yaml.load` without a safe loader and `shell=True` keep showing up in production
code. A
regex grep finds too much (comments, strings, unrelated `.eval()` methods) and misses
aliased imports.
This project walks Python's own AST instead, so it knows the difference between the
builtin `eval()`
and `model.eval()`, resolves `import hashlib as h; h.md5()`, and reports results in SARIF,
 the format
GitHub code scanning and most CI tools ingest.

## How it works / Architecture

```
 target dir ??? iter_python_files ??? read bytes ??? ast.parse()   (never import/exec)
 (no symlinks,     (excludes .git,       (?1 MB,          ?
  .venv, ...)       venv, build...)      UTF-8)          ?
                                        SecurityVisitor (one pass)
                                        ? import-alias table
                                        ? per-function SQL-variable table
                                        ? rule checks PY001-PY008
                                              ? raw findings
                                              ?
                                inline comments (tokenize) + allowlist (YAML/JSON)
                                  # nosec / # sast: ignore[PY006] reason
                                              ? findings (+suppressed flag)
                                              ?
                              text / JSON / SARIF 2.1.0 / HTML      Streamlit dashboard
```

| Rule  | Name | Severity | CWE | What is matched (syntactically) |
|-------|------|----------|-----|---------------------------------|
| PY001 | hardcoded-credential | HIGH | CWE-798 | string literal assigned / passed /
compared / defaulted to a secret-looking name (`password`, `api_key`, `client_secret`,
`token`...) unless it is an obvious placeholder |
| PY002 | sql-string-formatting | HIGH | CWE-89 | f-string, `+`, `%` or `.format()` SQL
passed to `execute/executemany/executescript/read_sql/raw`, directly or through a variable
 assigned in the same function |
| PY003 | eval-exec | HIGH (LOW for a constant string) | CWE-95 | builtin `eval()` /
`exec()` |
| PY004 | unsafe-pickle | HIGH | CWE-502 | `pickle/_pickle/cPickle/dill/marshal`
`load(s)`, `Unpickler`, `shelve.open`, `pandas.read_pickle`, `jsonpickle.decode` |
| PY005 | unsafe-yaml-load | HIGH (MEDIUM for `full_load`) | CWE-502 | `yaml.load` without
 `SafeLoader/CSafeLoader/BaseLoader`, `unsafe_load`, `full_load` |
| PY006 | weak-hash | MEDIUM | CWE-328 | `hashlib.md5/sha1`, `hashlib.new("md5"/"sha1")`,
PyCryptodome MD5/SHA1, unless `usedforsecurity=False` |
| PY007 | subprocess-shell-true | HIGH (MEDIUM for a constant command) | CWE-78 |
`subprocess.run/call/Popen/check_call/check_output` with `shell=True` |
| PY008 | os-system-shell | MEDIUM (LOW for a constant) | CWE-78 | `os.system`,

`os.popen`, `subprocess.getoutput` |

Each finding carries rule id, severity, confidence, file, line, column, end position, message, the
source line (credential values masked), a fix recommendation and a 16-hex-char fingerprint (sha256 of rule + path + normalised line, so it survives unrelated line shifts).

## Security design decisions

* **Parse, never execute.** Only `ast.parse()` touches scanned code. No import, `compile()` to
  bytecode, `setup.py`, `conftest.py` or test of the target is ever run. A test plants a file whose
  top-level code would write a marker file and call `os._exit(3)`; the scan finishes and the marker
  never appears.
* **Hostile input handling.** Symlinks are not followed (directories or files), so a repo cannot point
  the scanner at `/etc` or `~/.ssh`. Files over 1 MB and files that fail to parse or decode are skipped
  and listed as errors, not crashes. GitHub archives are extracted with zip-slip protection (every
  path must stay inside the destination), symlink entries are dropped, and archives over 50 MB
  compressed / 200 MB expanded are refused.
* **Reports don't leak.** Credential literals are masked (`"fa**********"`) in every report format.
  The HTML report is autoescaped (a `<script>` in scanned code renders as text), has no JavaScript
  and no external assets. SARIF uses a `SRCROOT` base id rather than the scanning machine's absolute
  path.
* **Placeholder awareness for PY001.** Empty strings, `changeme`, `<your-password>`, `${TOKEN}`,
  `{{ secret }}`, `xxxx`, `your_api_key`, values shorter than 6 characters, and names that hold
  metadata (`*_url`, `*_file`, `*_field`, `*_env`, `*_hint`, `*_min_length`...) are ignored.
* **Suppression is explicit and auditable.** `# nosec`, `# sast: ignore` or
  `# sast: ignore[PY006] reason` on any line of the flagged statement, read with `tokenize` so a
  `# nosec` inside a string literal doesn't count. Project-level suppressions live in an allowlist
  file (by rule + path glob or by fingerprint, each with a reason). Suppressed findings stay in every
  report, marked as suppressed with the reason (SARIF `suppressions` with `inSource` / `external`).
  `disable_rules` and `exclude_paths` remove results entirely.
* **Threat model.** The scanner is a review aid for code you are about to ship. It does not defend
  against a developer deliberately hiding dangerous code (e.g. `getattr(__builtins__, "ev"+"al")`).

## Project structure

```
07-python-sast-scanner/
??? cli.py                # argparse CLI: scan, evaluate, fetch-github, rules
??? app.py                  # Streamlit dashboard (Overview, Findings, Scan a snippet, Rules)
```

```
??? src/
?   ??? rules.py              # rule catalogue: id, severity, CWE, fix
?   ??? findings.py           # Finding dataclass + fingerprint
?   ??? analyzer.py           # AST visitor: all detection logic
?   ??? scanner.py            # file discovery, suppressions, allowlist, ScanResult
?   ??? reporters.py          # text / JSON / SARIF 2.1.0 / HTML
?   ??? fetch.py              # GitHub zip download + safe extraction (no execution)
?   ??? evaluate.py           # precision/recall against `# expect:` markers
??? samples/
?   ??? vulnerable/app/       # deliberately insecure demo app (fake strings only)
?   ??? benign/               # safe look-alike code (any finding = false positive)
?   ??? allowlists/requests-triage.yml
??? schemas/sarif-schema-2.1.0.json   # official OASIS schema, used by the tests
??? scripts/run_demo.sh       # end-to-end demo that produced outputs/
??? scripts/make_charts.py
??? tests/                    # 86 pytest tests
??? outputs/                  # real reports, charts, pytest + demo output
??? .sast-allowlist.example.yml
```

## Setup & how to run

```bash
pip install -r requirements.txt

python cli.py rules
python cli.py scan samples/vulnerable                    # text output
python cli.py scan samples/vulnerable --out outputs/reports/vulnerable   # + JSON, SARIF,
HTML
python cli.py scan path/to/repo --allowlist .sast-allowlist.yml --fail-on HIGH   # CI gate
 (exit 1)
python cli.py scan path/to/repo --format json --min-severity MEDIUM
python cli.py evaluate                                   # metrics on the sample
corpus

# Fetch a public repo as a zip (extract only, nothing is run) and scan it
python cli.py fetch-github pypa/sampleproject --ref
621e4974ca25ce531773def586ba3ed8e736b3fc --dest data/real/sampleproject
python cli.py scan
data/real/sampleproject/sampleproject-621e4974ca25ce531773def586ba3ed8e736b3fc

bash scripts/run_demo.sh      # everything above, regenerating outputs/
pytest -v                     # 86 tests (one needs internet: -m "not network" to skip
it)
streamlit run app.py
```

To use it in GitHub code scanning, upload the `.sarif` file with `github/codeql-
action/upload-sarif`.

## Results (from the run in this repo)

**Tests:** 86 passed, 0 failed, 0 skipped (`outputs/pytest_output.txt`). They cover 29
true-positive
snippets across all 8 rules, 24 benign look-alikes, inline and allowlist suppression,
fingerprint
stability, SARIF validation against the official OASIS 2.1.0 JSON schema (with findings
and empty),
HTML escaping, secret masking in all reports, no execution of scanned code, symlink and
zip-slip

handling, the CLI exit codes, the Streamlit app (AppTest), and a live fetch-and-scan of a
real repo.
Two tests deliberately pin known blind spots (see Limitations).

**Bundled sample corpus** (`python cli.py evaluate`):

| Corpus | Files | Lines | Planted issues | Detected | False positives |
|---|---|---|---|---|---|
| samples/vulnerable | 4 | 119 | 23 (all 8 rules) | 23 | 0 |
| samples/benign | 1 | 89 | 0 | n/a | 0 |

Plus one inline-suppressed MD5 in each corpus, correctly reported as suppressed. This
corpus was
written together with the rules, so 23/23 shows the rules behave as designed. It is
**not** an
independent accuracy benchmark.

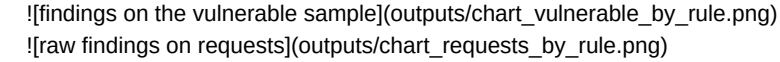
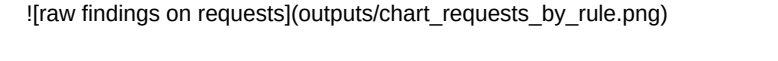
**Real public repositories** (downloaded as GitHub zip archives, never executed):

| Repo | Version / commit | Files | Lines | Findings |
|---|---|---|---|---|
| pypa/sampleproject | `621e4974ca25ce531773def586ba3ed8e736b3fc` (2024-11-06) | 5 | 77 |
**0** |
| psf/requests | tag v2.32.3, commit `0e322af87745eff34caffe4df68456ebc20d9068` | 36 |
11,209 | **14** (9 HIGH, 3 MEDIUM, 2 LOW) in 0.16 s |

Manual triage of the 14 requests findings (recorded in `samples/allowlists/requests-
triage.yml`):

| Finding | Where | Verdict |
|---|---|---|
| 7 x PY004 `pickle.loads` | tests/test_requests.py | pattern correct, not exploitable:
round-trips objects the test just pickled |
| 1 x PY001 `PASSWORD = "%!*'..."` | tests/test_utils.py | false positive: a fixture of
URL-special characters |
| 3 x PY006 MD5/SHA-1 | src/requests/auth.py | pattern correct, accepted risk: HTTP Digest
 auth (RFC 7616) requires them when the server chooses them |
| 1 x PY003 `exec(f.read(), about)` | setup.py | pattern correct, low risk: executes the
package's own `__version__.py` |
| 2 x PY008 `os.system("twine upload dist/*")` | setup.py | constant commands in a
maintainer helper (rated LOW) |

So on requests, 13 of 14 hits matched the pattern they look for, and none was an
exploitable
vulnerability on manual review. That is the expected behaviour of a syntactic scanner on
mature
code, and the reason the allowlist exists. With the triage allowlist the same scan reports
 0 active,
14 suppressed, and `--fail-on HIGH` passes.

## Sample output

```
$ python cli.py scan samples/vulnerable --show-suppressed
app/config.py:6:1: HIGH   PY001 hardcoded-credential: Hard-coded credential in assignment
to 'DB_PASSWORD'
    DB_PASSWORD = "fa**********"  # expect: PY001
```

```
app/db.py:9:5: HIGH   PY002 sql-string-formatting: SQL string built with
formatting/concatenation passed to .execute(); use bound parameters
    cur.execute(f"SELECT * FROM users WHERE name = '{username}'")  # expect: PY002
app/db.py:15:12: HIGH   PY002 sql-string-formatting: SQL string built with
formatting/concatenation passed to .execute() (via variable 'query'); use bound parameters
    return conn.execute(query).fetchall()  # expect: PY002
app/utils.py:30:12: HIGH   PY005 unsafe-yaml-load: yaml.load() with no Loader argument;
use yaml.safe_load()
    return yaml.load(text)  # expect: PY005
app/utils.py:54:12: HIGH   PY007 subprocess-shell-true: subprocess.check_output() with
shell=True and a non-literal command
    return co(f"du -sh {path}", shell=True)  # expect: PY007
app/utils.py:63:12: MEDIUM PY006 weak-hash: Weak hash algorithm MD5 (hashlib.md5)
[suppressed]
    return hashlib.md5(data).hexdigest()  # sast: ignore[PY006] cache key, not security
...
4 files, 119 lines scanned in 0.0033s | 23 findings (HIGH 19, MEDIUM 4, LOW 0), 1
suppressed, 0 skipped
```

Full output: `outputs/sample_output.txt`. Reports: `outputs/reports/` and `outputs/real/`
(`.json`,
`.sarif`, `.html`).

## Limitations & future work

* **No data-flow proof.** Every rule is a syntactic heuristic. The scanner does not check
whether
  attacker-controlled data reaches a sink, so it reports safe code that has a risky shape
(the pickle
  round-trips in requests) and misses risky code with an unusual shape. SQL built in a
helper function
  and passed as `db.execute(build_query(x))` is **not** detected (pinned by a test).
Variable tracking
  for PY002 covers only simple assignments in the same function.
* **PY001 is name-based.** A secret stored under an innocuous name (`conf_value =
"AKIA..."`) is
  missed. An entropy/regex secret engine catches those, which is what a git secret scanner
 covers.
* Weak-hash findings cannot tell checksums from security use unless the code says
  `usedforsecurity=False`.
* Dynamic tricks (`getattr(builtins, "eval")`, `importlib.import_module("pickle")`) evade
the rules.
* Python only; the optional JavaScript support from the original idea was not built.
* Future work: intra-procedural taint tracking (sources such as `request.args`, `input()`
to sinks),
  more rules (`tempfile.mktemp`, `requests(verify=False)`, Jinja2 `autoescape=False`,
`assert` for
  auth), a pre-commit hook, and a baseline file to report only new findings.

## Resume bullets

* Built an AST-based Python SAST scanner (8 rules mapped to CWE-78/89/95/502/798/328) with
 import-alias
  resolution, severity/confidence scoring, inline and allowlist suppression, and JSON,
SARIF 2.1.0 and
  HTML reporting; reports validate against the official OASIS SARIF schema.
* Detected all 23 planted vulnerabilities in a purpose-built vulnerable sample app with 0
false positives
  on a benign look-alike corpus, backed by an 86-test pytest suite covering true

positives, benign
  cases, suppression, schema validity and safe handling of untrusted repositories.
* Scanned psf/requests v2.32.3 (36 files, 11,209 lines) in 0.16 s and triaged all 14
findings by hand
  (0 exploitable). Documented why syntactic matching flags protocol-mandated MD5 and test-
only pickle,
  and encoded the triage as an auditable allowlist that brings the CI gate to green.
* Hardened the tool against hostile repositories: it never executes scanned code (verified
 by a test),
  doesn't follow symlinks, uses zip-slip-safe archive extraction, escapes HTML output and
masks
  credentials in every report.

```
== pysast rules
PY001  HIGH   CWE-798  hardcoded-credential     Hard-coded credential assigned to a secret-looking name
PY002  HIGH   CWE-89   sql-string-formatting    SQL query built with string formatting/concatenation and passed to execute()
PY003  HIGH   CWE-95   eval-exec                Use of eval()/exec() can execute arbitrary code
PY004  HIGH   CWE-502  unsafe-pickle            Deserialising with pickle/dill/marshal can execute attacker-controlled code
PY005  HIGH   CWE-502  unsafe-yaml-load        yaml.load() without SafeLoader can construct arbitrary Python objects
PY006  MEDIUM CWE-328  weak-hash               MD5/SHA-1 are broken for collision resistance
PY007  HIGH   CWE-78   subprocess-shell-true   subprocess call with shell=True enables shell command injection
PY008  MEDIUM CWE-78   os-system-shell         os.system()/os.popen() always run through a shell

== Ground-truth evaluation on samples/
{
  "expected": 23,
  "true_positives": 23,
  "false_negatives": [],
  "false_positives_vulnerable": [],
  "false_positives_benign": [],
  "benign_files": 1,
  "benign_lines": 89,
  "vulnerable_files": 4,
  "vulnerable_lines": 119,
  "suppressed_in_vulnerable": 1,
  "precision": 1.0,
  "recall": 1.0,
  "per_rule": {
    "PY001": {
      "expected": 7,

== Scan of the deliberately vulnerable sample repo
wrote outputs/reports/vulnerable.json
wrote outputs/reports/vulnerable.sarif
wrote outputs/reports/vulnerable.html
app/config.py:6:1: HIGH   PY001 hardcoded-credential: Hard-coded credential in assignment to 'DB_PASSWORD'
    DB_PASSWORD = "fa************"  # expect: PY001
app/config.py:7:1: HIGH   PY001 hardcoded-credential: Hard-coded credential in assignment to 'API_KEY'
    API_KEY = "de************"  # expect: PY001
app/config.py:8:1: HIGH   PY001 hardcoded-credential: Hard-coded credential in assignment to 'SECRET_KEY'
    SECRET_KEY: str = "no************"  # expect: PY001
app/config.py:14:17: HIGH   PY001 hardcoded-credential: Hard-coded credential in dict key 'password'
    "password": "fa************",  # expect: PY001
app/config.py:21:9: HIGH   PY001 hardcoded-credential: Hard-coded credential in assignment to 'smtp_password'
    self.smtp_password = "fa************"  # expect: PY001
app/db.py:9:5: HIGH   PY002 sql-string-formatting: SQL string built with formatting/concatenation passed to .execute(); use bound parameters
    cur.execute(f"SELECT * FROM users WHERE name = '{username}'")  # expect: PY002
app/db.py:15:12: HIGH   PY002 sql-string-formatting: SQL string built with formatting/concatenation passed to .execute() (via variable 'query'); use bound parameters
    return conn.execute(query).fetchall()  # expect: PY002
app/db.py:19:5: HIGH   PY002 sql-string-formatting: SQL string built with formatting/concatenation passed to .execute(); use bound parameters
    conn.execute("DELETE FROM notes WHERE id = %s" % note_id)  # expect: PY002
app/db.py:24:5: HIGH   PY002 sql-string-formatting: SQL string built with formatting/concatenation passed to .executemany() (via variable 'sql'); use bound parameters
    conn.cursor().executemany(sql, [])  # expect: PY002
app/db.py:28:8: HIGH   PY001 hardcoded-credential: Hard-coded credential in comparison with 'passw
```

```
ord'
    if password == "fa************":  # expect: PY001
app/db.py:34:68: HIGH   PY001 hardcoded-credential: Hard-coded credential in keyword argument 'password'
    return sqlite3.connect("reports.db", timeout=5), dict(password="fa************")  # expect: PY001
app/utils.py:13:12: HIGH   PY003 eval-exec: Call to eval() with a non-literal argument
    return eval(expression)  # expect: PY003
app/utils.py:17:5: HIGH   PY003 eval-exec: Call to exec() with a non-literal argument
    exec(code_text)  # expect: PY003
app/utils.py:21:12: HIGH   PY004 unsafe-pickle: Insecure deserialisation via pickle.loads()
    return pickle.loads(blob)  # expect: PY004
app/utils.py:26:16: HIGH   PY004 unsafe-pickle: Insecure deserialisation via pickle.load()
    return pickle.load(fh)  # expect: PY004
app/utils.py:30:12: HIGH   PY005 unsafe-yaml-load: yaml.load() with no Loader argument; use yaml.safe_load()
    return yaml.load(text)  # expect: PY005
app/utils.py:34:12: HIGH   PY005 unsafe-yaml-load: yaml.load() with Loader=Loader; use yaml.safe_load()
    return yaml.load(text, Loader=yaml.Loader)  # expect: PY005
app/utils.py:50:12: HIGH   PY007 subprocess-shell-true: subprocess.run() with shell=True and a non-literal command
    return subprocess.run("ping -c 1 " + host, shell=True)  # expect: PY007
app/utils.py:54:12: HIGH   PY007 subprocess-shell-true: subprocess.check_output() with shell=True and a non-literal command
    return co(f"du -sh {path}", shell=True)  # expect: PY007
app/utils.py:38:12: MEDIUM PY006 weak-hash: Weak hash algorithm MD5 (hashlib.md5)
    return hashlib.md5(pw.encode()).hexdigest()  # expect: PY006
app/utils.py:42:12: MEDIUM PY006 weak-hash: Weak hash algorithm SHA1 (hashlib.sha1)
    return hl.sha1(data).hexdigest()  # expect: PY006
app/utils.py:46:12: MEDIUM PY006 weak-hash: Weak hash algorithm MD5 (hashlib.new)
    return hashlib.new("md5", data).hexdigest()  # expect: PY006
app/utils.py:58:5: MEDIUM PY008 os-system-shell: os.system() runs its argument through a shell
    os.system("tar czf backup.tgz " + path)  # expect: PY008
app/utils.py:63:12: MEDIUM PY006 weak-hash: Weak hash algorithm MD5 (hashlib.md5) [suppressed]
    return hashlib.md5(data).hexdigest()  # sast: ignore[PY006] cache key, not security

4 files, 119 lines scanned in 0.0033s | 23 findings (HIGH 19, MEDIUM 4, LOW 0), 1 suppressed, 0 skipped

== Scan of the benign look-alike corpus
wrote outputs/reports/benign.json
wrote outputs/reports/benign.sarif
wrote outputs/reports/benign.html
safe_app.py:33:12: HIGH   PY002 sql-string-formatting: SQL string built with formatting/concatenation passed to .execute(); use bound parameters [suppressed]
    return conn.execute("SELECT * FROM users WHERE id IN (%s)" % placeholders, ids)  # sast: ignore[PY002] only '?' markers interpolated

1 files, 89 lines scanned in 0.0031s | 0 findings (HIGH 0, MEDIUM 0, LOW 0), 1 suppressed, 0 skipped

== Fetch pypa/sampleproject @ 621e4974ca25ce531773def586ba3ed8e736b3fc (download + extract only; nothing is run)
{
  "repo": "pypa/sampleproject",
  "ref": "621e4974ca25ce531773def586ba3ed8e736b3fc",
  "commit": "621e4974ca25ce531773def586ba3ed8e736b3fc",
  "archive_url": "https://codeload.github.com/pypa/sampleproject/zip/621e4974ca25ce531773def586ba3ed8e736b3fc",
  "archive_sha256": "b3eccda9bfb92813e361eed4f074b97165233eb94ebc60166608ce5412d08a07",
  "archive_bytes": 10342,
  "files_extracted": 12,
  "source_dir": "data/real/sampleproject/sampleproject-621e4974ca25ce531773def586ba3ed8e736b3fc",
  "fetched_at": "2026-10-27T08:19:29+00:00"
```

```
}
wrote outputs/real/sampleproject.json
wrote outputs/real/sampleproject.sarif
wrote outputs/real/sampleproject.html

5 files, 77 lines scanned in 0.0026s | 0 findings (HIGH 0, MEDIUM 0, LOW 0), 0 suppressed, 0 skipped

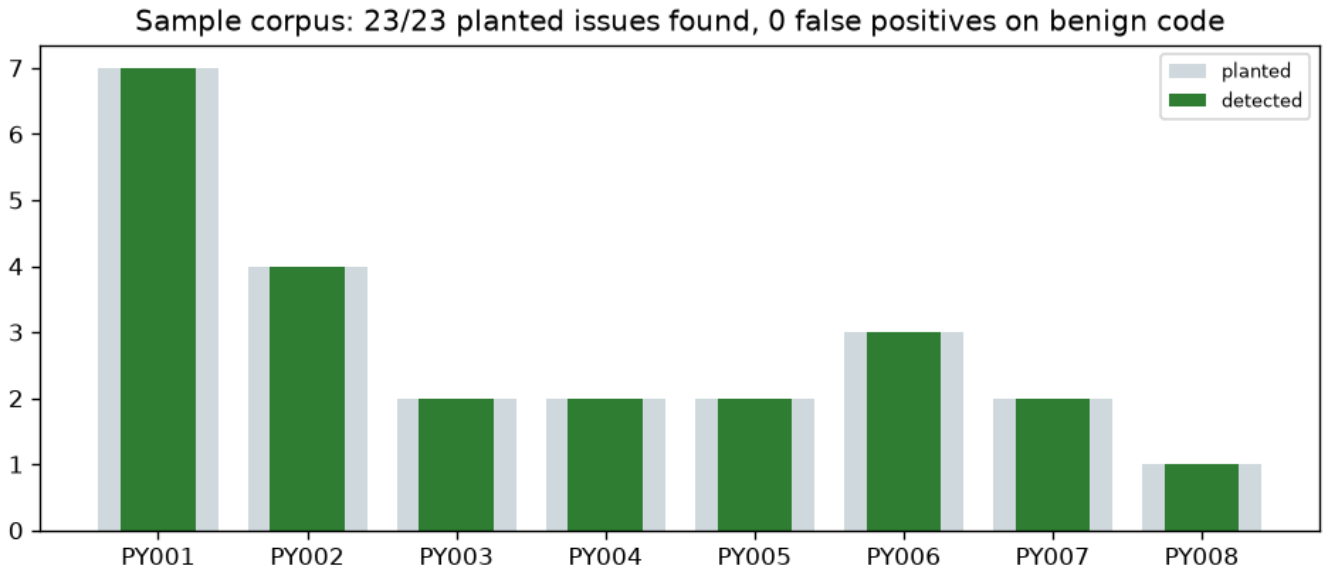
== Fetch psf/requests v2.32.3
{'repo': 'psf/requests', 'ref': 'v2.32.3', 'commit': '0e322af87745eff34caffe4df68456ebc20d9068', 'files_extracted': 129, 'archive_sha256': 'ce709c9c71109afc6d0308b3975dab231c03fb414ac572bcbd55e40909f0a2b3'}
wrote outputs/real/requests.json
wrote outputs/real/requests.sarif
wrote outputs/real/requests.html
setup.py:79:5: HIGH   PY003 eval-exec: Call to exec() with a non-literal argument
    exec(f.read(), about)
tests/test_requests.py:1484:16: HIGH   PY004 unsafe-pickle: Insecure deserialisation via pickle.loads()
    assert pickle.loads(pickle.dumps(r.request))
tests/test_requests.py:1488:14: HIGH   PY004 unsafe-pickle: Insecure deserialisation via pickle.loads()
    pr = pickle.loads(pickle.dumps(r))
tests/test_requests.py:1496:13: HIGH   PY004 unsafe-pickle: Insecure deserialisation via pickle.loads()
    r = pickle.loads(pickle.dumps(p))
tests/test_requests.py:1512:13: HIGH   PY004 unsafe-pickle: Insecure deserialisation via pickle.loads()
    r = pickle.loads(pickle.dumps(p))
tests/test_requests.py:1527:13: HIGH   PY004 unsafe-pickle: Insecure deserialisation via pickle.loads()
    r = pickle.loads(pickle.dumps(p))
tests/test_requests.py:1557:13: HIGH   PY004 unsafe-pickle: Insecure deserialisation via pickle.loads()
    s = pickle.loads(pickle.dumps(s))
tests/test_requests.py:2975:26: HIGH   PY004 unsafe-pickle: Insecure deserialisation via pickle.loads()
    deserialized_error = pickle.loads(pickle.dumps(json_decode_error))
tests/test_utils.py:411:1: HIGH   PY001 hardcoded-credential: Hard-coded credential in assignment to 'PASSWORD'
    USER = PASSWORD = "%!************"
src/requests/auth.py:148:24: MEDIUM PY006 weak-hash: Weak hash algorithm MD5 (hashlib.md5)
    return hashlib.md5(x).hexdigest()
src/requests/auth.py:156:24: MEDIUM PY006 weak-hash: Weak hash algorithm SHA1 (hashlib.sha1)
    return hashlib.sha1(x).hexdigest()
src/requests/auth.py:205:18: MEDIUM PY006 weak-hash: Weak hash algorithm SHA1 (hashlib.sha1)
    cnonce = hashlib.sha1(s).hexdigest()[:16]
setup.py:57:5: LOW    PY008 os-system-shell: os.system() runs its argument through a shell
    os.system("python setup.py sdist bdist_wheel")
setup.py:58:5: LOW    PY008 os-system-shell: os.system() runs its argument through a shell
    os.system("twine upload dist/*")

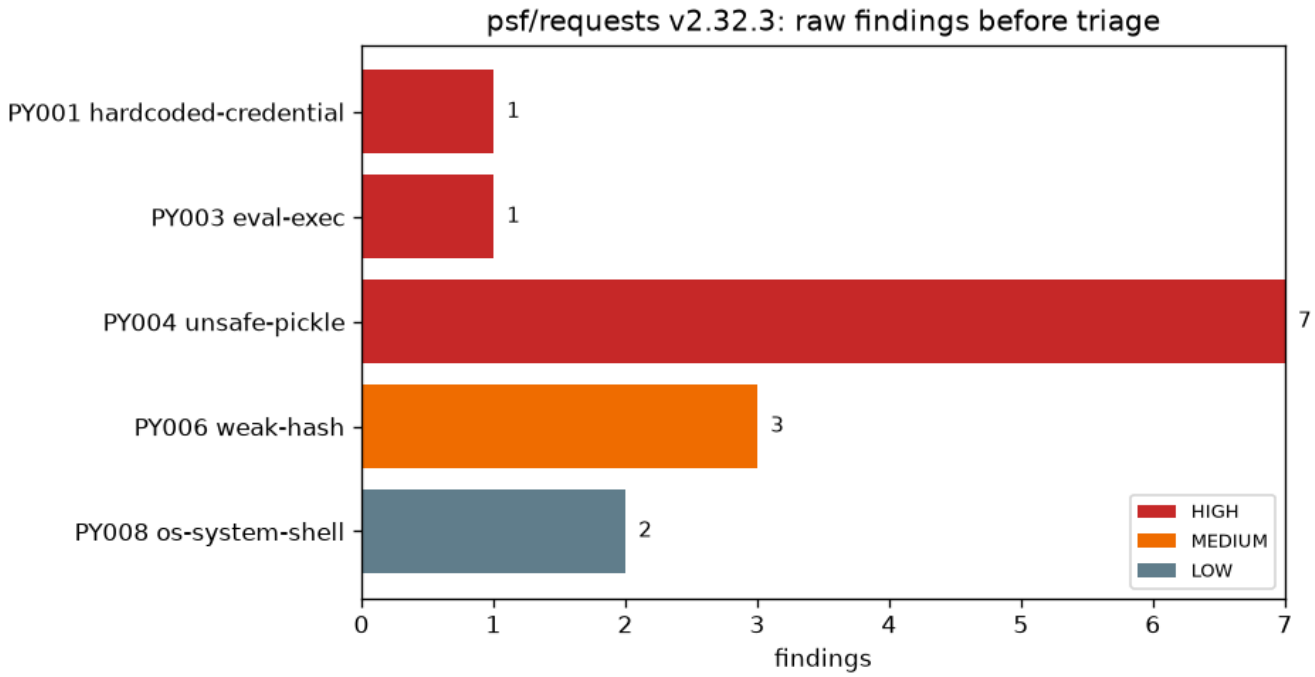
36 files, 11209 lines scanned in 0.157s | 14 findings (HIGH 9, MEDIUM 3, LOW 2), 0 suppressed, 0 skipped

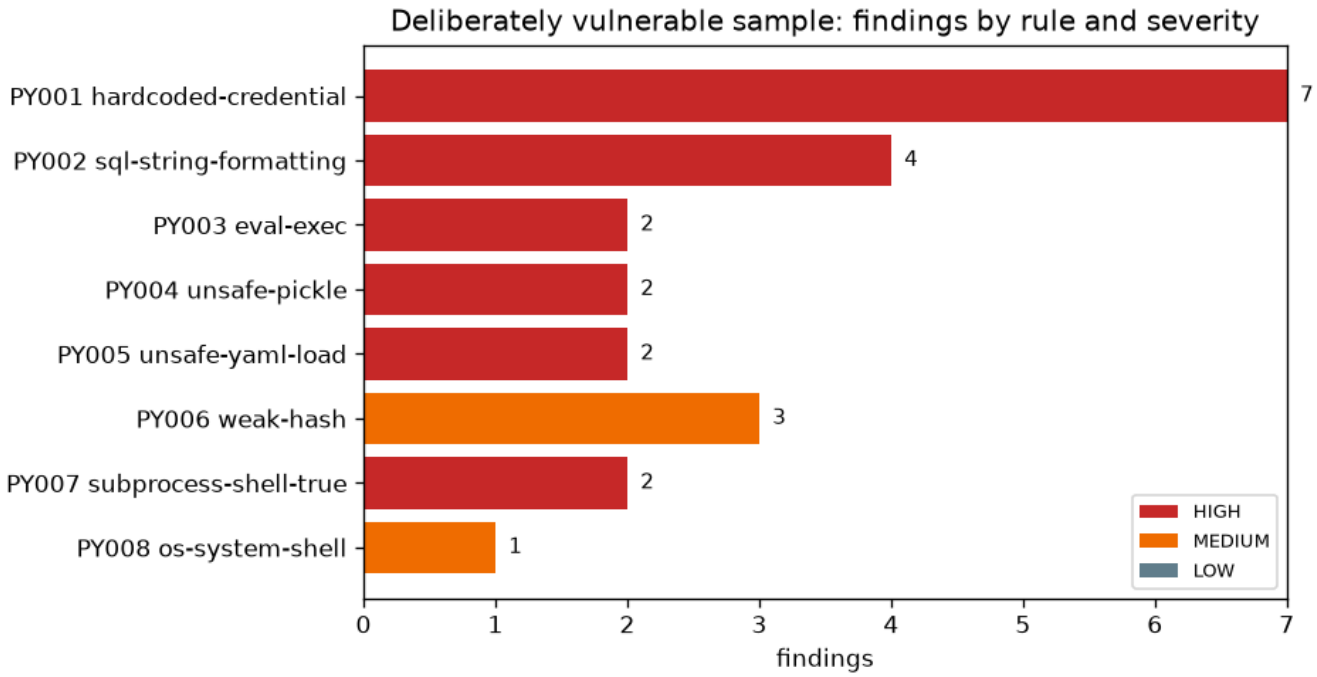
== Same scan with the manual-triage allowlist
wrote outputs/real/requests-triaged.json
wrote outputs/real/requests-triaged.sarif
wrote outputs/real/requests-triaged.html

36 files, 11209 lines scanned in 0.1515s | 0 findings (HIGH 0, MEDIUM 0, LOW 0), 14 suppressed, 0 skipped
CI gate (fail-on HIGH): PASS
charts written: chart_vulnerable_by_rule.png, chart_requests_by_rule.png, chart_corpus_recall.png

```


Sample corpus: 23/23 planted issues found, 0 false positives on benign code
planted
detected
0
1
2
3
4
5
6
7
PY001
PY002
PY003
PY004
PY005
PY006
PY007
PY008


psf/requests v2.32.3: raw findings before triage
PY001 hardcoded-credential
PY003 eval-exec
PY004 unsafe-pickle
PY006 weak-hash
PY008 os-system-shell
1
1
7
3
2
HIGH
MEDIUM
LOW
0
1
2
3
4
5
6
7
findings


Deliberately vulnerable sample: findings by rule and severity
PY001 hardcoded-credential
PY002 sql-string-formatting
PY003 eval-exec
PY004 unsafe-pickle
PY005 unsafe-yaml-load
PY006 weak-hash
PY007 subprocess-shell-true
PY008 os-system-shell
7
4
2
2
2
3
2
1
HIGH
MEDIUM
LOW
0
1
2
3
4
5
6
7
findings

## 07-python-sast-scanner/requirements.txt

```
jinja2>=3.1
pyyaml>=6.0
requests>=2.31
streamlit>=1.35
pandas>=2.0
plotly>=5.20
matplotlib>=3.8
pytest>=8.0
pytest-timeout>=2.3
jsonschema>=4.20

```

## 07-python-sast-scanner/cli.py

```
#!/usr/bin/env python3
"""pysast command-line interface.

Examples:
    python cli.py scan samples/vulnerable --format text
    python cli.py scan samples/vulnerable --out outputs/reports/vulnerable --report json sarif html
    python cli.py scan path/to/repo --allowlist .sast-allowlist.yml --fail-on HIGH
    python cli.py evaluate
    python cli.py fetch-github pypa/sampleproject --ref <commit-sha> --dest data/real/sampleproject
    python cli.py rules
"""
from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

from src import __version__
from src.evaluate import evaluate
from src.fetch import fetch_github
from src.reporters import to_text, write_reports
from src.rules import RULES, SEVERITY_ORDER
from src.scanner import Allowlist, scan_path


def cmd_scan(a) -> int:
    allow = Allowlist.load(a.allowlist) if a.allowlist else Allowlist()
    if a.disable:
        allow.disable_rules |= {r.strip().upper() for r in a.disable.split(",")}
    result = scan_path(a.target, allow, a.min_severity)
    if a.name:
        result.metadata["name"] = a.name
    if a.meta:
        result.metadata.update(json.loads(Path(a.meta).read_text()))
    if a.format == "text":
        print(to_text(result, show_suppressed=a.show_suppressed))
    else:
        from src.reporters import to_json
        print(json.dumps(to_json(result), indent=2))
    if a.out:
        for p in write_reports(result, a.out, a.report):
            print(f"wrote {p}", file=sys.stderr)
    if a.fail_on:
        worst = max((SEVERITY_ORDER[f.severity] for f in result.active), default=0)
        if worst >= SEVERITY_ORDER[a.fail_on]:
            return 1
    return 0


def cmd_evaluate(a) -> int:
    r = evaluate(a.vulnerable, a.benign)
    print(json.dumps(r, indent=2))
    if a.out:
        Path(a.out).write_text(json.dumps(r, indent=2))
    return 0


def cmd_fetch(a) -> int:
    meta = fetch_github(a.repo, a.ref, a.dest)
    print(json.dumps(meta, indent=2))
    return 0

```

```
64
65 def cmd_rules(a) -> int:
66     for r in RULES.values():
67         print(f"{r.id}  {r.severity:<6} {r.cwe:<8} {r.name:<24} {r.short}")
68     return 0
69
70
71 def build_parser() -> argparse.ArgumentParser:
72     p = argparse.ArgumentParser(prog="pysast", description="AST-based Python SAST scanner (parses,
    never executes)")
73     p.add_argument("--version", action="version", version=f"pysast {__version__}")
74     sub = p.add_subparsers(dest="cmd", required=True)
75
76     s = sub.add_parser("scan", help="scan a file or directory")
77     s.add_argument("target")
78     s.add_argument("--format", choices=["text", "json"], default="text", help="stdout format")
79     s.add_argument("--out", help="report base path, e.g. outputs/reports/myrepo")
80     s.add_argument("--report", nargs="+", default=["json", "sarif", "html"],
81                    choices=["json", "sarif", "html"], help="report files to write with --out")
82     s.add_argument("--allowlist", help="YAML/JSON allowlist file")
83     s.add_argument("--disable", help="comma-separated rule ids to disable")
84     s.add_argument("--min-severity", choices=list(SEVERITY_ORDER), default="LOW")
85     s.add_argument("--fail-on", choices=list(SEVERITY_ORDER), help="exit 1 if a finding at/above t
   his severity")
86     s.add_argument("--show-suppressed", action="store_true")
87     s.add_argument("--name", help="display name for reports")
88     s.add_argument("--meta", help="JSON file of metadata to embed in reports (e.g. fetch_metadata.
   json)")
89     s.set_defaults(func=cmd_scan)
90
91     e = sub.add_parser("evaluate", help="precision/recall on the bundled sample corpus")
92     e.add_argument("--vulnerable", default="samples/vulnerable")
93     e.add_argument("--benign", default="samples/benign")
94     e.add_argument("--out")
95     e.set_defaults(func=cmd_evaluate)
96
97     f = sub.add_parser("fetch-github", help="download a public GitHub repo zip (extract only, neve
   r run)")
98     f.add_argument("repo", help="owner/name")
99     f.add_argument("--ref", default="main", help="branch, tag or commit SHA (SHA = reproducible)")
100    f.add_argument("--dest", required=True)
101    f.set_defaults(func=cmd_fetch)
102
103    r = sub.add_parser("rules", help="list rules")
104    r.set_defaults(func=cmd_rules)
105    return p
106
107
108 def main(argv=None) -> int:
109     args = build_parser().parse_args(argv)
110     return args.func(args)
111
112
113 if __name__ == "__main__":
114     sys.exit(main())
115
```

```
"""pysast Streamlit dashboard.

Reads the JSON reports in outputs/ (no network needed) and lets you paste code
to scan it live. Pasted code is only parsed with ast.parse(); it is never run.

    streamlit run app.py
"""
import json
from pathlib import Path

import pandas as pd
import plotly.express as px
import streamlit as st

from src.reporters import to_sarif
from src.rules import RULES
from src.scanner import ScanResult, scan_source

ROOT = Path(__file__).parent
REPORTS = {
    "Deliberately vulnerable sample": ROOT / "outputs/reports/vulnerable.json",
    "Benign look-alike code": ROOT / "outputs/reports/benign.json",
    "pypa/sampleproject (real repo)": ROOT / "outputs/real/sampleproject.json",
    "psf/requests v2.32.3 (real repo, raw)": ROOT / "outputs/real/requests.json",
    "psf/requests v2.32.3 (real repo, triaged)": ROOT / "outputs/real/requests-triaged.json",
}
SEV_COLORS = {"HIGH": "#c62828", "MEDIUM": "#ef6c00", "LOW": "#607d8b"}

EXAMPLE = """import hashlib, pickle, subprocess, yaml

DB_PASSWORD = "fake-Pa55w0rd-demo"

def run(cur, user, host, blob, text):
    cur.execute(f"SELECT * FROM users WHERE name = '{user}'")
    subprocess.run("ping " + host, shell=True)
    data = pickle.loads(blob)
    cfg = yaml.load(text)
    return hashlib.md5(user.encode()).hexdigest()  # nosec
"""

st.set_page_config(page_title="pysast - Python SAST scanner", layout="wide")
st.title("pysast: AST-based Python security scanner")
st.caption("Syntactic pattern matching only: findings are leads to review, not proof of an "
           "exploitable vulnerability. Scanned code is parsed, never executed.")


@st.cache_data
def load_report(path: str) -> dict | None:
    p = Path(path)
    return json.loads(p.read_text()) if p.exists() else None


tab_overview, tab_findings, tab_live, tab_rules = st.tabs(
    ["Overview", "Findings", "Scan a snippet", "Rules"])

with tab_overview:
    rows = []
    for name, path in REPORTS.items():
        rep = load_report(str(path))
        if rep:
            s = rep["summary"]
            rows.append({"report": name, "files": s["files_scanned"], "lines": s["lines_scanned"],
                         "active findings": s["total"], "HIGH": s["by_severity"]["HIGH"],
                         "MEDIUM": s["by_severity"]["MEDIUM"], "LOW": s["by_severity"]["LOW"],
                         "suppressed": s["suppressed"], "scan time (s)": s["duration_s"]})
```

```
    if rows:
        df = pd.DataFrame(rows)
        st.dataframe(df, hide_index=True, width="stretch")
        long = df.melt(id_vars="report", value_vars=["HIGH", "MEDIUM", "LOW"],
                       var_name="severity", value_name="count")
        fig = px.bar(long, x="count", y="report", color="severity", orientation="h",
                     color_discrete_map=SEV_COLORS, title="Active findings by report and severity"
)
        st.plotly_chart(fig, width="stretch")
    else:
        st.info("No reports yet. Run: bash scripts/run_demo.sh")
    ev_path = ROOT / "outputs/evaluation.json"
    if ev_path.exists():
        ev = json.loads(ev_path.read_text())
        c1, c2, c3, c4 = st.columns(4)
        c1.metric("Planted issues", ev["expected"])
        c2.metric("Detected", ev["true_positives"])
        c3.metric("Benign false positives", len(ev["false_positives_benign"]))
        c4.metric("Recall on own corpus", f"{ev['recall']:.0%}")
        st.caption("The sample corpus was written alongside the rules, so these numbers show the "
                   "rules work as designed; they are not an independent benchmark.")

with tab_findings:
    choice = st.selectbox("Report", list(REPORTS))
    rep = load_report(str(REPORTS[choice]))
    if not rep:
        st.info("Report not generated yet. Run: bash scripts/run_demo.sh")
    else:
        meta = {k: v for k, v in (rep.get("metadata") or {}).items()
                if k in ("repo", "ref", "commit", "archive_sha256", "fetched_at")}
        if meta:
            st.json(meta)
        fdf = pd.DataFrame(rep["findings"])
        if fdf.empty:
            st.success("No findings in this report.")
        else:
            sev = st.multiselect("Severity", ["HIGH", "MEDIUM", "LOW"], default=["HIGH", "MEDIUM",
 "LOW"])
            show_sup = st.checkbox("Show suppressed findings", value=True)
            view = fdf[fdf["severity"].isin(sev)]
            if not show_sup:
                view = view[~view["suppressed"]]
            st.dataframe(view[["severity", "rule_id", "rule_name", "path", "line", "message",
                               "suppressed", "suppression_reason", "snippet"]],
                         hide_index=True, width="stretch")
            by_rule = view.groupby(["rule_id", "severity"]).size().reset_index(name="count")
            if not by_rule.empty:
                st.plotly_chart(px.bar(by_rule, x="rule_id", y="count", color="severity",
                                       color_discrete_map=SEV_COLORS, title="Findings by rule"),
                                width="stretch")

with tab_live:
    st.write("Paste Python code. It is parsed with `ast.parse()` and never executed.")
    code = st.text_area("Python source", EXAMPLE, height=260)
    if code.strip():
        findings = None
        try:
            findings = scan_source(code, "snippet.py")
        except SyntaxError as exc:
            st.error(f"Could not parse the code: {exc}")
        if findings is not None:
            active = [f for f in findings if not f.suppressed]
            st.metric("Findings", len(active), help=f"{len(findings) - len(active)} suppressed")
            for f in findings:
                label = f"{f.severity} {f.rule_id} {f.rule.name} - line {f.line}"
```

```
                with st.expander(label + (" (suppressed)" if f.suppressed else ""),
                                 expanded=not f.suppressed):
                    st.write(f.message)
                    st.code(f.snippet, language="python")
                    st.success("Fix: " + f.rule.fix)
            res = ScanResult("snippet", findings, 1, code.count("\n") + 1, [], 0.0)
            st.download_button("Download SARIF", json.dumps(to_sarif(res), indent=2),
                               file_name="snippet.sarif", mime="application/json")

with tab_rules:
    st.dataframe(pd.DataFrame([r.to_dict() for r in RULES.values()]), hide_index=True,
                 width="stretch")

```

## 07-python-sast-scanner/src/__init__.py

```
"""pysast - a small AST-based static application security testing (SAST) scanner for Python.

The scanner parses source with :func:`ast.parse` only. Scanned code is never imported,
compiled to bytecode, or executed.
"""
__version__ = "1.0.0"

```

## 07-python-sast-scanner/src/analyzer.py

```
"""AST analysis: one pass over a module's syntax tree, emitting Findings.

Everything here is *syntactic*. The analyzer looks at the shape of the code
(call names, argument kinds, literal values). It does not run the code, follow
values across functions or files, or prove that attacker input reaches a sink.
Expect false positives (a flagged pattern may be safe in context) and false
negatives (dangerous code written in a shape the rules do not recognise).
"""
from __future__ import annotations

import ast
import re
from typing import Iterable

from .findings import Finding
from .rules import RULES

# ---------------------------------------------------------------------------- #
# Heuristic vocabularies
# ---------------------------------------------------------------------------- #

# Variable / key / keyword names that suggest a secret value.
SECRET_NAME_RE = re.compile(
    r"(pass(word|wd|phrase)?|pwd|secret|token|api[_-]?key|apikey|access[_-]?key|"
    r"private[_-]?key|client[_-]?secret|auth[_-]?key|credential|signing[_-]?key)",
    re.IGNORECASE,
)
# Names that contain a secret word but usually hold metadata, not the secret itself.
SECRET_NAME_EXCLUDE_RE = re.compile(
    r"(_url|_uri|_file|_path|_dir|_env|_var|_name|_field|_header|_type|_len|_length|"
    r"_min|_max|_regex|_re|_pattern|_prompt|_label|_hint|_policy|_endpoint|_id|"
    r"_expir\w*|_ttl|_count|_param|_key_name|_reset\w*|^is_|^has_|^use_|^show_|^num_)$|"
    r"^(is|has|use|show|num|min|max|forgot|reset)_",
    re.IGNORECASE,
)
# Literal values that are obviously placeholders rather than real secrets.
PLACEHOLDER_RE = re.compile(
    r"^\s*$|^(x+|\*+|\.+|-+|#+|0+)$|"
    r"^(changeme|change[_-]me|password|passwd|secret|token|none|null|nil|todo|tbd|fixme|"
    r"example|dummy|placeholder|redacted|hidden|masked|test|testing|sample|default|"
    r"your[_-]?\w*|my[_-]?\w*|insert[_-]?\w*|replace[_-]?\w*|enter[_-]?\w*)$|"
    r"^<.*>$|^\$\{.*\}$|^\{\{.*\}\}$|^%\(.*\)s$|^\$[A-Z_][A-Z0-9_]*$|^\{[a-z_]*\}$",
    re.IGNORECASE,
)
MIN_SECRET_LEN = 6

SQL_SINK_METHODS = {"execute", "executemany", "executescript", "raw", "read_sql",
                    "read_sql_query", "mogrify"}
SQL_KEYWORD_RE = re.compile(r"\b(select|insert|update|delete|replace|create|drop|alter|"
                            r"where|from|into|values|union|order\s+by|group\s+by)\b",
                            re.IGNORECASE)

PICKLE_SINKS = {"pickle.loads", "pickle.load", "pickle.Unpickler", "_pickle.loads",
                "_pickle.load", "cPickle.loads", "cPickle.load", "dill.loads", "dill.load",
                "marshal.loads", "marshal.load", "jsonpickle.decode", "shelve.open",
                "pandas.read_pickle"}
SAFE_YAML_LOADERS = {"SafeLoader", "CSafeLoader", "BaseLoader", "CBaseLoader"}
WEAK_HASHES = {"md5", "sha1", "md4", "md2"}
SUBPROCESS_FUNCS = {"subprocess." + f for f in
                    ("call", "run", "Popen", "check_call", "check_output")}
OS_SHELL_FUNCS = {"os.system", "os.popen", "os.popen2", "os.popen3", "commands.getoutput",
                  "subprocess.getoutput", "subprocess.getstatusoutput"}


def is_placeholder(value: str) -> bool:
```

```
    """True when a string literal is obviously not a real secret."""
    return len(value) < MIN_SECRET_LEN or bool(PLACEHOLDER_RE.match(value))


def looks_like_secret_name(name: str) -> bool:
    return bool(SECRET_NAME_RE.search(name)) and not SECRET_NAME_EXCLUDE_RE.search(name)


def _const_str(node: ast.AST) -> str | None:
    if isinstance(node, ast.Constant) and isinstance(node.value, str):
        return node.value
    return None


def _target_name(node: ast.AST) -> str | None:
    """Name of an assignment target: x, self.x, cfg['x']."""
    if isinstance(node, ast.Name):
        return node.id
    if isinstance(node, ast.Attribute):
        return node.attr
    if isinstance(node, ast.Subscript):
        return _const_str(node.slice)
    return None


def _is_dynamic_string(node: ast.AST) -> bool:
    """True when an expression builds a string from non-literal parts.

    Recognised shapes: f-strings with placeholders, "..." + x, "..." % x and
    "...".format(x). Only literal-on-one-side shapes are matched; this is a
    syntactic check, not taint tracking.
    """
    if isinstance(node, ast.JoinedStr):
        return any(isinstance(v, ast.FormattedValue) for v in node.values)
    if isinstance(node, ast.BinOp) and isinstance(node.op, ast.Add):
        parts = _flatten_add(node)
        has_lit = any(_const_str(p) is not None or isinstance(p, ast.JoinedStr) for p in parts)
        has_dyn = any(_const_str(p) is None for p in parts)
        return has_lit and has_dyn
    if isinstance(node, ast.BinOp) and isinstance(node.op, ast.Mod):
        return _const_str(node.left) is not None
    if (isinstance(node, ast.Call) and isinstance(node.func, ast.Attribute)
            and node.func.attr == "format" and _const_str(node.func.value) is not None):
        return bool(node.args or node.keywords)
    return False


def _flatten_add(node: ast.AST) -> list[ast.AST]:
    if isinstance(node, ast.BinOp) and isinstance(node.op, ast.Add):
        return _flatten_add(node.left) + _flatten_add(node.right)
    return [node]


def _literal_text(node: ast.AST) -> str:
    """Concatenated literal fragments of a string-building expression (for SQL keyword check)."""
    out: list[str] = []
    for sub in ast.walk(node):
        s = _const_str(sub)
        if s:
            out.append(s)
    return " ".join(out)


class SecurityVisitor(ast.NodeVisitor):
    """Walks one module and collects findings.
```

```
    Import aliases are tracked so that ``import hashlib as h; h.md5()`` and
    ``from subprocess import run; run(cmd, shell=True)`` resolve to their
    canonical dotted names.
    """

    def __init__(self, path: str, source_lines: list[str]):
        self.path = path
        self.lines = source_lines
        self.findings: list[Finding] = []
        self.aliases: dict[str, str] = {}
        # Per-function map of local names assigned a dynamically built SQL string,
        # so `q = f"... {x}"; cur.execute(q)` is caught (same scope only).
        self._scopes: list[dict[str, ast.AST]] = [{}]

    # ------------------------------------------------------------------ utils
    def _snippet(self, line: int, redact: str | None = None) -> str:
        text = self.lines[line - 1].rstrip("\n") if 0 < line <= len(self.lines) else ""
        if redact:
            text = text.replace(redact, _mask(redact))
        return text.strip()[:240]

    def _add(self, rule_id: str, node: ast.AST, message: str, *, severity: str | None = None,
             confidence: str = "MEDIUM", redact: str | None = None) -> None:
        line = getattr(node, "lineno", 1)
        self.findings.append(Finding(
            rule_id=rule_id,
            severity=severity or RULES[rule_id].severity,
            path=self.path,
            line=line,
            column=getattr(node, "col_offset", 0) + 1,
            end_line=getattr(node, "end_lineno", line) or line,
            end_column=(getattr(node, "end_col_offset", 0) or 0) + 1,
            message=message,
            snippet=self._snippet(line, redact),
            confidence=confidence,
        ))

    def qualname(self, node: ast.AST) -> str | None:
        """Resolve a call target to a dotted name using the import alias table."""
        if isinstance(node, ast.Name):
            return self.aliases.get(node.id, node.id)
        if isinstance(node, ast.Attribute):
            base = self.qualname(node.value)
            return f"{base}.{node.attr}" if base else None
        return None

    # --------------------------------------------------------------- imports
    def visit_Import(self, node: ast.Import) -> None:
        for a in node.names:
            if a.asname:
                self.aliases[a.asname] = a.name
        self.generic_visit(node)

    def visit_ImportFrom(self, node: ast.ImportFrom) -> None:
        if node.module and node.level == 0:
            for a in node.names:
                self.aliases[a.asname or a.name] = f"{node.module}.{a.name}"
        self.generic_visit(node)

    # ---------------------------------------------------------------- scopes
    def _visit_scope(self, node: ast.AST) -> None:
        self._scopes.append({})
        self.generic_visit(node)
        self._scopes.pop()
```

```
196
197     visit_FunctionDef = _visit_scope
198     visit_AsyncFunctionDef = _visit_scope
199     visit_Lambda = _visit_scope
200
201     # ------------------------------------------------- PY001 credentials
202     def _check_secret(self, name: str | None, value: ast.AST, node: ast.AST, where: str) -> None:
203         lit = _const_str(value)
204         if name is None or lit is None or not looks_like_secret_name(name):
205             return
206         if is_placeholder(lit):
207             return
208         self._add("PY001", node, f"Hard-coded credential in {where} '{name}'", redact=lit,
209                   confidence="MEDIUM")
210
211     def visit_Assign(self, node: ast.Assign) -> None:
212         for t in node.targets:
213             targets = t.elts if isinstance(t, (ast.Tuple, ast.List)) else [t]
214             values = (node.value.elts if isinstance(node.value, (ast.Tuple, ast.List))
215                       and len(node.value.elts) == len(targets) else [node.value] * len(targets))
216             for tgt, val in zip(targets, values):
217                 self._check_secret(_target_name(tgt), val, node, "assignment to")
218                 if isinstance(tgt, ast.Name):
219                     self._track_sql(tgt.id, val)
220         self.generic_visit(node)
221
222     def visit_AnnAssign(self, node: ast.AnnAssign) -> None:
223         if node.value is not None:
224             self._check_secret(_target_name(node.target), node.value, node, "assignment to")
225             if isinstance(node.target, ast.Name):
226                 self._track_sql(node.target.id, node.value)
227         self.generic_visit(node)
228
229     def visit_Dict(self, node: ast.Dict) -> None:
230         for k, v in zip(node.keys, node.values):
231             if k is not None:
232                 self._check_secret(_const_str(k), v, v, "dict key")
233         self.generic_visit(node)
234
235     def visit_Compare(self, node: ast.Compare) -> None:
236         # if password == "s3cr3t-literal": ...
237         if len(node.ops) == 1 and isinstance(node.ops[0], (ast.Eq, ast.NotEq)):
238             left, right = node.left, node.comparators[0]
239             for a, b in ((left, right), (right, left)):
240                 self._check_secret(_target_name(a), b, node, "comparison with")
241         self.generic_visit(node)
242
243     def visit_arguments(self, node: ast.arguments) -> None:
244         # def connect(password="hunter2-literal"): default values of secret-named params
245         pos = node.posonlyargs + node.args
246         for arg, default in zip(pos[len(pos) - len(node.defaults):], node.defaults):
247             self._check_secret(arg.arg, default, default, "default value of parameter")
248         for arg, default in zip(node.kwonlyargs, node.kw_defaults):
249             if default is not None:
250                 self._check_secret(arg.arg, default, default, "default value of parameter")
251         self.generic_visit(node)
252
253     # ------------------------------------------------------ PY002 helpers
254     def _track_sql(self, name: str, value: ast.AST) -> None:
255         scope = self._scopes[-1]
256         if _is_dynamic_string(value):
257             scope[name] = value
258         else:
259             scope.pop(name, None)   # re-assigned to something else: forget it
260
```

```
    # ------------------------------------------------------------------ calls
    def visit_Call(self, node: ast.Call) -> None:
        q = self.qualname(node.func) or ""
        kw = {k.arg: k.value for k in node.keywords if k.arg}

        # keyword arguments named like secrets: connect(password="literal")
        for k in node.keywords:
            if k.arg:
                self._check_secret(k.arg, k.value, k.value, "keyword argument")

        # PY002: SQL built by formatting, passed to an execute-like method
        if isinstance(node.func, ast.Attribute) and node.func.attr in SQL_SINK_METHODS and node.ar
gs:
            arg = node.args[0]
            src = arg
            via = ""
            if isinstance(arg, ast.Name):
                src = self._scopes[-1].get(arg.id)
                via = f" (via variable '{arg.id}')"
            if src is not None and _is_dynamic_string(src) and SQL_KEYWORD_RE.search(_literal_text
(src)):
                self._add("PY002", node,
                          f"SQL string built with formatting/concatenation passed to "
                          f".{node.func.attr}(){via}; use bound parameters",
                          confidence="HIGH" if not via else "MEDIUM")

        # PY003: eval / exec (builtins only, not obj.eval())
        if isinstance(node.func, ast.Name) and q in ("eval", "exec", "builtins.eval", "builtins.ex
ec"):
            literal = bool(node.args) and _const_str(node.args[0]) is not None
            self._add("PY003", node,
                      f"Call to {node.func.id}() " + ("with a constant string" if literal
                                                      else "with a non-literal argument"),
                      severity="LOW" if literal else "HIGH",
                      confidence="HIGH")
        elif q in ("builtins.eval", "builtins.exec"):
            self._add("PY003", node, f"Call to {q}()", confidence="HIGH")

        # PY004: insecure deserialisation
        if q in PICKLE_SINKS:
            self._add("PY004", node, f"Insecure deserialisation via {q}()", confidence="HIGH")

        # PY005: yaml.load without a safe loader
        if q in ("yaml.load", "yaml.load_all"):
            loader = kw.get("Loader") or (node.args[1] if len(node.args) > 1 else None)
            loader_name = (self.qualname(loader) or "").split(".")[-1] if loader is not None else
""
            if loader_name not in SAFE_YAML_LOADERS:
                how = f"Loader={loader_name}" if loader_name else "no Loader argument"
                self._add("PY005", node, f"{q}() with {how}; use yaml.safe_load()",
                          confidence="HIGH")
        elif q in ("yaml.unsafe_load", "yaml.unsafe_load_all", "yaml.full_load", "yaml.full_load_a
ll"):
            self._add("PY005", node, f"{q}() can construct arbitrary objects",
                      severity="HIGH" if "unsafe" in q else "MEDIUM", confidence="HIGH")

        # PY006: weak hashes
        if q.startswith("hashlib.") or q.startswith("Crypto.Hash.") or q.startswith("cryptography.
"):
            algo = q.split(".")[-1].lower()
            if q == "hashlib.new" and node.args:
                algo = (_const_str(node.args[0]) or "").lower()
            if algo in WEAK_HASHES or algo in ("md5.new", "sha.new"):
                not_security = (isinstance(kw.get("usedforsecurity"), ast.Constant)
                                and kw["usedforsecurity"].value is False)
```

```
                if not not_security:
                    self._add("PY006", node, f"Weak hash algorithm {algo.upper()} ({q})",
                              confidence="MEDIUM")
        if q.startswith("Crypto.Hash.MD5.") or q.startswith("Crypto.Hash.SHA1."):
            self._add("PY006", node, f"Weak hash algorithm via {q}", confidence="MEDIUM")

        # PY007: subprocess with shell=True
        if q in SUBPROCESS_FUNCS:
            shell = kw.get("shell")
            if isinstance(shell, ast.Constant) and shell.value is True:
                cmd = node.args[0] if node.args else kw.get("args")
                literal = cmd is not None and _const_str(cmd) is not None
                self._add("PY007", node,
                          f"{q}() with shell=True and "
                          + ("a constant command" if literal else "a non-literal command"),
                          severity="MEDIUM" if literal else "HIGH",
                          confidence="HIGH")
            elif shell is not None and not isinstance(shell, ast.Constant):
                self._add("PY007", node, f"{q}() with a non-constant shell= argument",
                          severity="MEDIUM", confidence="LOW")

        # PY008: os.system / os.popen
        if q in OS_SHELL_FUNCS:
            literal = bool(node.args) and _const_str(node.args[0]) is not None
            self._add("PY008", node, f"{q}() runs its argument through a shell",
                      severity="LOW" if literal else "MEDIUM", confidence="HIGH")

        self.generic_visit(node)


def _mask(secret: str) -> str:
    """Redact a secret for reports: keep 2 leading chars, mask the rest."""
    if len(secret) <= 4:
        return "*" * len(secret)
    return secret[:2] + "*" * min(len(secret) - 2, 12)


def analyze_source(source: str, path: str = "<string>") -> list[Finding]:
    """Parse source text (never executes it) and return raw, unsuppressed findings.

    Raises SyntaxError for code that does not parse.
    """
    tree = ast.parse(source, filename=path, type_comments=False)
    visitor = SecurityVisitor(path, source.splitlines())
    visitor.visit(tree)
    # Deduplicate identical (rule, line, col) findings from overlapping checks.
    seen: set[tuple] = set()
    out: list[Finding] = []
    for f in visitor.findings:
        key = (f.rule_id, f.line, f.column)
        if key not in seen:
            seen.add(key)
            out.append(f)
    return out


def rules_in(findings: Iterable[Finding]) -> set[str]:
    return {f.rule_id for f in findings}

```

## 07-python-sast-scanner/src/evaluate.py

```
"""Measure the scanner against ground-truth markers in the sample corpus.

Each deliberately vulnerable line in samples/vulnerable carries a trailing
comment `# expect: PY00X` (several ids separated by commas). Every finding in
samples/benign is a false positive by construction.
"""
from __future__ import annotations

import re
from pathlib import Path

from .scanner import scan_path

EXPECT_RE = re.compile(r"#\s*expect:\s*([A-Z0-9, ]+)")


def expected_labels(root: Path) -> set[tuple[str, int, str]]:
    labels = set()
    for p in sorted(root.rglob("*.py")):
        rel = p.relative_to(root).as_posix()
        for i, line in enumerate(p.read_text().splitlines(), 1):
            m = EXPECT_RE.search(line)
            if m:
                for rid in m.group(1).split(","):
                    if rid.strip():
                        labels.add((rel, i, rid.strip()))
    return labels


def evaluate(vulnerable_dir: str | Path, benign_dir: str | Path) -> dict:
    vroot, broot = Path(vulnerable_dir), Path(benign_dir)
    expected = expected_labels(vroot)
    vres = scan_path(vroot)
    got = {(f.path, f.line, f.rule_id) for f in vres.active}
    tp = expected & got
    fp_v = got - expected
    fn = expected - got
    bres = scan_path(broot)
    fp_b = [(f.path, f.line, f.rule_id) for f in bres.active]
    fp = len(fp_v) + len(fp_b)
    precision = len(tp) / (len(tp) + fp) if (tp or fp) else 1.0
    recall = len(tp) / len(expected) if expected else 1.0
    per_rule: dict[str, dict] = {}
    for _, _, rid in expected:
        per_rule.setdefault(rid, {"expected": 0, "detected": 0})["expected"] += 1
    for _, _, rid in tp:
        per_rule[rid]["detected"] += 1
    return {
        "expected": len(expected), "true_positives": len(tp),
        "false_negatives": sorted(fn), "false_positives_vulnerable": sorted(fp_v),
        "false_positives_benign": sorted(fp_b),
        "benign_files": bres.files_scanned, "benign_lines": bres.lines_scanned,
        "vulnerable_files": vres.files_scanned, "vulnerable_lines": vres.lines_scanned,
        "suppressed_in_vulnerable": vres.counts()["suppressed"],
        "precision": round(precision, 4), "recall": round(recall, 4), "per_rule": per_rule,
    }

```

```
"""Download a public GitHub repository as a zip archive and extract it safely.

The archive is only *extracted*: no setup.py, build hook or test is ever run.
Extraction guards against zip-slip (paths escaping the destination), absolute
paths, symlink entries and oversized archives.
"""
from __future__ import annotations

import hashlib
import io
import json
import stat
import zipfile
from datetime import datetime, timezone
from pathlib import Path

import requests

MAX_ZIP_BYTES = 50 * 1024 * 1024
MAX_UNCOMPRESSED = 200 * 1024 * 1024
UA = {"User-Agent": "pysast-educational-scanner/1.0"}


def resolve_commit(repo: str, ref: str, timeout: float = 20) -> str | None:
    """Ask the GitHub API which commit a branch/tag points to (None if rate-limited/offline)."""
    try:
        r = requests.get(f"https://api.github.com/repos/{repo}/commits/{ref}", headers=UA, timeout
=timeout)
        if r.ok:
            return r.json().get("sha")
    except requests.RequestException:
        pass
    return None


def safe_extract(data: bytes, dest: Path) -> int:
    """Extract zip bytes into dest; returns the number of files written."""
    dest = dest.resolve()
    dest.mkdir(parents=True, exist_ok=True)
    n = 0
    with zipfile.ZipFile(io.BytesIO(data)) as zf:
        total = sum(i.file_size for i in zf.infolist())
        if total > MAX_UNCOMPRESSED:
            raise ValueError(f"archive expands to {total} bytes (> {MAX_UNCOMPRESSED})")
        for info in zf.infolist():
            name = info.filename
            mode = (info.external_attr >> 16) & 0o170000
            if mode == stat.S_IFLNK:
                continue                                # never materialise symlinks
            target = (dest / name).resolve()
            if not str(target).startswith(str(dest) + "/") and target != dest:
                raise ValueError(f"unsafe path in archive: {name!r}")
            if info.is_dir():
                target.mkdir(parents=True, exist_ok=True)
                continue
            target.parent.mkdir(parents=True, exist_ok=True)
            with zf.open(info) as src, open(target, "wb") as out:
                out.write(src.read())
            n += 1
    return n


def fetch_github(repo: str, ref: str, dest: str | Path, timeout: float = 60) -> dict:
    """Download https://github.com/<repo>/archive/<ref>.zip into dest and return metadata.

```

```
    `ref` may be a branch, tag or full commit SHA; pass a SHA for a reproducible scan.
    """
    url = f"https://codeload.github.com/{repo}/zip/{ref}"
    with requests.get(url, headers=UA, timeout=timeout, stream=True) as r:
        r.raise_for_status()
        buf = io.BytesIO()
        for chunk in r.iter_content(64 * 1024):
            buf.write(chunk)
            if buf.tell() > MAX_ZIP_BYTES:
                raise ValueError("archive larger than 50 MB; refusing")
    data = buf.getvalue()
    dest = Path(dest)
    n = safe_extract(data, dest)
    # GitHub archives contain one top-level folder "<repo>-<ref>/"
    tops = [p for p in dest.iterdir() if p.is_dir()]
    meta = {
        "repo": repo,
        "ref": ref,
        "commit": ref if len(ref) == 40 else resolve_commit(repo, ref),
        "archive_url": url,
        "archive_sha256": hashlib.sha256(data).hexdigest(),
        "archive_bytes": len(data),
        "files_extracted": n,
        "source_dir": str(tops[0]) if len(tops) == 1 else str(dest),
        "fetched_at": datetime.now(timezone.utc).replace(microsecond=0).isoformat(),
    }
    (dest / "fetch_metadata.json").write_text(json.dumps(meta, indent=2))
    return meta

```

## 07-python-sast-scanner/src/findings.py

```python
"""Finding model shared by the analyzer, scanner and reporters."""
from __future__ import annotations

import hashlib
from dataclasses import dataclass, field, asdict

from .rules import RULES, SEVERITY_ORDER


@dataclass
class Finding:
    rule_id: str
    severity: str
    path: str               # path relative to the scan root, POSIX separators
    line: int
    column: int             # 1-based column (SARIF convention)
    end_line: int
    end_column: int
    message: str
    snippet: str = ""       # the offending source line (credentials are redacted)
    confidence: str = "MEDIUM"
    suppressed: bool = False
    suppression_reason: str = ""
    fingerprint: str = field(default="")

    def __post_init__(self) -> None:
        if not self.fingerprint:
            # Stable across runs and unaffected by line shifts elsewhere in the file:
            # rule + file + normalised (whitespace-collapsed) source line.
            norm = " ".join(self.snippet.split())
            raw = f"{self.rule_id}|{self.path}|{norm}".encode()
            self.fingerprint = hashlib.sha256(raw).hexdigest()[:16]

    @property
    def rule(self):
        return RULES[self.rule_id]

    def sort_key(self):
        return (-SEVERITY_ORDER[self.severity], self.path, self.line, self.column, self.rule_id)

    def to_dict(self) -> dict:
        d = asdict(self)
        d["rule_name"] = self.rule.name
        d["cwe"] = self.rule.cwe
        d["fix"] = self.rule.fix
        return d

```

```
"""Report writers: JSON, SARIF 2.1.0 and a single self-contained HTML page."""
from __future__ import annotations

import datetime as _dt
import json
from pathlib import Path

from jinja2 import Environment, select_autoescape

from . import __version__
from .rules import RULES, SARIF_LEVEL, SECURITY_SEVERITY
from .scanner import ScanResult

TOOL_NAME = "pysast"
TOOL_URI = "https://github.com/dharma/python-sast-scanner"
SARIF_SCHEMA = "https://json.schemastore.org/sarif-2.1.0.json"


def _now() -> str:
    return _dt.datetime.now(_dt.timezone.utc).replace(microsecond=0).isoformat()


# ---------------------------------------------------------------------- JSON
def to_json(result: ScanResult) -> dict:
    return {
        "tool": {"name": TOOL_NAME, "version": __version__},
        "generated_at": _now(),
        "root": Path(result.root).name,
        "metadata": result.metadata,
        "summary": {"files_scanned": result.files_scanned, "lines_scanned": result.lines_scanned,
                    "duration_s": result.duration_s, **result.counts(),
                    "parse_errors": len(result.errors)},
        "findings": [f.to_dict() for f in result.findings],
        "errors": result.errors,
        "disclaimer": "Syntactic pattern matches only; no data-flow proof. Review each finding.",
    }


# ---------------------------------------------------------------------- SARIF
def to_sarif(result: ScanResult) -> dict:
    """Build a SARIF 2.1.0 log. Suppressed findings carry a `suppressions` entry so
    viewers (e.g. GitHub code scanning) show them as dismissed rather than dropping them."""
    rule_ids = list(RULES)
    rules = []
    for rid in rule_ids:
        r = RULES[rid]
        rules.append({
            "id": r.id,
            "name": r.name,
            "shortDescription": {"text": r.short},
            "fullDescription": {"text": f"{r.short}. {r.cwe}."},
            "help": {"text": r.fix, "markdown": f"**Fix:** {r.fix}"},
            "defaultConfiguration": {"level": SARIF_LEVEL[r.severity]},
            "properties": {"tags": ["security", r.cwe], "security-severity": SECURITY_SEVERITY[r.s
everity],
                           "precision": "medium"},
        })
    results = []
    for f in result.findings:
        res = {
            "ruleId": f.rule_id,
            "ruleIndex": rule_ids.index(f.rule_id),
            "level": SARIF_LEVEL[f.severity],
            "message": {"text": f"{f.message}. Fix: {RULES[f.rule_id].fix}"},
            "locations": [{"physicalLocation": {
```

```
                "artifactLocation": {"uri": f.path, "uriBaseId": "SRCROOT"},
                "region": {"startLine": f.line, "startColumn": f.column,
                           "endLine": f.end_line, "endColumn": max(f.end_column, 1),
                           "snippet": {"text": f.snippet}},
            }}],
            "partialFingerprints": {"pysast/v1": f.fingerprint},
            "properties": {"severity": f.severity, "confidence": f.confidence,
                           "security-severity": SECURITY_SEVERITY[f.severity]},
        }
        if f.suppressed:
            res["suppressions"] = [{"kind": "inSource" if f.suppression_reason.startswith("inline")
                                    else "external", "justification": f.suppression_reason}]
        results.append(res)
    return {
        "$schema": SARIF_SCHEMA,
        "version": "2.1.0",
        "runs": [{
            "tool": {"driver": {"name": TOOL_NAME, "version": __version__,
                                "informationUri": TOOL_URI, "rules": rules}},
            # SRCROOT is left for the consumer to bind (e.g. the repo checkout in CI), so
            # reports never embed the absolute path of the machine that ran the scan.
            "invocations": [{"executionSuccessful": True, "endTimeUtc": _now().replace("+00:00", "Z"),
                             "toolExecutionNotifications": [
                                 {"level": "warning", "message": {"text": f"{e['path']}: {e['error']}"}}
                                 for e in result.errors]}],
            "results": results,
            "properties": {"filesScanned": result.files_scanned, "linesScanned": result.lines_scanned},
        }],
    }


# ---------------------------------------------------------------------- HTML
_HTML = """<!doctype html>
<html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>pysast report - {{ name }}</title>
<style>
 body{font-family:-apple-system,Segoe UI,Roboto,Helvetica,Arial,sans-serif;margin:0;background:#f6f7f9;color:#1d2330}
 header{background:#1d2330;color:#fff;padding:18px 24px} header h1{margin:0;font-size:20px}
 header p{margin:4px 0 0;color:#b8c0d0;font-size:13px}
 main{padding:18px 24px;max-width:1100px}
 .cards{display:flex;gap:12px;flex-wrap:wrap;margin-bottom:18px}
 .card{background:#fff;border-radius:8px;padding:12px 16px;min-width:120px;box-shadow:0 1px 2px #0001}
 .card b{display:block;font-size:24px} .card span{font-size:12px;color:#5b6475}
 .sev{font-weight:700;padding:2px 8px;border-radius:10px;font-size:11px;color:#fff}
 .HIGH{background:#c62828}.MEDIUM{background:#ef6c00}.LOW{background:#607d8b}
 .f{background:#fff;border-radius:8px;padding:12px 16px;margin:10px 0;box-shadow:0 1px 2px #0001;border-left:4px solid #c62828}
 .f.MEDIUMb{border-left-color:#ef6c00}.f.LOWb{border-left-color:#607d8b}.f.sup{opacity:.55}
 .loc{font-family:ui-monospace,Menlo,Consolas,monospace;font-size:13px;color:#37474f}
 pre{background:#263238;color:#eceff1;padding:8px 10px;border-radius:6px;overflow-x:auto;font-size:12.5px;margin:8px 0}
 .fix{font-size:13px;color:#2e7d32} table{border-collapse:collapse;background:#fff;font-size:13px}
 td,th{border:1px solid #dde1e7;padding:5px 9px;text-align:left} .note{font-size:12px;color:#5b6475}
</style></head><body>
<header><h1>Static analysis report: {{ name }}</h1>
<p>pysast {{ version }} &middot; generated {{ generated }} &middot; {{ files }} files, {{ lines }} lines in {{ duration }} s</p></header>
```

```
<main>
<div class="cards">
 <div class="card"><b>{{ counts.total }}</b><span>active findings</span></div>
 <div class="card"><b>{{ counts.by_severity.HIGH }}</b><span>high</span></div>
 <div class="card"><b>{{ counts.by_severity.MEDIUM }}</b><span>medium</span></div>
 <div class="card"><b>{{ counts.by_severity.LOW }}</b><span>low</span></div>
 <div class="card"><b>{{ counts.suppressed }}</b><span>suppressed</span></div>
</div>
{% if metadata %}<p class="note">{% for k, v in metadata.items() %}<b>{{ k }}</b>: {{ v }} &n
bsp; {% endfor %}</p>{% endif %}
<h2>Findings by rule</h2>
<table><tr><th>Rule</th><th>Name</th><th>CWE</th><th>Count</th></tr>
{% for rid, n in counts.by_rule.items() %}<tr><td>{{ rid }}</td><td>{{ rules[rid].name }}</td><td>
{{ rules[rid].cwe }}</td><td>{{ n }}</td></tr>{% endfor %}
</table>
<h2>Findings</h2>
{% if not findings %}<p>No findings.</p>{% endif %}
{% for f in findings %}
<div class="f {{ f.severity }}b {% if f.suppressed %}sup{% endif %}">
 <span class="sev {{ f.severity }}">{{ f.severity }}</span> <b>{{ f.rule_id }} {{ rules[f.rule_id]
.name }}</b>
 <span class="note">{{ rules[f.rule_id].cwe }} &middot; confidence {{ f.confidence }}{% if f.suppr
essed %} &middot; SUPPRESSED ({{ f.suppression_reason }}){% endif %}</span>
 <div class="loc">{{ f.path }}:{{ f.line }}:{{ f.column }}</div>
 <div>{{ f.message }}</div>
 <pre>{{ f.snippet }}</pre>
 <div class="fix">Fix: {{ rules[f.rule_id].fix }}</div>
</div>
{% endfor %}
{% if errors %}<h2>Skipped files</h2><ul>{% for e in errors %}<li class="loc">{{ e.path }}: {{ e.e
rror }}</li>{% endfor %}</ul>{% endif %}
<p class="note">These are syntactic pattern matches. The scanner does not prove that untrusted dat
a reaches
a sink, so some findings will be false positives and some real issues will be missed. Review each
one.</p>
</main></body></html>
"""

_env = Environment(autoescape=select_autoescape(default=True, default_for_string=True))


def to_html(result: ScanResult) -> str:
    """Self-contained HTML (inline CSS, no JavaScript, no external assets).
    Autoescaping is on, so code snippets like `<script>` are rendered inert."""
    tpl = _env.from_string(_HTML)
    return tpl.render(name=result.metadata.get("name") or Path(result.root).name,
                      version=__version__, generated=_now(), files=result.files_scanned,
                      lines=result.lines_scanned, duration=result.duration_s,
                      counts=result.counts(), rules=RULES, findings=result.findings,
                      errors=result.errors, metadata=result.metadata)


def to_text(result: ScanResult, show_suppressed: bool = False) -> str:
    out = []
    for f in result.findings:
        if f.suppressed and not show_suppressed:
            continue
        tag = " [suppressed]" if f.suppressed else ""
        out.append(f"{f.path}:{f.line}:{f.column}: {f.severity:<6} {f.rule_id} "
                   f"{RULES[f.rule_id].name}: {f.message}{tag}\n    {f.snippet}")
    c = result.counts()
    out.append(f"\n{result.files_scanned} files, {result.lines_scanned} lines scanned in "
               f"{result.duration_s}s | {c['total']} findings (HIGH {c['by_severity']['HIGH']}, "
               f"MEDIUM {c['by_severity']['MEDIUM']}, LOW {c['by_severity']['LOW']}), "
               f"{c['suppressed']} suppressed, {len(result.errors)} skipped")
```

```
    return "\n".join(out)


def write_reports(result: ScanResult, out_base: str | Path, formats=("json", "sarif", "html")) -> list[Path]:
    base = Path(out_base)
    base.parent.mkdir(parents=True, exist_ok=True)
    written = []
    for fmt in formats:
        if fmt == "json":
            p = base.with_suffix(".json"); p.write_text(json.dumps(to_json(result), indent=2))
        elif fmt == "sarif":
            p = base.with_suffix(".sarif"); p.write_text(json.dumps(to_sarif(result), indent=2))
        elif fmt == "html":
            p = base.with_suffix(".html"); p.write_text(to_html(result), encoding="utf-8")
        else:
            raise ValueError(f"unknown format {fmt}")
        written.append(p)
    return written

```

```python
"""Rule catalogue: every rule the scanner can emit, with severity, CWE and fix guidance."""
from __future__ import annotations

from dataclasses import dataclass, asdict

# Severity order used for sorting and --fail-on thresholds.
SEVERITY_ORDER = {"LOW": 1, "MEDIUM": 2, "HIGH": 3}
# SARIF "level" for each severity, and a numeric score GitHub code scanning understands.
SARIF_LEVEL = {"HIGH": "error", "MEDIUM": "warning", "LOW": "note"}
SECURITY_SEVERITY = {"HIGH": "8.0", "MEDIUM": "5.5", "LOW": "3.0"}


@dataclass(frozen=True)
class Rule:
    id: str
    name: str
    severity: str          # default severity; some checks adjust it per finding
    cwe: str
    short: str             # one-line description
    fix: str               # remediation guidance shown in every report

    def to_dict(self) -> dict:
        return asdict(self)


RULES: dict[str, Rule] = {r.id: r for r in [
    Rule("PY001", "hardcoded-credential", "HIGH", "CWE-798",
         "Hard-coded credential assigned to a secret-looking name",
         "Load secrets from environment variables or a secret manager (os.environ, Vault, "
         "AWS Secrets Manager) and rotate any value that was committed."),
    Rule("PY002", "sql-string-formatting", "HIGH", "CWE-89",
         "SQL query built with string formatting/concatenation and passed to execute()",
         "Use parameterised queries: cursor.execute(\"... WHERE id = ?\", (user_id,)) "
         "or the ORM's bound parameters; never interpolate input into SQL text."),
    Rule("PY003", "eval-exec", "HIGH", "CWE-95",
         "Use of eval()/exec() can execute arbitrary code",
         "Remove dynamic evaluation. For data use ast.literal_eval() or json.loads(); "
         "for dispatch use an explicit dict of allowed functions."),
    Rule("PY004", "unsafe-pickle", "HIGH", "CWE-502",
         "Deserialising with pickle/dill/marshal can execute attacker-controlled code",
         "Only unpickle data you produced and authenticated (e.g. HMAC); prefer JSON, "
         "msgpack or protobuf for untrusted input."),
    Rule("PY005", "unsafe-yaml-load", "HIGH", "CWE-502",
         "yaml.load() without SafeLoader can construct arbitrary Python objects",
         "Use yaml.safe_load(data) or yaml.load(data, Loader=yaml.SafeLoader)."),
    Rule("PY006", "weak-hash", "MEDIUM", "CWE-328",
         "MD5/SHA-1 are broken for collision resistance",
         "Use hashlib.sha256()/sha3_256() for integrity, and a password KDF "
         "(argon2, bcrypt, scrypt, PBKDF2) for passwords. If the hash is not security "
         "relevant, pass usedforsecurity=False to document that."),
    Rule("PY007", "subprocess-shell-true", "HIGH", "CWE-78",
         "subprocess call with shell=True enables shell command injection",
         "Pass the command as a list with shell=False (the default); use shlex.quote() "
         "only as a last resort."),
    Rule("PY008", "os-system-shell", "MEDIUM", "CWE-78",
         "os.system()/os.popen() always run through a shell",
         "Replace with subprocess.run([...], check=True) using an argument list."),
]}


def get_rule(rule_id: str) -> Rule:
    return RULES[rule_id]

```

```
"""File-system scanning: discovery, parsing, inline suppressions and the allowlist.

Safety: files are read as text and handed to ast.parse(). Nothing is imported,
executed, or compiled to bytecode; symlinks are not followed; oversized files and
files that do not parse are skipped and reported as errors.
"""
from __future__ import annotations

import fnmatch
import io
import json
import os
import re
import time
import tokenize
from dataclasses import dataclass, field
from pathlib import Path

from .analyzer import analyze_source
from .findings import Finding
from .rules import RULES, SEVERITY_ORDER

DEFAULT_EXCLUDES = [".git", ".hg", ".svn", ".tox", ".nox", ".venv", "venv", "env",
                    "node_modules", "__pycache__", "build", "dist", ".eggs", "site-packages"]
MAX_FILE_BYTES = 1_000_000

# Inline suppression comments:
#   # nosec                    -> suppress every rule on this line
#   # sast: ignore             -> same
#   # sast: ignore[PY006,PY001] reason text  -> only the listed rules
SUPPRESS_RE = re.compile(r"#\s*(?:nosec\b|sast:\s*ignore(?:\[(?P<ids>[A-Z0-9_, ]+)\])?)(?P<reason>.*)",
                         re.IGNORECASE)


@dataclass
class Allowlist:
    """Project-level suppression configuration (YAML or JSON).

    Example (.sast-allowlist.yml)::

        disable_rules: [PY008]
        exclude_paths: ["tests/*", "docs/*"]
        suppress:
          - rule: PY006
            path: "legacy/*.py"
            reason: "MD5 used only as a cache key"
          - fingerprint: 3f2a9c0d1b7e4a55
            reason: "reviewed 2026-10-27"
    """
    disable_rules: set[str] = field(default_factory=set)
    exclude_paths: list[str] = field(default_factory=list)
    suppress: list[dict] = field(default_factory=list)

    @classmethod
    def load(cls, path: str | os.PathLike | None) -> "Allowlist":
        if not path:
            return cls()
        text = Path(path).read_text(encoding="utf-8")
        if str(path).endswith(".json"):
            data = json.loads(text)
        else:
            import yaml  # safe_load only: the allowlist itself is untrusted input
            data = yaml.safe_load(text) or {}
        unknown = set(data.get("disable_rules", [])) - set(RULES)
```

```
        if unknown:
            raise ValueError(f"Unknown rule id(s) in allowlist: {sorted(unknown)}")
        return cls(set(data.get("disable_rules", [])), list(data.get("exclude_paths", [])),
                   list(data.get("suppress", [])))

    def excludes(self, rel: str) -> bool:
        return any(fnmatch.fnmatch(rel, pat) for pat in self.exclude_paths)

    def match(self, f: Finding) -> str | None:
        """Return a suppression reason if an allowlist entry covers this finding."""
        for entry in self.suppress:
            if "fingerprint" in entry and entry["fingerprint"] != f.fingerprint:
                continue
            if "rule" in entry and entry["rule"] != f.rule_id:
                continue
            if "path" in entry and not fnmatch.fnmatch(f.path, entry["path"]):
                continue
            if not any(k in entry for k in ("fingerprint", "rule", "path")):
                continue
            return "allowlist: " + str(entry.get("reason", "no reason given"))
        return None


@dataclass
class ScanResult:
    root: str
    findings: list[Finding]
    files_scanned: int
    lines_scanned: int
    errors: list[dict]
    duration_s: float
    metadata: dict = field(default_factory=dict)

    @property
    def active(self) -> list[Finding]:
        return [f for f in self.findings if not f.suppressed]

    def counts(self) -> dict:
        by_sev = {s: 0 for s in ("HIGH", "MEDIUM", "LOW")}
        by_rule = {r: 0 for r in RULES}
        for f in self.active:
            by_sev[f.severity] += 1
            by_rule[f.rule_id] += 1
        return {"total": len(self.active), "suppressed": len(self.findings) - len(self.active),
                "by_severity": by_sev, "by_rule": by_rule}


def suppressions_by_line(source: str) -> dict[int, tuple[set[str] | None, str]]:
    """Map line number -> (rule ids or None for all, reason) from comment tokens.

    tokenize is used (not a regex over raw text) so '# nosec' inside a string
    literal is not mistaken for a suppression.
    """
    out: dict[int, tuple[set[str] | None, str]] = {}
    try:
        for tok in tokenize.generate_tokens(io.StringIO(source).readline):
            if tok.type == tokenize.COMMENT:
                m = SUPPRESS_RE.search(tok.string)
                if m:
                    ids = m.group("ids")
                    rules = {i.strip().upper() for i in ids.split(",") if i.strip()} if ids else N
one
                    out[tok.start[0]] = (rules, m.group("reason").strip(" -:"))
    except (tokenize.TokenError, IndentationError, SyntaxError):
        pass
```

```
    return out


def apply_suppressions(findings: list[Finding], source: str, allow: Allowlist) -> list[Finding]:
    inline = suppressions_by_line(source)
    kept = []
    for f in findings:
        if f.rule_id in allow.disable_rules:
            continue                                  # disabled rules are not reported at all
        hit = None
        for ln in range(f.line, f.end_line + 1):      # comment may sit on any line of the call
            if ln in inline:
                rules, reason = inline[ln]
                if rules is None or f.rule_id in rules:
                    hit = "inline: " + (reason or "suppressed by comment")
                    break
        hit = hit or allow.match(f)
        if hit:
            f.suppressed, f.suppression_reason = True, hit
        kept.append(f)
    return kept


def scan_source(source: str, path: str = "<string>", allow: Allowlist | None = None) -> list[Findi
ng]:
    """Analyse one source string and apply suppressions."""
    allow = allow or Allowlist()
    return apply_suppressions(analyze_source(source, path), source, allow)


def iter_python_files(root: Path, allow: Allowlist, excludes=DEFAULT_EXCLUDES):
    """Yield (absolute path, relative posix path) for .py files under root.

    os.walk(followlinks=False) plus an is_symlink() check means a symlink can
    never lead the scanner outside the tree being scanned.
    """
    if root.is_file():
        yield root, root.name
        return
    for dirpath, dirnames, filenames in os.walk(root, followlinks=False):
        dirnames[:] = sorted(d for d in dirnames if d not in excludes and not d.endswith(".egg-inf
o"))
        for name in sorted(filenames):
            if not name.endswith(".py"):
                continue
            p = Path(dirpath) / name
            if p.is_symlink():
                continue
            rel = p.relative_to(root).as_posix()
            if allow.excludes(rel):
                continue
            yield p, rel


def scan_path(target: str | os.PathLike, allow: Allowlist | None = None,
              min_severity: str = "LOW") -> ScanResult:
    root = Path(target).resolve()
    allow = allow or Allowlist()
    t0 = time.perf_counter()
    findings: list[Finding] = []
    errors: list[dict] = []
    files = lines = 0
    for p, rel in iter_python_files(root, allow):
        try:
            if p.stat().st_size > MAX_FILE_BYTES:
```

```
                errors.append({"path": rel, "error": "skipped: file larger than 1 MB"})
                continue
            raw = p.read_bytes()
            source = raw.decode("utf-8")
        except (OSError, UnicodeDecodeError) as exc:
            errors.append({"path": rel, "error": f"unreadable: {exc.__class__.__name__}"})
            continue
        try:
            found = scan_source(source, rel, allow)
        except (SyntaxError, ValueError, RecursionError) as exc:
            errors.append({"path": rel, "error": f"parse error: {exc.__class__.__name__}: {exc}"})
            continue
        files += 1
        lines += source.count("\n") + (0 if source.endswith("\n") or not source else 1)
        findings.extend(f for f in found
                        if SEVERITY_ORDER[f.severity] >= SEVERITY_ORDER[min_severity])
    findings.sort(key=Finding.sort_key)
    return ScanResult(str(root), findings, files, lines, errors,
                      round(time.perf_counter() - t0, 4))

```

## 07-python-sast-scanner/tests/conftest.py

```
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

```

## 07-python-sast-scanner/tests/test_app.py

```python
"""Streamlit dashboard smoke test (headless, no server)."""
from pathlib import Path

from streamlit.testing.v1 import AppTest

ROOT = Path(__file__).resolve().parents[1]


def test_dashboard_runs_and_scans_example_snippet():
    at = AppTest.from_file(str(ROOT / "app.py"), default_timeout=120).run()
    assert not at.exception
    assert len(at.tabs) == 4
    # the live-scan tab scans the bundled example: 5 active findings + 1 suppressed
    findings_metric = next(m for m in at.metric if m.label == "Findings")
    assert findings_metric.value == "5"


def test_dashboard_reports_parse_error_gracefully():
    at = AppTest.from_file(str(ROOT / "app.py"), default_timeout=120).run()
    at.text_area[0].input("def broken(:\n").run()
    assert not at.exception
    assert any("Could not parse" in e.value for e in at.error)

```

## 07-python-sast-scanner/tests/test_corpus_and_cli.py

```python
"""End-to-end: bundled corpus metrics and the command-line interface."""
import json
import subprocess
import sys
from pathlib import Path

from src.evaluate import evaluate

ROOT = Path(__file__).resolve().parents[1]


def run_cli(*args):
    return subprocess.run([sys.executable, "cli.py", *args], cwd=ROOT, capture_output=True,
                          text=True, timeout=60)


def test_corpus_detects_every_planted_issue_with_no_benign_false_positives():
    r = evaluate(ROOT / "samples/vulnerable", ROOT / "samples/benign")
    assert r["expected"] >= 20
    assert r["false_negatives"] == [] and r["recall"] == 1.0
    assert r["false_positives_benign"] == [] and r["false_positives_vulnerable"] == []
    assert set(r["per_rule"]) == {f"PY00{i}" for i in range(1, 9)}


def test_cli_help_and_rules():
    assert run_cli("--help").returncode == 0
    out = run_cli("rules").stdout
    assert all(f"PY00{i}" in out for i in range(1, 9))


def test_cli_scan_writes_reports_and_fail_on(tmp_path):
    base = tmp_path / "rep"
    p = run_cli("scan", "samples/vulnerable", "--out", str(base), "--fail-on", "HIGH")
    assert p.returncode == 1                    # HIGH findings present -> CI gate fails
    for ext in (".json", ".sarif", ".html"):
        assert base.with_suffix(ext).exists()
    data = json.loads(base.with_suffix(".json").read_text())
    assert data["summary"]["total"] == 23


def test_cli_scan_benign_passes_gate():
    p = run_cli("scan", "samples/benign", "--fail-on", "LOW")
    assert p.returncode == 0, p.stdout
    assert "0 findings" in p.stdout

```

## 07-python-sast-scanner/tests/test_network.py

```
"""Fetch and scan a real public repository (needs internet)."""
import pytest

from src.fetch import fetch_github
from src.scanner import scan_path

SAMPLEPROJECT_SHA = "621e4974ca25ce531773def586ba3ed8e736b3fc"


@pytest.mark.network
def test_fetch_and_scan_pypa_sampleproject(tmp_path):
    meta = fetch_github("pypa/sampleproject", SAMPLEPROJECT_SHA, tmp_path / "sp")
    assert meta["commit"] == SAMPLEPROJECT_SHA and meta["files_extracted"] >= 5
    res = scan_path(meta["source_dir"])
    assert res.files_scanned >= 3 and res.errors == []
    assert res.counts()["total"] == 0          # recorded result: zero findings at this commit

```

```python
"""JSON / SARIF / HTML report structure and safety."""
import json
from pathlib import Path

import jsonschema
import pytest

from src.reporters import to_html, to_json, to_sarif, write_reports
from src.scanner import scan_path

ROOT = Path(__file__).resolve().parents[1]
SCHEMA = json.loads((ROOT / "schemas" / "sarif-schema-2.1.0.json").read_text())


@pytest.fixture(scope="module")
def result():
    return scan_path(ROOT / "samples")


def test_sarif_validates_against_official_2_1_0_schema(result):
    sarif = to_sarif(result)
    jsonschema.validate(sarif, SCHEMA)          # raises on any structural error
    assert sarif["version"] == "2.1.0"


def test_sarif_results_reference_rules_and_locations(result):
    run = to_sarif(result)["runs"][0]
    rule_ids = [r["id"] for r in run["tool"]["driver"]["rules"]]
    assert len(run["results"]) == len(result.findings)
    for res in run["results"]:
        assert rule_ids[res["ruleIndex"]] == res["ruleId"]
        region = res["locations"][0]["physicalLocation"]["region"]
        assert region["startLine"] >= 1 and region["startColumn"] >= 1
        assert res["level"] in {"error", "warning", "note"}
    suppressed = [r for r in run["results"] if "suppressions" in r]
    assert len(suppressed) == result.counts()["suppressed"] == 2


def test_empty_scan_is_valid_sarif(tmp_path):
    (tmp_path / "ok.py").write_text("print('hi')\n")
    sarif = to_sarif(scan_path(tmp_path))
    jsonschema.validate(sarif, SCHEMA)
    assert sarif["runs"][0]["results"] == []


def test_json_report_summary_matches(result):
    data = to_json(result)
    assert data["summary"]["total"] == len(result.active)
    assert data["summary"]["files_scanned"] == result.files_scanned
    assert all({"rule_id", "severity", "path", "line", "fix", "cwe"} <= set(f) for f in data["find
ings"])


def test_html_is_self_contained_and_escapes_code(tmp_path):
    (tmp_path / "x.py").write_text('eval("<script>alert(1)</script>" + q)\n')
    html = to_html(scan_path(tmp_path))
    assert "<script>alert" not in html and "&lt;script&gt;" in html
    assert "<script" not in html.lower()                    # no JS at all
    assert "http://" not in html and "src=" not in html     # no external assets


def test_no_report_leaks_planted_secret(result, tmp_path):
    paths = write_reports(result, tmp_path / "r")
    blob = "".join(p.read_text() for p in paths)
    assert "fake-Pa55w0rd-demo-only" not in blob
```

## 07-python-sast-scanner/tests/test_reports.py (continued)

```
    assert "demo_live_9f8e7d6c5b4a3210fake" not in blob
    assert {p.suffix for p in paths} == {".json", ".sarif", ".html"}

```

```python
"""True positives and benign look-alikes for every rule (pure AST, nothing executed)."""
import textwrap

import pytest

from src.scanner import scan_source


def rules_for(code: str) -> list[str]:
    return sorted(f.rule_id for f in scan_source(textwrap.dedent(code)) if not f.suppressed)


TRUE_POSITIVES = [
    ("PY001", 'DB_PASSWORD = "fake-Pa55w0rd-demo"'),
    ("PY001", 'class C:\n    def __init__(self):\n        self.api_key = "fake-key-1234567890"'),
    ("PY001", 'cfg = {"client_secret": "fake-client-secret-demo"}'),
    ("PY001", 'connect(host="db", password="fake-kwarg-pw-1")'),
    ("PY001", 'def login(user, password="fake-default-pw-9"):\n    pass'),
    ("PY001", 'if token == "fake-static-token-42":\n    pass'),
    ("PY002", 'cur.execute(f"SELECT * FROM t WHERE id = {uid}")'),
    ("PY002", 'cur.execute("SELECT * FROM t WHERE name = \'" + name + "\'")'),
    ("PY002", 'cur.execute("DELETE FROM t WHERE id = %s" % uid)'),
    ("PY002", 'cur.execute("UPDATE t SET a = {}".format(v))'),
    ("PY002", 'def f(x):\n    q = f"SELECT * FROM t WHERE x = {x}"\n    return db.execute(q)'),
    ("PY002", 'pd.read_sql(f"SELECT * FROM {table}", conn)'),
    ("PY003", 'eval(user_input)'),
    ("PY003", 'exec(code)'),
    ("PY004", 'import pickle\npickle.loads(blob)'),
    ("PY004", 'from pickle import load\nload(fh)'),
    ("PY004", 'import dill\ndill.loads(b)'),
    ("PY005", 'import yaml\nyaml.load(s)'),
    ("PY005", 'import yaml\nyaml.load(s, Loader=yaml.UnsafeLoader)'),
    ("PY005", 'import yaml\nyaml.unsafe_load(s)'),
    ("PY006", 'import hashlib\nhashlib.md5(b"x")'),
    ("PY006", 'import hashlib as h\nh.sha1(b"x")'),
    ("PY006", 'from hashlib import md5\nmd5(b"x")'),
    ("PY006", 'import hashlib\nhashlib.new("SHA1", b"x")'),
    ("PY007", 'import subprocess\nsubprocess.run(cmd, shell=True)'),
    ("PY007", 'import subprocess as sp\nsp.Popen("ls " + d, shell=True)'),
    ("PY007", 'from subprocess import check_output\ncheck_output(f"du {p}", shell=True)'),
    ("PY008", 'import os\nos.system("rm -rf " + path)'),
    ("PY008", 'import os\nos.popen(cmd)'),
]


@pytest.mark.parametrize("rule,code", TRUE_POSITIVES, ids=[f"{r}-{i}" for i, (r, _) in enumerate(TRUE_POSITIVES)])
def test_true_positive(rule, code):
    assert rule in rules_for(code)


BENIGN = [
    'DB_PASSWORD = os.environ["DB_PASSWORD"]',
    'API_KEY = os.getenv("API_KEY", "")',
    'SECRET_KEY = "changeme"',
    'password = "<your-password>"',
    'token = "${GITHUB_TOKEN}"',
    'api_key = "xxxxxxxx"',
    'password = "short"',
    'password_field = "password"',
    'TOKEN_URL = "https://example.invalid/token"',
    'reset_password_template = "emails/reset.html"',
    'cur.execute("SELECT * FROM t WHERE id = ?", (uid,))',
    'cur.execute("SELECT count(*) FROM t")',
    'def f():\n    q = "SELECT 1"\n    return db.execute(q)',
```

```
    'log.info(f"user {name} logged in")',
    'model.eval()',
    'ast.literal_eval(text)',
    'import yaml\nyaml.safe_load(s)',
    'import yaml\nyaml.load(s, Loader=yaml.SafeLoader)',
    'import yaml\nfrom yaml import CSafeLoader\nyaml.load(s, CSafeLoader)',
    'import hashlib\nhashlib.sha256(b"x")',
    'import hashlib\nhashlib.md5(b"x", usedforsecurity=False)',
    'import subprocess\nsubprocess.run(["ls", "-la", path])',
    'import subprocess\nsubprocess.run(["ls"], shell=False)',
    'import json\njson.loads(s)',
]


@pytest.mark.parametrize("code", BENIGN)
def test_benign_has_no_findings(code):
    assert rules_for(code) == []


def test_severity_adjusts_for_literals():
    lit = scan_source('import subprocess\nsubprocess.run("ls -la", shell=True)')
    dyn = scan_source('import subprocess\nsubprocess.run("ls " + d, shell=True)')
    assert lit[0].severity == "MEDIUM" and dyn[0].severity == "HIGH"
    assert scan_source('eval("1+1")')[0].severity == "LOW"


def test_finding_has_location_message_and_fix():
    f = scan_source('import pickle\n\nx = pickle.loads(b)\n', "pkg/mod.py")[0]
    assert (f.path, f.line, f.column) == ("pkg/mod.py", 3, 5)
    assert "pickle.loads" in f.message
    assert f.rule.cwe == "CWE-502" and "JSON" in f.rule.fix
    assert len(f.fingerprint) == 16


def test_credential_value_is_redacted_in_snippet():
    f = scan_source('DB_PASSWORD = "fake-Pa55w0rd-demo"')[0]
    assert "fake-Pa55w0rd-demo" not in f.snippet and f.snippet.startswith('DB_PASSWORD = "fa*')


def test_sql_variable_tracking_is_scope_local():
    # a dynamic query in one function must not taint a same-named variable in another
    code = ('def a(x):\n    q = f"SELECT * FROM t WHERE x={x}"\n'
            'def b():\n    q = "SELECT 1"\n    return db.execute(q)\n')
    assert rules_for(code) == []


# ---- documented limitations: these assert the scanner's known blind spots -----
def test_limitation_no_interprocedural_flow():
    code = 'def build(x):\n    return f"SELECT * FROM t WHERE x={x}"\ndb.execute(build(v))'
    assert "PY002" not in rules_for(code)   # needs data-flow analysis, out of scope


def test_limitation_flags_safe_roundtrip_pickle():
    # pickle of our own object is safe, but syntax alone cannot know that -> false positive
    assert rules_for('import pickle\npickle.loads(pickle.dumps(obj))') == ["PY004"]
```

```python
"""The scanner must never execute, import or follow anything in the scanned tree."""
import io
import os
import zipfile

import pytest

from src.fetch import safe_extract
from src.scanner import scan_path


def test_scanned_code_is_never_executed(tmp_path):
    marker = tmp_path / "PWNED"
    (tmp_path / "evil.py").write_text(
        "import os, pathlib\n"
        f"pathlib.Path({str(marker)!r}).write_text('ran')\n"
        "os._exit(3)\n"
        "eval(input())\n")
    (tmp_path / "conftest.py").write_text(f"open({str(marker)!r}, 'w').write('ran')\n")
    res = scan_path(tmp_path)
    assert not marker.exists()
    assert res.files_scanned == 2 and any(f.rule_id == "PY003" for f in res.findings)


def test_syntax_error_is_reported_not_raised(tmp_path):
    (tmp_path / "broken.py").write_text("def f(:\n")
    (tmp_path / "ok.py").write_text("x = 1\n")
    res = scan_path(tmp_path)
    assert res.files_scanned == 1 and res.errors[0]["path"] == "broken.py"


def test_symlinks_are_not_followed(tmp_path):
    outside = tmp_path / "outside"
    outside.mkdir()
    (outside / "secret.py").write_text("eval(x)\n")
    repo = tmp_path / "repo"
    repo.mkdir()
    (repo / "a.py").write_text("x = 1\n")
    os.symlink(outside, repo / "linkdir")
    os.symlink(outside / "secret.py", repo / "link.py")
    res = scan_path(repo)
    assert res.files_scanned == 1 and res.findings == []


def test_excluded_dirs_skipped(tmp_path):
    for d in (".git", "node_modules", ".venv"):
        (tmp_path / d).mkdir()
        (tmp_path / d / "x.py").write_text("eval(x)\n")
    assert scan_path(tmp_path).files_scanned == 0


def _zip(entries):
    buf = io.BytesIO()
    with zipfile.ZipFile(buf, "w") as zf:
        for name, data, mode in entries:
            info = zipfile.ZipInfo(name)
            if mode:
                info.external_attr = mode << 16
            zf.writestr(info, data)
    return buf.getvalue()


def test_zip_slip_rejected(tmp_path):
    data = _zip([("repo/ok.py", "x=1", 0), ("../../escape.py", "x=1", 0)])
    with pytest.raises(ValueError, match="unsafe path"):
```

```
        safe_extract(data, tmp_path / "dest")
    assert not (tmp_path / "escape.py").exists()


def test_zip_symlink_entries_skipped(tmp_path):
    data = _zip([("repo/ok.py", "x=1", 0), ("repo/link", "/etc/passwd", 0o120777)])
    n = safe_extract(data, tmp_path / "dest")
    assert n == 1 and not (tmp_path / "dest" / "repo" / "link").exists()

```

```python
"""Inline suppressions and the allowlist file."""
import json

import pytest

from src.scanner import Allowlist, scan_path, scan_source


def test_nosec_suppresses_all_rules_on_line():
    f = scan_source('import hashlib\nhashlib.md5(b"x")  # nosec')[0]
    assert f.suppressed and f.suppression_reason.startswith("inline")


def test_rule_specific_suppression_only_covers_listed_rule():
    code = 'import hashlib, os\nos.system("x " + hashlib.md5(b).hexdigest())  # sast: ignore[PY006] cache key'
    by_rule = {f.rule_id: f for f in scan_source(code)}
    assert by_rule["PY006"].suppressed
    assert "cache key" in by_rule["PY006"].suppression_reason
    assert not by_rule["PY008"].suppressed


def test_nosec_inside_string_is_not_a_suppression():
    f = scan_source('import os\nos.system("echo # nosec " + x)')[0]
    assert not f.suppressed


def test_suppression_on_continuation_line_of_multiline_call():
    code = 'import subprocess\nsubprocess.run(\n    cmd,\n    shell=True,  # nosec\n)'
    assert scan_source(code)[0].suppressed


def test_allowlist_rule_path_and_fingerprint(tmp_path):
    (tmp_path / "legacy").mkdir()
    (tmp_path / "legacy" / "old.py").write_text('import hashlib\nhashlib.md5(b"a")\n')
    (tmp_path / "new.py").write_text('import hashlib\nhashlib.md5(b"a")\nimport pickle\npickle.loads(b)\n')
    first = scan_path(tmp_path)
    pickle_fp = next(f.fingerprint for f in first.findings if f.rule_id == "PY004")
    cfg = tmp_path / "allow.yml"
    cfg.write_text(
        "suppress:\n"
        "  - rule: PY006\n    path: 'legacy/*'\n    reason: cache key\n"
        f"  - fingerprint: '{pickle_fp}'\n    reason: reviewed\n")
    res = scan_path(tmp_path, Allowlist.load(cfg))
    state = {(f.path, f.rule_id): f.suppressed for f in res.findings}
    assert state == {("legacy/old.py", "PY006"): True, ("new.py", "PY006"): False,
                     ("new.py", "PY004"): True}
    assert res.counts()["suppressed"] == 2 and res.counts()["total"] == 1


def test_allowlist_disable_and_exclude(tmp_path):
    (tmp_path / "tests").mkdir()
    (tmp_path / "tests" / "t.py").write_text("eval(x)\n")
    (tmp_path / "a.py").write_text("import os\nos.system(c)\neval(y)\n")
    cfg = tmp_path / "allow.json"
    cfg.write_text(json.dumps({"disable_rules": ["PY008"], "exclude_paths": ["tests/*"]}))
    res = scan_path(tmp_path, Allowlist.load(cfg))
    assert [(f.path, f.rule_id) for f in res.findings] == [("a.py", "PY003")]
    assert res.files_scanned == 1


def test_allowlist_rejects_unknown_rule(tmp_path):
    cfg = tmp_path / "a.yml"
    cfg.write_text("disable_rules: [PY999]\n")
```

```
    with pytest.raises(ValueError):
        Allowlist.load(cfg)


def test_fingerprint_stable_when_lines_shift():
    a = scan_source('import pickle\npickle.loads(b)', "m.py")[0]
    b = scan_source('import pickle\n\n\n# comment\npickle.loads(b)', "m.py")[0]
    assert a.line != b.line and a.fingerprint == b.fingerprint

```

## 07-python-sast-scanner/.gitignore

```
__pycache__/
*.py[cod]
.pytest_cache/
.venv/
venv/
*.db
data/real/
.streamlit/secrets.toml

```

**07-python-sast-scanner/.sast-allowlist.example.yml**

```
# Example allowlist for pysast. Copy to .sast-allowlist.yml and pass --allowlist.
disable_rules: []            # e.g. [PY008]
exclude_paths:               # fnmatch globs relative to the scan root
  - "tests/*"
  - "docs/*"
suppress:
  - rule: PY006
    path: "legacy/*.py"
    reason: "MD5 used only as a cache key"
  # - fingerprint: 0123456789abcdef   # from a JSON/SARIF report
  #   reason: "reviewed 2026-10-27, input is a constant"

```

## 07-python-sast-scanner/data/README.md

```
`scripts/run_demo.sh` downloads the real repositories it scans into `data/real/` (gitignored).

```

## 07-python-sast-scanner/pytest.ini

```
[pytest]
pythonpath = .
testpaths = tests
timeout = 120
markers =
    network: needs internet access (downloads a public GitHub repository)

```

```
# Sample corpus

**Nothing in this folder is meant to be run.** The scanner only parses these files.

* `vulnerable/` - a tiny, deliberately insecure "notes app". Every insecure line ends with
  `# expect: <RULE_ID>`; `pysast evaluate` uses those markers as ground truth.
  All credentials are fake strings invented for this demo (they contain `fake`/`demo`).
* `benign/` - safe code that *resembles* the risky patterns (parameterised SQL,
  `yaml.safe_load`, SHA-256, argument lists, placeholder passwords, env vars).
  Any finding here counts as a false positive.

```

**07-python-sast-scanner/samples/allowlists/requests-triage.yml**

```
# Triage of the 14 findings pysast reports on psf/requests v2.32.3 (manual review).
# Every entry keeps the finding in the report, marked suppressed with this reason.
suppress:
  - rule: PY004
    path: "tests/*"
    reason: "pickle round-trip of objects the test itself just created (pickle.loads(pickle.dumps(
x)))"
  - rule: PY001
    path: "tests/*"
    reason: "test fixture of URL-special characters, not a credential"
  - rule: PY006
    path: "src/requests/auth.py"
    reason: "HTTP Digest auth (RFC 7616) requires MD5/SHA-1 when the server selects them"
  - rule: PY003
    path: "setup.py"
    reason: "exec of the package's own __version__.py at build time; input is a trusted repo file"
  - rule: PY008
    path: "setup.py"
    reason: "maintainer publish helper running constant commands"

```

## 07-python-sast-scanner/samples/benign/safe_app.py

```
"""Safe code that looks similar to the vulnerable samples. Expected findings: none."""
import ast
import hashlib
import json
import os
import shlex
import sqlite3
import subprocess

import yaml

# Placeholders and indirection, not real secrets
DB_PASSWORD = os.environ.get("DB_PASSWORD", "")
API_KEY = os.getenv("API_KEY")
SECRET_KEY = "changeme"
PASSWORD_PLACEHOLDER = "<your-password>"
token = "${GITHUB_TOKEN}"
password_field = "password"
PASSWORD_MIN_LENGTH = 12
TOKEN_URL = "https://auth.example.invalid/oauth/token"
api_key_env = "SERVICE_API_KEY"
DEFAULTS = {"password": "", "token": None, "api_key": "xxxx"}

conn = sqlite3.connect(":memory:")


def find_user(username):
    return conn.execute("SELECT * FROM users WHERE name = ?", (username,)).fetchone()


def find_by_ids(ids):
    placeholders = ",".join("?" for _ in ids)
    return conn.execute("SELECT * FROM users WHERE id IN (%s)" % placeholders, ids)  # sast: ignor
e[PY002] only '?' markers interpolated


def table_count():
    query = "SELECT COUNT(*) FROM notes"
    return conn.execute(query).fetchone()


def greet(name):
    message = f"Hello {name}, welcome back"
    print(message)


def parse_literal(text):
    return ast.literal_eval(text)


def load_config(text):
    return yaml.safe_load(text)


def load_config_explicit(text):
    return yaml.load(text, Loader=yaml.SafeLoader)


def load_json(text):
    return json.loads(text)


def digest(data):
    return hashlib.sha256(data).hexdigest()

```

## 07-python-sast-scanner/samples/benign/safe_app.py (continued)

```
65
66 def cache_key(data):
67     return hashlib.md5(data, usedforsecurity=False).hexdigest()
68
69
70 def ping(host):
71     return subprocess.run(["ping", "-c", "1", host], check=False)
72
73
74 def list_dir(path):
75     return subprocess.check_output(["ls", "-la", shlex.quote(path)], shell=False)
76
77
78 class Model:
79     def eval(self):
80         """A method named eval is not the builtin."""
81         return self
82
83
84 def train(model):
85     model.eval()
86
87
88 def compare(user_input):
89     return user_input == "yes"
90
```

## 07-python-sast-scanner/samples/vulnerable/app/__init__.py

```
"""Deliberately vulnerable demo package. Never run this code."""

```

## 07-python-sast-scanner/samples/vulnerable/app/config.py

```
"""Configuration with hard-coded (fake) secrets."""
import os

DEBUG = True
DB_HOST = "localhost"
DB_PASSWORD = "fake-Pa55w0rd-demo-only"  # expect: PY001
API_KEY = "demo_live_9f8e7d6c5b4a3210fake"  # expect: PY001
SECRET_KEY: str = "not-a-real-flask-secret-demo"  # expect: PY001
password_hint = "your pet's name"           # metadata, not a secret
TOKEN_URL = "https://auth.example.invalid/token"   # a URL, not a token

SETTINGS = {
    "user": "notes_app",
    "password": "fake-dict-passw0rd-demo",  # expect: PY001
    "client_secret": os.environ.get("CLIENT_SECRET", ""),
}


class Mailer:
    def __init__(self):
        self.smtp_password = "fake-smtp-Secret-demo-1"  # expect: PY001

```

## 07-python-sast-scanner/samples/vulnerable/app/db.py

```
"""Data access layer with SQL built from strings."""
import sqlite3

conn = sqlite3.connect(":memory:")


def find_user(username):
    cur = conn.cursor()
    cur.execute(f"SELECT * FROM users WHERE name = '{username}'")  # expect: PY002
    return cur.fetchone()


def find_notes(owner_id, order):
    query = "SELECT id, body FROM notes WHERE owner = " + str(owner_id) + " ORDER BY " + order
    return conn.execute(query).fetchall()  # expect: PY002


def delete_note(note_id):
    conn.execute("DELETE FROM notes WHERE id = %s" % note_id)  # expect: PY002


def search(term):
    sql = "SELECT * FROM notes WHERE body LIKE '%{}%'".format(term)
    conn.cursor().executemany(sql, [])  # expect: PY002


def login(username, password):
    if password == "fake-backdoor-pass-demo":  # expect: PY001
        return True
    return conn.execute("SELECT 1 FROM users WHERE name = ? AND pw = ?", (username, password)).fet
chone()


def connect_reporting():
    return sqlite3.connect("reports.db", timeout=5), dict(password="fake-kwarg-pw-demo1")  # expec
t: PY001

```

## 07-python-sast-scanner/samples/vulnerable/app/utils.py

```python
"""Grab-bag of insecure helpers."""
import hashlib
import hashlib as hl
import os
import pickle
import subprocess
from subprocess import check_output as co

import yaml


def calc(expression):
    return eval(expression)  # expect: PY003


def run_plugin(code_text):
    exec(code_text)  # expect: PY003


def load_session(blob):
    return pickle.loads(blob)  # expect: PY004


def load_cache(path):
    with open(path, "rb") as fh:
        return pickle.load(fh)  # expect: PY004


def load_config(text):
    return yaml.load(text)  # expect: PY005


def load_config_unsafe(text):
    return yaml.load(text, Loader=yaml.Loader)  # expect: PY005


def hash_password(pw):
    return hashlib.md5(pw.encode()).hexdigest()  # expect: PY006


def file_id(data):
    return hl.sha1(data).hexdigest()  # expect: PY006


def legacy_digest(data):
    return hashlib.new("md5", data).hexdigest()  # expect: PY006


def ping(host):
    return subprocess.run("ping -c 1 " + host, shell=True)  # expect: PY007


def disk_usage(path):
    return co(f"du -sh {path}", shell=True)  # expect: PY007


def backup(path):
    os.system("tar czf backup.tgz " + path)  # expect: PY008


def etag(data):
    # Reviewed: MD5 is only a cache key here, so it is suppressed inline.
    return hashlib.md5(data).hexdigest()  # sast: ignore[PY006] cache key, not security

```

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "title": "Static Analysis Results Format (SARIF) Version 2.1.0 JSON Schema",
  "id": "https://docs.oasis-open.org/sarif/sarif/v2.1.0/errata01/os/schemas/sarif-schema-2.1.0.json",
  "description": "Static Analysis Results Format (SARIF) Version 2.1.0 JSON Schema: a standard format for the output of static analysis tools.",
  "additionalProperties": false,
  "type": "object",
  "properties": {

    "$schema": {
      "description": "The URI of the JSON schema corresponding to the version.",
      "type": "string",
      "format": "uri"
    },

    "version": {
      "description": "The SARIF format version of this log file.",
      "enum": [ "2.1.0" ],
      "type": "string"
    },

    "runs": {
      "description": "The set of runs contained in this log file.",
      "type": [ "array", "null" ],
      "minItems": 0,
      "uniqueItems": false,
      "items": {
        "$ref": "#/definitions/run"
      }
    },

    "inlineExternalProperties": {
      "description": "References to external property files that share data between runs.",
      "type": "array",
      "minItems": 0,
      "uniqueItems": true,
      "items": {
        "$ref": "#/definitions/externalProperties"
      }
    },

    "properties": {
      "description": "Key/value pairs that provide additional information about the log file.",
      "$ref": "#/definitions/propertyBag"
    }
  },

  "required": [ "version", "runs" ],

  "definitions": {

    "address": {
      "description": "A physical or virtual address, or a range of addresses, in an 'addressable region' (memory or a binary file).",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "absoluteAddress": {
          "description": "The address expressed as a byte offset from the start of the addressable region.",
          "type": "integer",
          "minimum": -1,
```

```
          "default": -1

        },

        "relativeAddress": {
          "description": "The address expressed as a byte offset from the absolute address of the top-most parent object.",
          "type": "integer"

        },

        "length": {
          "description": "The number of bytes in this range of addresses.",
          "type": "integer"
        },

        "kind": {
          "description": "An open-ended string that identifies the address kind. 'data', 'function', 'header','instruction', 'module', 'page', 'section', 'segment', 'stack', 'stackFrame', 'table' are well-known values.",
          "type": "string"
        },

        "name": {
          "description": "A name that is associated with the address, e.g., '.text'.",
          "type": "string"
        },

        "fullyQualifiedName": {
          "description": "A human-readable fully qualified name that is associated with the address.",
          "type": "string"
        },

        "offsetFromParent": {
          "description": "The byte offset of this address from the absolute or relative address of the parent object.",
          "type": "integer"
        },

        "index": {
          "description": "The index within run.addresses of the cached object for this address.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "parentIndex": {
          "description": "The index within run.addresses of the parent object.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the address.",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "artifact": {
      "description": "A single artifact. In some cases, this artifact might be nested within another artifact.",
      "additionalProperties": false,
```

```
      "type": "object",
      "properties": {

        "description": {
          "description": "A short description of the artifact.",
          "$ref": "#/definitions/message"
        },

        "location": {
          "description": "The location of the artifact.",
          "$ref": "#/definitions/artifactLocation"
        },

        "parentIndex": {
          "description": "Identifies the index of the immediate parent of the artifact, if this ar
tifact is nested.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "offset": {
          "description": "The offset in bytes of the artifact within its containing artifact.",
          "type": "integer",
          "minimum": 0
        },

        "length": {
          "description": "The length of the artifact in bytes.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "roles": {
          "description": "The role or roles played by the artifact in the analysis.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "enum": [
              "analysisTarget",
              "attachment",
              "responseFile",
              "resultFile",
              "standardStream",
              "tracedFile",
              "unmodified",
              "modified",
              "added",
              "deleted",
              "renamed",
              "uncontrolled",
              "driver",
              "extension",
              "translation",
              "taxonomy",
              "policy",
              "referencedOnCommandLine",
              "memoryContents",
              "directory",
              "userSpecifiedConfiguration",
              "toolSpecifiedConfiguration",
              "debugOutputFile"
```

```
            ],
            "type": "string"
          }
        },

        "mimeType": {
          "description": "The MIME type (RFC 2045) of the artifact.",
          "type": "string",
          "pattern": "[^/]+/.+"
        },

        "contents": {
          "description": "The contents of the artifact.",
          "$ref": "#/definitions/artifactContent"
        },

        "encoding": {
          "description": "Specifies the encoding for an artifact object that refers to a text file
.",
          "type": "string"
        },

        "sourceLanguage": {
          "description": "Specifies the source language for any artifact object that refers to a t
ext file that contains source code.",
          "type": "string"
        },

        "hashes": {
          "description": "A dictionary, each of whose keys is the name of a hash function and each
 of whose values is the hashed value of the artifact produced by the specified hash function.",
          "type": "object",
          "additionalProperties": {
            "type": "string"
          }
        },

        "lastModifiedTimeUtc": {
          "description": "The Coordinated Universal Time (UTC) date and time at which the artifact
 was most recently modified. See \"Date/time properties\" in the SARIF spec for the required forma
t.",
          "type": "string",
          "format": "date-time"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the artifact."
,
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "artifactChange": {
      "description": "A change to a single artifact.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "artifactLocation": {
          "description": "The location of the artifact to change.",
          "$ref": "#/definitions/artifactLocation"
        },

        "replacements": {
```

```
          "description": "An array of replacement objects, each of which represents the replacement of a single region in a single artifact specified by 'artifactLocation'.",
          "type": "array",
          "minItems": 1,
          "uniqueItems": false,
          "items": {
            "$ref": "#/definitions/replacement"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the change.",
          "$ref": "#/definitions/propertyBag"
        }

      },

      "required": [ "artifactLocation", "replacements" ]
    },

    "artifactContent": {
      "description": "Represents the contents of an artifact.",
      "type": "object",
      "additionalProperties": false,
      "properties": {

        "text": {
          "description": "UTF-8-encoded content from a text artifact.",
          "type": "string"
        },

        "binary": {
          "description": "MIME Base64-encoded content from a binary artifact, or from a text artifact in its original encoding.",
          "type": "string"
        },

        "rendered": {
          "description": "An alternate rendered representation of the artifact (e.g., a decompiled representation of a binary region).",
          "$ref": "#/definitions/multiformatMessageString"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the artifact content.",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "artifactLocation": {
      "description": "Specifies the location of an artifact.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "uri": {
          "description": "A string containing a valid relative or absolute URI.",
          "type": "string",
          "format": "uri-reference"
        },

        "uriBaseId": {
          "description": "A string which indirectly specifies the absolute URI with respect to whi
```

```
ch a relative URI in the \"uri\" property is interpreted.",
          "type": "string"
        },

        "index": {
          "description": "The index within the run artifacts array of the artifact object associat
ed with the artifact location.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "description": {
          "description": "A short description of the artifact location.",
          "$ref": "#/definitions/message"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the artifact l
ocation.",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "attachment": {
      "description": "An artifact relevant to a result.",
      "type": "object",
      "additionalProperties": false,
      "properties": {

        "description": {
          "description": "A message describing the role played by the attachment.",
          "$ref": "#/definitions/message"
        },

        "artifactLocation": {
          "description": "The location of the attachment.",
          "$ref": "#/definitions/artifactLocation"
        },

        "regions": {
          "description": "An array of regions of interest within the attachment.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/region"
          }
        },

        "rectangles": {
          "description": "An array of rectangles specifying areas of interest within the image.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/rectangle"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the attachment
```

```
    .",
367           "$ref": "#/definitions/propertyBag"
368         }
369       },
370
371       "required": [ "artifactLocation" ]
372     },
373
374     "codeFlow": {
375       "description": "A set of threadFlows which together describe a pattern of code execution rel
    evant to detecting a result.",
376       "additionalProperties": false,
377       "type": "object",
378       "properties": {
379
380         "message": {
381           "description": "A message relevant to the code flow.",
382           "$ref": "#/definitions/message"
383         },
384
385         "threadFlows": {
386           "description": "An array of one or more unique threadFlow objects, each of which describ
    es the progress of a program through a thread of execution.",
387           "type": "array",
388           "minItems": 1,
389           "uniqueItems": false,
390           "items": {
391             "$ref": "#/definitions/threadFlow"
392           }
393         },
394
395         "properties": {
396           "description": "Key/value pairs that provide additional information about the code flow.
    ",
397           "$ref": "#/definitions/propertyBag"
398         }
399       },
400
401       "required": [ "threadFlows" ]
402     },
403
404     "configurationOverride": {
405       "description": "Information about how a specific rule or notification was reconfigured at ru
    ntime.",
406       "type": "object",
407       "additionalProperties": false,
408       "properties": {
409
410         "configuration": {
411           "description": "Specifies how the rule or notification was configured during the scan.",
412           "$ref": "#/definitions/reportingConfiguration"
413         },
414
415         "descriptor": {
416           "description": "A reference used to locate the descriptor whose configuration was overri
    dden.",
417           "$ref": "#/definitions/reportingDescriptorReference"
418         },
419
420         "properties": {
421           "description": "Key/value pairs that provide additional information about the configurat
    ion override.",
422           "$ref": "#/definitions/propertyBag"
423         }
424       },
```

```
      "required": [ "configuration", "descriptor" ]
    },

    "conversion": {
      "description": "Describes how a converter transformed the output of a static analysis tool from the analysis tool's native output format into the SARIF format.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "tool": {
          "description": "A tool object that describes the converter.",
          "$ref": "#/definitions/tool"
        },

        "invocation": {
          "description": "An invocation object that describes the invocation of the converter.",
          "$ref": "#/definitions/invocation"
        },

        "analysisToolLogFiles": {
          "description": "The locations of the analysis tool's per-run log files.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/artifactLocation"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the conversion.",
          "$ref": "#/definitions/propertyBag"
        }

      },

      "required": [ "tool" ]
    },

    "edge": {
      "description": "Represents a directed edge in a graph.",
      "type": "object",
      "additionalProperties": false,
      "properties": {

        "id": {
          "description": "A string that uniquely identifies the edge within its graph.",
          "type": "string"
        },

        "label": {
          "description": "A short description of the edge.",
          "$ref": "#/definitions/message"
        },

        "sourceNodeId": {
          "description": "Identifies the source node (the node at which the edge starts).",
          "type": "string"
        },

        "targetNodeId": {
          "description": "Identifies the target node (the node at which the edge ends).",
```

```
488           "type": "string"
489         },
490
491         "properties": {
492           "description": "Key/value pairs that provide additional information about the edge.",
493           "$ref": "#/definitions/propertyBag"
494         }
495       },
496
497       "required": [ "id", "sourceNodeId", "targetNodeId" ]
498     },
499
500     "edgeTraversal": {
501       "description": "Represents the traversal of a single edge during a graph traversal.",
502       "type": "object",
503       "additionalProperties": false,
504       "properties": {
505
506         "edgeId": {
507           "description": "Identifies the edge being traversed.",
508           "type": "string"
509         },
510
511         "message": {
512           "description": "A message to display to the user as the edge is traversed.",
513           "$ref": "#/definitions/message"
514         },
515
516         "finalState": {
517           "description": "The values of relevant expressions after the edge has been traversed.",
518           "type": "object",
519           "additionalProperties": {
520             "$ref": "#/definitions/multiformatMessageString"
521           }
522         },
523
524         "stepOverEdgeCount": {
525           "description": "The number of edge traversals necessary to return from a nested graph.",
526           "type": "integer",
527           "minimum": 0
528         },
529
530         "properties": {
531           "description": "Key/value pairs that provide additional information about the edge trave
    rsal.",
532           "$ref": "#/definitions/propertyBag"
533         }
534       },
535
536       "required": [ "edgeId" ]
537     },
538
539     "exception": {
540       "description": "Describes a runtime exception encountered during the execution of an analysi
    s tool.",
541       "type": "object",
542       "additionalProperties": false,
543       "properties": {
544
545         "kind": {
546           "type": "string",
547           "description": "A string that identifies the kind of exception, for example, the fully q
    ualified type name of an object that was thrown, or the symbolic name of a signal."
548         },
549
```

```
        "message": {
          "description": "A message that describes the exception.",
          "type": "string"
        },

        "stack": {
          "description": "The sequence of function calls leading to the exception.",
          "$ref": "#/definitions/stack"
        },

        "innerExceptions": {
          "description": "An array of exception objects each of which is considered a cause of thi
s exception.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": false,
          "default": [],
          "items": {
            "$ref": "#/definitions/exception"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the exception.
",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "externalProperties": {
      "description": "The top-level element of an external property file.",
      "type": "object",
      "additionalProperties": false,
      "properties": {

        "schema": {
          "description": "The URI of the JSON schema corresponding to the version of the external
property file format.",
          "type": "string",
          "format": "uri"
        },

        "version": {
          "description": "The SARIF format version of this external properties object.",
          "enum": [ "2.1.0" ],
          "type": "string"
        },

        "guid": {
          "description": "A stable, unique identifier for this external properties object, in the
form of a GUID.",
          "type": "string",
          "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
-9a-fA-F]{12}$"
        },

        "runGuid": {
          "description": "A stable, unique identifier for the run associated with this external pr
operties object, in the form of a GUID.",
          "type": "string",
          "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
-9a-fA-F]{12}$"
        },

```

```
608         "conversion": {
609           "description": "A conversion object that will be merged with a separate run.",
610           "$ref": "#/definitions/conversion"
611         },
612
613         "graphs": {
614           "description": "An array of graph objects that will be merged with a separate run.",
615           "type": "array",
616           "minItems": 0,
617           "default": [],
618           "uniqueItems": true,
619           "items": {
620             "$ref": "#/definitions/graph"
621           }
622         },
623
624         "externalizedProperties": {
625           "description": "Key/value pairs that provide additional information that will be merged
    with a separate run.",
626           "$ref": "#/definitions/propertyBag"
627         },
628
629         "artifacts": {
630           "description": "An array of artifact objects that will be merged with a separate run.",
631           "type": "array",
632           "minItems": 0,
633           "uniqueItems": true,
634           "items": {
635             "$ref": "#/definitions/artifact"
636           }
637         },
638
639         "invocations": {
640           "description": "Describes the invocation of the analysis tool that will be merged with a
     separate run.",
641           "type": "array",
642           "minItems": 0,
643           "uniqueItems": false,
644           "default": [],
645           "items": {
646             "$ref": "#/definitions/invocation"
647           }
648         },
649
650         "logicalLocations": {
651           "description": "An array of logical locations such as namespaces, types or functions tha
    t will be merged with a separate run.",
652           "type": "array",
653           "minItems": 0,
654           "uniqueItems": true,
655           "default": [],
656           "items": {
657             "$ref": "#/definitions/logicalLocation"
658           }
659         },
660
661         "threadFlowLocations": {
662           "description": "An array of threadFlowLocation objects that will be merged with a separa
    te run.",
663           "type": "array",
664           "minItems": 0,
665           "uniqueItems": true,
666           "default": [],
667           "items": {
668             "$ref": "#/definitions/threadFlowLocation"
```

```
669           }
670         },
671
672         "results": {
673           "description": "An array of result objects that will be merged with a separate run.",
674           "type": "array",
675           "minItems": 0,
676           "uniqueItems": false,
677           "default": [],
678           "items": {
679             "$ref": "#/definitions/result"
680           }
681         },
682
683         "taxonomies": {
684           "description": "Tool taxonomies that will be merged with a separate run.",
685           "type": "array",
686           "minItems": 0,
687           "uniqueItems": true,
688           "default": [],
689           "items": {
690             "$ref": "#/definitions/toolComponent"
691           }
692         },
693
694         "driver": {
695           "description": "The analysis tool object that will be merged with a separate run.",
696           "$ref": "#/definitions/toolComponent"
697         },
698
699         "extensions": {
700           "description": "Tool extensions that will be merged with a separate run.",
701           "type": "array",
702           "minItems": 0,
703           "uniqueItems": true,
704           "default": [],
705           "items": {
706             "$ref": "#/definitions/toolComponent"
707           }
708         },
709
710         "policies": {
711           "description": "Tool policies that will be merged with a separate run.",
712           "type": "array",
713           "minItems": 0,
714           "uniqueItems": true,
715           "default": [],
716           "items": {
717             "$ref": "#/definitions/toolComponent"
718           }
719         },
720
721         "translations": {
722           "description": "Tool translations that will be merged with a separate run.",
723           "type": "array",
724           "minItems": 0,
725           "uniqueItems": true,
726           "default": [],
727           "items": {
728             "$ref": "#/definitions/toolComponent"
729           }
730         },
731
732         "addresses": {
733           "description": "Addresses that will be merged with a separate run.",
```

```
734           "type": "array",
735           "minItems": 0,
736           "uniqueItems": false,
737           "default": [],
738           "items": {
739             "$ref": "#/definitions/address"
740           }
741         },
742
743         "webRequests": {
744           "description": "Requests that will be merged with a separate run.",
745           "type": "array",
746           "minItems": 0,
747           "uniqueItems": true,
748           "default": [],
749           "items": {
750             "$ref": "#/definitions/webRequest"
751           }
752         },
753
754         "webResponses": {
755           "description": "Responses that will be merged with a separate run.",
756           "type": "array",
757           "minItems": 0,
758           "uniqueItems": true,
759           "default": [],
760           "items": {
761             "$ref": "#/definitions/webResponse"
762           }
763         },
764
765         "properties": {
766           "description": "Key/value pairs that provide additional information about the external p
    roperties.",
767           "$ref": "#/definitions/propertyBag"
768         }
769       }
770     },
771
772     "externalPropertyFileReference": {
773       "description": "Contains information that enables a SARIF consumer to locate the external pr
    operty file that contains the value of an externalized property associated with the run.",
774       "type": "object",
775       "additionalProperties": false,
776       "properties": {
777
778         "location": {
779           "description": "The location of the external property file.",
780           "$ref": "#/definitions/artifactLocation"
781         },
782
783         "guid": {
784           "description": "A stable, unique identifier for the external property file in the form o
    f a GUID.",
785           "type": "string",
786           "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
    -9a-fA-F]{12}$"
787         },
788
789         "itemCount": {
790           "description": "A non-negative integer specifying the number of items contained in the e
    xternal property file.",
791           "type": "integer",
792           "default": -1,
793           "minimum": -1
```

```
794         },
795
796         "properties": {
797           "description": "Key/value pairs that provide additional information about the external p
    roperty file.",
798           "$ref": "#/definitions/propertyBag"
799         }
800       },
801       "anyOf": [
802         { "required": [ "location" ] },
803         { "required": [ "guid" ] }
804       ]
805     },
806
807     "externalPropertyFileReferences": {
808       "description": "References to external property files that should be inlined with the conten
    t of a root log file.",
809       "additionalProperties": false,
810       "type": "object",
811       "properties": {
812
813         "conversion": {
814           "description": "An external property file containing a run.conversion object to be merge
    d with the root log file.",
815           "$ref": "#/definitions/externalPropertyFileReference"
816         },
817
818         "graphs": {
819           "description": "An array of external property files containing a run.graphs object to be
     merged with the root log file.",
820           "type": "array",
821           "minItems": 0,
822           "uniqueItems": true,
823           "default": [],
824           "items": {
825             "$ref": "#/definitions/externalPropertyFileReference"
826           }
827         },
828
829         "externalizedProperties": {
830           "description": "An external property file containing a run.properties object to be merge
    d with the root log file.",
831           "$ref": "#/definitions/externalPropertyFileReference"
832         },
833
834         "artifacts": {
835           "description": "An array of external property files containing run.artifacts arrays to b
    e merged with the root log file.",
836           "type": "array",
837           "minItems": 0,
838           "uniqueItems": true,
839           "default": [],
840           "items": {
841             "$ref": "#/definitions/externalPropertyFileReference"
842           }
843         },
844
845         "invocations": {
846           "description": "An array of external property files containing run.invocations arrays to
     be merged with the root log file.",
847           "type": "array",
848           "minItems": 0,
849           "uniqueItems": true,
850           "default": [],
851           "items": {
```

```
852             "$ref": "#/definitions/externalPropertyFileReference"
853           }
854         },
855
856         "logicalLocations": {
857           "description": "An array of external property files containing run.logicalLocations arra
    ys to be merged with the root log file.",
858           "type": "array",
859           "minItems": 0,
860           "uniqueItems": true,
861           "default": [],
862           "items": {
863             "$ref": "#/definitions/externalPropertyFileReference"
864           }
865         },
866
867         "threadFlowLocations": {
868           "description": "An array of external property files containing run.threadFlowLocations a
    rrays to be merged with the root log file.",
869           "type": "array",
870           "minItems": 0,
871           "uniqueItems": true,
872           "default": [],
873           "items": {
874             "$ref": "#/definitions/externalPropertyFileReference"
875           }
876         },
877
878         "results": {
879           "description": "An array of external property files containing run.results arrays to be
    merged with the root log file.",
880           "type": "array",
881           "minItems": 0,
882           "uniqueItems": true,
883           "default": [],
884           "items": {
885             "$ref": "#/definitions/externalPropertyFileReference"
886           }
887         },
888
889         "taxonomies": {
890           "description": "An array of external property files containing run.taxonomies arrays to
    be merged with the root log file.",
891           "type": "array",
892           "minItems": 0,
893           "uniqueItems": true,
894           "default": [],
895           "items": {
896             "$ref": "#/definitions/externalPropertyFileReference"
897           }
898         },
899
900         "addresses": {
901           "description": "An array of external property files containing run.addresses arrays to b
    e merged with the root log file.",
902           "type": "array",
903           "minItems": 0,
904           "uniqueItems": true,
905           "default": [],
906           "items": {
907             "$ref": "#/definitions/externalPropertyFileReference"
908           }
909         },
910
911         "driver": {
```

```
912           "description": "An external property file containing a run.driver object to be merged wi
    th the root log file.",
913           "$ref": "#/definitions/externalPropertyFileReference"
914         },
915
916         "extensions": {
917           "description": "An array of external property files containing run.extensions arrays to
    be merged with the root log file.",
918           "type": "array",
919           "minItems": 0,
920           "uniqueItems": true,
921           "default": [],
922           "items": {
923             "$ref": "#/definitions/externalPropertyFileReference"
924           }
925         },
926
927         "policies": {
928           "description": "An array of external property files containing run.policies arrays to be
     merged with the root log file.",
929           "type": "array",
930           "minItems": 0,
931           "uniqueItems": true,
932           "default": [],
933           "items": {
934             "$ref": "#/definitions/externalPropertyFileReference"
935           }
936         },
937
938         "translations": {
939           "description": "An array of external property files containing run.translations arrays t
    o be merged with the root log file.",
940           "type": "array",
941           "minItems": 0,
942           "uniqueItems": true,
943           "default": [],
944           "items": {
945             "$ref": "#/definitions/externalPropertyFileReference"
946           }
947         },
948
949         "webRequests": {
950           "description": "An array of external property files containing run.requests arrays to be
     merged with the root log file.",
951           "type": "array",
952           "minItems": 0,
953           "uniqueItems": true,
954           "default": [],
955           "items": {
956             "$ref": "#/definitions/externalPropertyFileReference"
957           }
958         },
959
960         "webResponses": {
961           "description": "An array of external property files containing run.responses arrays to b
    e merged with the root log file.",
962           "type": "array",
963           "minItems": 0,
964           "uniqueItems": true,
965           "default": [],
966           "items": {
967             "$ref": "#/definitions/externalPropertyFileReference"
968           }
969         },
970
```

```
971         "properties": {
972           "description": "Key/value pairs that provide additional information about the external p
    roperty files.",
973           "$ref": "#/definitions/propertyBag"
974         }
975       }
976     },
977
978     "fix": {
979       "description": "A proposed fix for the problem represented by a result object. A fix specifi
    es a set of artifacts to modify. For each artifact, it specifies a set of bytes to remove, and pro
    vides a set of new bytes to replace them.",
980       "additionalProperties": false,
981       "type": "object",
982       "properties": {
983
984         "description": {
985           "description": "A message that describes the proposed fix, enabling viewers to present t
    he proposed change to an end user.",
986           "$ref": "#/definitions/message"
987         },
988
989         "artifactChanges": {
990           "description": "One or more artifact changes that comprise a fix for a result.",
991           "type": "array",
992           "minItems": 1,
993           "uniqueItems": true,
994           "items": {
995             "$ref": "#/definitions/artifactChange"
996           }
997         },
998
999         "properties": {
1000          "description": "Key/value pairs that provide additional information about the fix.",
1001          "$ref": "#/definitions/propertyBag"
1002        }
1003      },
1004      "required": [ "artifactChanges" ]
1005    },
1006
1007    "graph": {
1008      "description": "A network of nodes and directed edges that describes some aspect of the stru
    cture of the code (for example, a call graph).",
1009      "type": "object",
1010      "additionalProperties": false,
1011      "properties": {
1012
1013        "description": {
1014          "description": "A description of the graph.",
1015          "$ref": "#/definitions/message"
1016        },
1017
1018        "nodes": {
1019          "description": "An array of node objects representing the nodes of the graph.",
1020          "type": "array",
1021          "minItems": 0,
1022          "uniqueItems": true,
1023          "default": [],
1024          "items": {
1025            "$ref": "#/definitions/node"
1026          }
1027        },
1028
1029        "edges": {
1030          "description": "An array of edge objects representing the edges of the graph.",
```

```
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/edge"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the graph.",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "graphTraversal": {
      "description": "Represents a path through a graph.",
      "type": "object",
      "additionalProperties": false,
      "properties": {

        "runGraphIndex": {
          "description": "The index within the run.graphs to be associated with the result.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "resultGraphIndex": {
          "description": "The index within the result.graphs to be associated with the result.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "description": {
          "description": "A description of this graph traversal.",
          "$ref": "#/definitions/message"
        },

        "initialState": {
          "description": "Values of relevant expressions at the start of the graph traversal that may change during graph traversal.",
          "type": "object",
          "additionalProperties": {
            "$ref": "#/definitions/multiformatMessageString"
          }
        },

        "immutableState": {
          "description": "Values of relevant expressions at the start of the graph traversal that remain constant for the graph traversal.",
          "type": "object",
          "additionalProperties": {
            "$ref": "#/definitions/multiformatMessageString"
          }
        },

        "edgeTraversals": {
          "description": "The sequences of edges traversed by this graph traversal.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": false,
          "default": [],
```

```
          "items": {
            "$ref": "#/definitions/edgeTraversal"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the graph traversal.",
          "$ref": "#/definitions/propertyBag"
        }
      },
      "oneOf": [
        { "required": [ "runGraphIndex" ] },
        { "required": [ "resultGraphIndex" ] }
      ]
    },

    "invocation": {
      "description": "The runtime environment of the analysis tool run.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "commandLine": {
          "description": "The command line used to invoke the tool.",
          "type": "string"
        },

        "arguments": {
          "description": "An array of strings, containing in order the command line arguments passed to the tool from the operating system.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": false,
          "items": {
            "type": "string"
          }
        },

        "responseFiles": {
          "description": "The locations of any response files specified on the tool's command line.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "items": {
            "$ref": "#/definitions/artifactLocation"
          }
        },

        "startTimeUtc": {
          "description": "The Coordinated Universal Time (UTC) date and time at which the invocation started. See \"Date/time properties\" in the SARIF spec for the required format.",
          "type": "string",
          "format": "date-time"
        },

        "endTimeUtc": {
          "description": "The Coordinated Universal Time (UTC) date and time at which the invocation ended. See \"Date/time properties\" in the SARIF spec for the required format.",
          "type": "string",
          "format": "date-time"
        },

        "exitCode": {
```

```
          "description": "The process exit code.",
          "type": "integer"
        },

        "ruleConfigurationOverrides": {
          "description": "An array of configurationOverride objects that describe rules related ru
ntime overrides.",
          "type": "array",
          "minItems": 0,
          "default": [],
          "uniqueItems": true,
          "items": {
            "$ref": "#/definitions/configurationOverride"
          }
        },

        "notificationConfigurationOverrides": {
          "description": "An array of configurationOverride objects that describe notifications re
lated runtime overrides.",
          "type": "array",
          "minItems": 0,
          "default": [],
          "uniqueItems": true,
          "items": {
            "$ref": "#/definitions/configurationOverride"
          }
        },

        "toolExecutionNotifications": {
          "description": "A list of runtime conditions detected by the tool during the analysis.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": false,
          "default": [],
          "items": {
            "$ref": "#/definitions/notification"
          }
        },

        "toolConfigurationNotifications": {
          "description": "A list of conditions detected by the tool that are relevant to the tool'
s configuration.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": false,
          "default": [],
          "items": {
            "$ref": "#/definitions/notification"
          }
        },

        "exitCodeDescription": {
          "description": "The reason for the process exit.",
          "type": "string"
        },

        "exitSignalName": {
          "description": "The name of the signal that caused the process to exit.",
          "type": "string"
        },

        "exitSignalNumber": {
          "description": "The numeric value of the signal that caused the process to exit.",
          "type": "integer"
        },
```

```
1216
1217        "processStartFailureMessage": {
1218          "description": "The reason given by the operating system that the process failed to star
    t.",
1219          "type": "string"
1220        },
1221
1222        "executionSuccessful": {
1223          "description": "Specifies whether the tool's execution completed successfully.",
1224          "type": "boolean"
1225        },
1226
1227        "machine": {
1228          "description": "The machine on which the invocation occurred.",
1229          "type": "string"
1230        },
1231
1232        "account": {
1233          "description": "The account under which the invocation occurred.",
1234          "type": "string"
1235        },
1236
1237        "processId": {
1238          "description": "The id of the process in which the invocation occurred.",
1239          "type": "integer"
1240        },
1241
1242        "executableLocation": {
1243          "description": "An absolute URI specifying the location of the executable that was invok
    ed.",
1244          "$ref": "#/definitions/artifactLocation"
1245        },
1246
1247        "workingDirectory": {
1248          "description": "The working directory for the invocation.",
1249          "$ref": "#/definitions/artifactLocation"
1250        },
1251
1252        "environmentVariables": {
1253          "description": "The environment variables associated with the analysis tool process, exp
    ressed as key/value pairs.",
1254          "type": "object",
1255          "additionalProperties": {
1256            "type": "string"
1257          }
1258        },
1259
1260        "stdin": {
1261          "description": "A file containing the standard input stream to the process that was invo
    ked.",
1262          "$ref": "#/definitions/artifactLocation"
1263        },
1264
1265        "stdout": {
1266          "description": "A file containing the standard output stream from the process that was i
    nvoked.",
1267          "$ref": "#/definitions/artifactLocation"
1268        },
1269
1270        "stderr": {
1271          "description": "A file containing the standard error stream from the process that was in
    voked.",
1272          "$ref": "#/definitions/artifactLocation"
1273        },
1274
```

```
1275          "stdoutStderr": {
1276            "description": "A file containing the interleaved standard output and standard error str
      eam from the process that was invoked.",
1277            "$ref": "#/definitions/artifactLocation"
1278          },
1279
1280          "properties": {
1281            "description": "Key/value pairs that provide additional information about the invocation
      .",
1282            "$ref": "#/definitions/propertyBag"
1283          }
1284        },
1285        "required": [ "executionSuccessful" ]
1286      },
1287
1288      "location": {
1289        "description": "A location within a programming artifact.",
1290        "additionalProperties": false,
1291        "type": "object",
1292        "properties": {
1293
1294          "id": {
1295            "description": "Value that distinguishes this location from all other locations within a
       single result object.",
1296            "type": "integer",
1297            "minimum": -1,
1298            "default": -1
1299          },
1300
1301          "physicalLocation": {
1302            "description": "Identifies the artifact and region.",
1303            "$ref": "#/definitions/physicalLocation"
1304          },
1305
1306          "logicalLocations": {
1307            "description": "The logical locations associated with the result.",
1308            "type": "array",
1309            "minItems": 0,
1310            "uniqueItems": true,
1311            "default": [],
1312            "items": {
1313              "$ref": "#/definitions/logicalLocation"
1314            }
1315          },
1316
1317          "message": {
1318            "description": "A message relevant to the location.",
1319            "$ref": "#/definitions/message"
1320          },
1321
1322          "annotations": {
1323            "description": "A set of regions relevant to the location.",
1324            "type": "array",
1325            "minItems": 0,
1326            "uniqueItems": true,
1327            "default": [],
1328            "items": {
1329              "$ref": "#/definitions/region"
1330            }
1331          },
1332
1333          "relationships": {
1334            "description": "An array of objects that describe relationships between this location an
      d others.",
1335            "type": "array",
```

```
          "default": [],
          "minItems": 0,
          "uniqueItems": true,
          "items": {
            "$ref": "#/definitions/locationRelationship"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the location."
,
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "locationRelationship": {
      "description": "Information about the relation of one location to another.",
      "type": "object",
      "additionalProperties": false,
      "properties": {

        "target": {
          "description": "A reference to the related location.",
          "type": "integer",
          "minimum": 0
        },

        "kinds": {
          "description": "A set of distinct strings that categorize the relationship. Well-known k
inds include 'includes', 'isIncludedBy' and 'relevant'.",
          "type": "array",
          "default": [ "relevant" ],
          "uniqueItems": true,
          "items": {
            "type": "string"
          }
        },

        "description": {
          "description": "A description of the location relationship.",
          "$ref": "#/definitions/message"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the location r
elationship.",
          "$ref": "#/definitions/propertyBag"
        }
      },
      "required": [ "target" ]
    },

    "logicalLocation": {
      "description": "A logical location of a construct that produced a result.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "name": {
          "description": "Identifies the construct in which the result occurred. For example, this
 property might contain the name of a class or a method.",
          "type": "string"
        },

```

```
        "index": {
          "description": "The index within the logical locations array.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "fullyQualifiedName": {
          "description": "The human-readable fully qualified name of the logical location.",
          "type": "string"
        },

        "decoratedName": {
          "description": "The machine-readable name for the logical location, such as a mangled fu
nction name provided by a C++ compiler that encodes calling convention, return type and other deta
ils along with the function name.",
          "type": "string"
        },

        "parentIndex": {
          "description": "Identifies the index of the immediate parent of the construct in which t
he result was detected. For example, this property might point to a logical location that represen
ts the namespace that holds a type.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "kind": {
          "description": "The type of construct this logical location component refers to. Should
be one of 'function', 'member', 'module', 'namespace', 'parameter', 'resource', 'returnType', 'typ
e', 'variable', 'object', 'array', 'property', 'value', 'element', 'text', 'attribute', 'comment',
 'declaration', 'dtd' or 'processingInstruction', if any of those accurately describe the construc
t.",
          "type": "string"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the logical lo
cation.",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "message": {
      "description": "Encapsulates a message intended to be read by the end user.",
      "type": "object",
      "additionalProperties": false,

      "properties": {

        "text": {
          "description": "A plain text message string.",
          "type": "string"
        },

        "markdown": {
          "description": "A Markdown message string.",
          "type": "string"
        },

        "id": {
          "description": "The identifier for this message.",
          "type": "string"
```

```
        },

        "arguments": {
          "description": "An array of strings to substitute into the message string.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": false,
          "default": [],
          "items": {
            "type": "string"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the message.",
          "$ref": "#/definitions/propertyBag"
        }
      },
      "anyOf": [
        { "required": [ "text" ] },
        { "required": [ "id" ] }
      ]
    },

    "multiformatMessageString": {
      "description": "A message string or message format string rendered in multiple formats.",
      "type": "object",
      "additionalProperties": false,

      "properties": {

        "text": {
          "description": "A plain text message string or format string.",
          "type": "string"
        },

        "markdown": {
          "description": "A Markdown message string or format string.",
          "type": "string"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the message.",
          "$ref": "#/definitions/propertyBag"
        }
      },
      "required": [ "text" ]
    },

    "node": {
      "description": "Represents a node in a graph.",
      "type": "object",
      "additionalProperties": false,

      "properties": {

        "id": {
          "description": "A string that uniquely identifies the node within its graph.",
          "type": "string"
        },

        "label": {
          "description": "A short description of the node.",
          "$ref": "#/definitions/message"
        },
```

```
1518
1519        "location": {
1520          "description": "A code location associated with the node.",
1521          "$ref": "#/definitions/location"
1522        },
1523
1524        "children": {
1525          "description": "Array of child nodes.",
1526          "type": "array",
1527          "minItems": 0,
1528          "uniqueItems": true,
1529          "default": [],
1530          "items": {
1531            "$ref": "#/definitions/node"
1532          }
1533        },
1534
1535        "properties": {
1536          "description": "Key/value pairs that provide additional information about the node.",
1537          "$ref": "#/definitions/propertyBag"
1538        }
1539      },
1540
1541      "required": [ "id" ]
1542    },
1543
1544    "notification": {
1545      "description": "Describes a condition relevant to the tool itself, as opposed to being relev
     ant to a target being analyzed by the tool.",
1546      "type": "object",
1547      "additionalProperties": false,
1548      "properties": {
1549
1550        "locations": {
1551          "description": "The locations relevant to this notification.",
1552          "type": "array",
1553          "minItems": 0,
1554          "uniqueItems": true,
1555          "default": [],
1556          "items": {
1557            "$ref": "#/definitions/location"
1558          }
1559        },
1560
1561        "message": {
1562          "description": "A message that describes the condition that was encountered.",
1563          "$ref": "#/definitions/message"
1564        },
1565
1566        "level": {
1567          "description": "A value specifying the severity level of the notification.",
1568          "default": "warning",
1569          "enum": [ "none", "note", "warning", "error" ],
1570          "type": "string"
1571        },
1572
1573        "threadId": {
1574          "description": "The thread identifier of the code that generated the notification.",
1575          "type": "integer"
1576        },
1577
1578        "timeUtc": {
1579          "description": "The Coordinated Universal Time (UTC) date and time at which the analysis
      tool generated the notification.",
1580          "type": "string",
```

```
1581            "format": "date-time"
1582          },
1583
1584          "exception": {
1585            "description": "The runtime exception, if any, relevant to this notification.",
1586            "$ref": "#/definitions/exception"
1587          },
1588
1589          "descriptor": {
1590            "description": "A reference used to locate the descriptor relevant to this notification.
      ",
1591            "$ref": "#/definitions/reportingDescriptorReference"
1592          },
1593
1594          "associatedRule": {
1595            "description": "A reference used to locate the rule descriptor associated with this noti
      fication.",
1596            "$ref": "#/definitions/reportingDescriptorReference"
1597          },
1598
1599          "properties": {
1600            "description": "Key/value pairs that provide additional information about the notificati
      on.",
1601            "$ref": "#/definitions/propertyBag"
1602          }
1603        },
1604
1605        "required": [ "message" ]
1606      },
1607
1608      "physicalLocation": {
1609        "description": "A physical location relevant to a result. Specifies a reference to a program
      ming artifact together with a range of bytes or characters within that artifact.",
1610        "additionalProperties": false,
1611        "type": "object",
1612        "properties": {
1613
1614          "address": {
1615            "description": "The address of the location.",
1616            "$ref": "#/definitions/address"
1617          },
1618
1619          "artifactLocation": {
1620            "description": "The location of the artifact.",
1621            "$ref": "#/definitions/artifactLocation"
1622          },
1623
1624          "region": {
1625            "description": "Specifies a portion of the artifact.",
1626            "$ref": "#/definitions/region"
1627          },
1628
1629          "contextRegion": {
1630            "description": "Specifies a portion of the artifact that encloses the region. Allows a v
      iewer to display additional context around the region.",
1631            "$ref": "#/definitions/region"
1632          },
1633
1634          "properties": {
1635            "description": "Key/value pairs that provide additional information about the physical l
      ocation.",
1636            "$ref": "#/definitions/propertyBag"
1637          }
1638        },
1639
```

```
      "anyOf": [
        {
          "required": [ "address" ]
        },
        {
          "required": [ "artifactLocation" ]
        }
      ]
    },

    "propertyBag": {
      "description": "Key/value pairs that provide additional information about the object.",
      "type": "object",
      "additionalProperties": true,
      "properties": {
        "tags": {

          "description": "A set of distinct strings that provide additional information.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "type": "string"
          }
        }
      }
    },

    "rectangle": {
      "description": "An area within an image.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "top": {
          "description": "The Y coordinate of the top edge of the rectangle, measured in the image
's natural units.",
          "type": "number"
        },

        "left": {
          "description": "The X coordinate of the left edge of the rectangle, measured in the imag
e's natural units.",
          "type": "number"
        },

        "bottom": {
          "description": "The Y coordinate of the bottom edge of the rectangle, measured in the im
age's natural units.",
          "type": "number"
        },

        "right": {
          "description": "The X coordinate of the right edge of the rectangle, measured in the ima
ge's natural units.",
          "type": "number"
        },

        "message": {
          "description": "A message relevant to the rectangle.",
          "$ref": "#/definitions/message"
        },

        "properties": {
```

```
          "description": "Key/value pairs that provide additional information about the rectangle.
",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "region": {
      "description": "A region within an artifact where a result was detected.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "startLine": {
          "description": "The line number of the first character in the region.",
          "type": "integer",
          "minimum": 1
        },

        "startColumn": {
          "description": "The column number of the first character in the region.",
          "type": "integer",
          "minimum": 1
        },

        "endLine": {
          "description": "The line number of the last character in the region.",
          "type": "integer",
          "minimum": 1
        },

        "endColumn": {
          "description": "The column number of the character following the end of the region.",
          "type": "integer",
          "minimum": 1
        },

        "charOffset": {
          "description": "The zero-based offset from the beginning of the artifact of the first ch
aracter in the region.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "charLength": {
          "description": "The length of the region in characters.",
          "type": "integer",
          "minimum": 0
        },

        "byteOffset": {
          "description": "The zero-based offset from the beginning of the artifact of the first by
te in the region.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "byteLength": {
          "description": "The length of the region in bytes.",
          "type": "integer",
          "minimum": 0
        },

```

```
        "snippet": {
          "description": "The portion of the artifact contents within the specified region.",
          "$ref": "#/definitions/artifactContent"
        },

        "message": {
          "description": "A message relevant to the region.",
          "$ref": "#/definitions/message"
        },

        "sourceLanguage": {
          "description": "Specifies the source language, if any, of the portion of the artifact specified by the region object.",
          "type": "string"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the region.",
          "$ref": "#/definitions/propertyBag"
        }
      },

      "anyOf": [
        { "required": [ "startLine" ] },
        { "required": [ "charOffset" ] },
        { "required": [ "byteOffset" ] }
      ]
    },

    "replacement": {
      "description": "The replacement of a single region of an artifact.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "deletedRegion": {
          "description": "The region of the artifact to delete.",
          "$ref": "#/definitions/region"
        },

        "insertedContent": {
          "description": "The content to insert at the location specified by the 'deletedRegion' property.",
          "$ref": "#/definitions/artifactContent"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the replacement.",
          "$ref": "#/definitions/propertyBag"
        }
      },

      "required": [ "deletedRegion" ]
    },

    "reportingDescriptor": {
      "description": "Metadata that describes a specific report produced by the tool, as part of the analysis it provides or its runtime reporting.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "id": {
          "description": "A stable, opaque identifier for the report.",
```

```
1824            "type": "string"
1825          },
1826
1827          "deprecatedIds": {
1828            "description": "An array of stable, opaque identifiers by which this report was known in
       some previous version of the analysis tool.",
1829            "type": "array",
1830            "minItems": 0,
1831            "uniqueItems": true,
1832            "items": {
1833              "type": "string"
1834            }
1835          },
1836
1837          "guid": {
1838            "description": "A unique identifier for the reporting descriptor in the form of a GUID."
       ,
1839            "type": "string",
1840            "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
       -9a-fA-F]{12}$"
1841          },
1842
1843          "deprecatedGuids": {
1844            "description": "An array of unique identifies in the form of a GUID by which this report
       was known in some previous version of the analysis tool.",
1845            "type": "array",
1846            "minItems": 0,
1847            "uniqueItems": true,
1848            "items": {
1849              "type": "string",
1850              "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-
       [0-9a-fA-F]{12}$"
1851            }
1852          },
1853
1854          "name": {
1855            "description": "A report identifier that is understandable to an end user.",
1856            "type": "string"
1857          },
1858
1859          "deprecatedNames": {
1860            "description": "An array of readable identifiers by which this report was known in some
       previous version of the analysis tool.",
1861            "type": "array",
1862            "minItems": 0,
1863            "uniqueItems": true,
1864            "items": {
1865              "type": "string"
1866            }
1867          },
1868
1869          "shortDescription": {
1870            "description": "A concise description of the report. Should be a single sentence that is
       understandable when visible space is limited to a single line of text.",
1871            "$ref": "#/definitions/multiformatMessageString"
1872          },
1873
1874          "fullDescription": {
1875            "description": "A description of the report. Should, as far as possible, provide details
       sufficient to enable resolution of any problem indicated by the result.",
1876            "$ref": "#/definitions/multiformatMessageString"
1877          },
1878
1879          "messageStrings": {
1880            "description": "A set of name/value pairs with arbitrary names. Each value is a multifor
```

```
matMessageString object, which holds message strings in plain text and (optionally) Markdown forma
t. The strings can include placeholders, which can be used to construct a message in combination w
ith an arbitrary number of additional string arguments.",
          "type": "object",
          "additionalProperties": {
            "$ref": "#/definitions/multiformatMessageString"
          }
        },

        "defaultConfiguration": {
          "description": "Default reporting configuration information.",
          "$ref": "#/definitions/reportingConfiguration"
        },

        "helpUri": {
          "description": "A URI where the primary documentation for the report can be found.",
          "type": "string",
          "format": "uri"
        },

        "help": {
          "description": "Provides the primary documentation for the report, useful when there is
no online documentation.",
          "$ref": "#/definitions/multiformatMessageString"
        },

        "relationships": {
          "description": "An array of objects that describe relationships between this reporting d
escriptor and others.",
          "type": "array",
          "default": [],
          "minItems": 0,
          "uniqueItems": true,
          "items": {
            "$ref": "#/definitions/reportingDescriptorRelationship"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the report.",
          "$ref": "#/definitions/propertyBag"
        }
      },
      "required": [ "id" ]
    },

    "reportingConfiguration": {
      "description": "Information about a rule or notification that can be configured at runtime."
,
      "type": "object",
      "additionalProperties": false,
      "properties": {

        "enabled": {
          "description": "Specifies whether the report may be produced during the scan.",
          "type": "boolean",
          "default": true
        },

        "level": {
          "description": "Specifies the failure level for the report.",
          "default": "warning",
          "enum": [ "none", "note", "warning", "error" ],
          "type": "string"
        },
```

```
1940
1941        "rank": {
1942          "description": "Specifies the relative priority of the report. Used for analysis output
      only.",
1943          "type": "number",
1944          "default": -1.0,
1945          "minimum": -1.0,
1946          "maximum": 100.0
1947        },
1948
1949        "parameters": {
1950          "description": "Contains configuration information specific to a report.",
1951          "$ref": "#/definitions/propertyBag"
1952        },
1953
1954        "properties": {
1955          "description": "Key/value pairs that provide additional information about the reporting
      configuration.",
1956          "$ref": "#/definitions/propertyBag"
1957        }
1958      }
1959    },
1960
1961    "reportingDescriptorReference": {
1962      "description": "Information about how to locate a relevant reporting descriptor.",
1963      "type": "object",
1964      "additionalProperties": false,
1965      "properties": {
1966
1967        "id": {
1968          "description": "The id of the descriptor.",
1969          "type": "string"
1970        },
1971
1972        "index": {
1973          "description": "The index into an array of descriptors in toolComponent.ruleDescriptors,
       toolComponent.notificationDescriptors, or toolComponent.taxonomyDescriptors, depending on context
      .",
1974          "type": "integer",
1975          "default": -1,
1976          "minimum": -1
1977        },
1978
1979        "guid": {
1980          "description": "A guid that uniquely identifies the descriptor.",
1981          "type": "string",
1982          "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
      -9a-fA-F]{12}$"
1983        },
1984
1985        "toolComponent": {
1986          "description": "A reference used to locate the toolComponent associated with the descrip
      tor.",
1987          "$ref": "#/definitions/toolComponentReference"
1988        },
1989
1990        "properties": {
1991          "description": "Key/value pairs that provide additional information about the reporting
      descriptor reference.",
1992          "$ref": "#/definitions/propertyBag"
1993        }
1994      },
1995      "anyOf": [
1996        { "required": [ "index" ] },
1997        { "required": [ "guid" ] },
```

```
1998          { "required": [ "id" ] }
1999        ]
2000      },
2001
2002      "reportingDescriptorRelationship": {
2003        "description": "Information about the relation of one reporting descriptor to another.",
2004        "type": "object",
2005        "additionalProperties": false,
2006        "properties": {
2007
2008          "target": {
2009            "description": "A reference to the related reporting descriptor.",
2010            "$ref": "#/definitions/reportingDescriptorReference"
2011          },
2012
2013          "kinds": {
2014            "description": "A set of distinct strings that categorize the relationship. Well-known k
      inds include 'canPrecede', 'canFollow', 'willPrecede', 'willFollow', 'superset', 'subset', 'equal'
      , 'disjoint', 'relevant', and 'incomparable'.",
2015            "type": "array",
2016            "default": [ "relevant" ],
2017            "uniqueItems": true,
2018            "items": {
2019              "type": "string"
2020            }
2021          },
2022
2023          "description": {
2024            "description": "A description of the reporting descriptor relationship.",
2025            "$ref": "#/definitions/message"
2026          },
2027
2028          "properties": {
2029            "description": "Key/value pairs that provide additional information about the reporting
      descriptor reference.",
2030            "$ref": "#/definitions/propertyBag"
2031          }
2032        },
2033        "required": [ "target" ]
2034      },
2035
2036      "result": {
2037        "description": "A result produced by an analysis tool.",
2038        "additionalProperties": false,
2039        "type": "object",
2040        "properties": {
2041
2042          "ruleId": {
2043            "description": "The stable, unique identifier of the rule, if any, to which this result
      is relevant.",
2044            "type": "string"
2045          },
2046
2047          "ruleIndex": {
2048            "description": "The index within the tool component rules array of the rule object assoc
      iated with this result.",
2049            "type": "integer",
2050            "default": -1,
2051            "minimum": -1
2052          },
2053
2054          "rule": {
2055            "description": "A reference used to locate the rule descriptor relevant to this result."
      ,
2056            "$ref": "#/definitions/reportingDescriptorReference"
```

```
2057         },
2058
2059         "kind": {
2060           "description": "A value that categorizes results by evaluation state.",
2061           "default": "fail",
2062           "enum": [ "notApplicable", "pass", "fail", "review", "open", "informational" ],
2063           "type": "string"
2064         },
2065
2066         "level": {
2067           "description": "A value specifying the severity level of the result.",
2068           "default": "warning",
2069           "enum": [ "none", "note", "warning", "error" ],
2070           "type": "string"
2071         },
2072
2073         "message": {
2074           "description": "A message that describes the result. The first sentence of the message o
     nly will be displayed when visible space is limited.",
2075           "$ref": "#/definitions/message"
2076         },
2077
2078         "analysisTarget": {
2079           "description": "Identifies the artifact that the analysis tool was instructed to scan. T
     his need not be the same as the artifact where the result actually occurred.",
2080           "$ref": "#/definitions/artifactLocation"
2081         },
2082
2083         "locations": {
2084           "description": "The set of locations where the result was detected. Specify only one loc
     ation unless the problem indicated by the result can only be corrected by making a change at every
      specified location.",
2085           "type": "array",
2086           "minItems": 0,
2087           "uniqueItems": false,
2088           "default": [],
2089           "items": {
2090             "$ref": "#/definitions/location"
2091           }
2092         },
2093
2094         "guid": {
2095           "description": "A stable, unique identifier for the result in the form of a GUID.",
2096           "type": "string",
2097           "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
     -9a-fA-F]{12}$"
2098         },
2099
2100         "correlationGuid": {
2101           "description": "A stable, unique identifier for the equivalence class of logically ident
     ical results to which this result belongs, in the form of a GUID.",
2102           "type": "string",
2103           "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
     -9a-fA-F]{12}$"
2104         },
2105
2106         "occurrenceCount": {
2107           "description": "A positive integer specifying the number of times this logically unique
     result was observed in this run.",
2108           "type": "integer",
2109           "minimum": 1
2110         },
2111
2112         "partialFingerprints": {
2113           "description": "A set of strings that contribute to the stable, unique identity of the r
```

```
esult.",
          "type": "object",
          "additionalProperties": {
            "type": "string"
          }
        },

        "fingerprints": {
          "description": "A set of strings each of which individually defines a stable, unique ide
ntity for the result.",
          "type": "object",
          "additionalProperties": {
            "type": "string"
          }
        },

        "stacks": {
          "description": "An array of 'stack' objects relevant to the result.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/stack"
          }
        },

        "codeFlows": {
          "description": "An array of 'codeFlow' objects relevant to the result.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": false,
          "default": [],
          "items": {
            "$ref": "#/definitions/codeFlow"
          }
        },

        "graphs": {
          "description": "An array of zero or more unique graph objects associated with the result
.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/graph"
          }
        },

        "graphTraversals": {
          "description": "An array of one or more unique 'graphTraversal' objects.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/graphTraversal"
          }
        },

        "relatedLocations": {
          "description": "A set of locations relevant to this result.",
          "type": "array",
          "minItems": 0,
```

```
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/location"
          }
        },

        "suppressions": {
          "description": "A set of suppressions relevant to this result.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "items": {
            "$ref": "#/definitions/suppression"
          }
        },

        "baselineState": {
          "description": "The state of a result relative to a baseline of a previous run.",
          "enum": [
            "new",
            "unchanged",
            "updated",
            "absent"
          ],
          "type": "string"
        },

        "rank": {
          "description": "A number representing the priority or importance of the result.",
          "type": "number",
          "default": -1.0,
          "minimum": -1.0,
          "maximum": 100.0
        },

        "attachments": {
          "description": "A set of artifacts relevant to the result.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/attachment"
          }
        },

        "hostedViewerUri": {
          "description": "An absolute URI at which the result can be viewed.",
          "type": "string",
          "format": "uri"
        },

        "workItemUris": {
          "description": "The URIs of the work items associated with this result.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "items": {
            "type": "string",
            "format": "uri"
          }
        },

        "provenance": {
```

```
          "description": "Information about how and when the result was detected.",
          "$ref": "#/definitions/resultProvenance"
        },

        "fixes": {
          "description": "An array of 'fix' objects, each of which represents a proposed fix to th
e problem indicated by the result.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/fix"
          }
        },

        "taxa": {
          "description": "An array of references to taxonomy reporting descriptors that are applic
able to the result.",
          "type": "array",
          "default": [],
          "minItems": 0,
          "uniqueItems": true,
          "items": {
            "$ref": "#/definitions/reportingDescriptorReference"
          }
        },

        "webRequest": {
          "description": "A web request associated with this result.",
          "$ref": "#/definitions/webRequest"
        },

        "webResponse": {
          "description": "A web response associated with this result.",
          "$ref": "#/definitions/webResponse"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the result.",
          "$ref": "#/definitions/propertyBag"
        }
      },
      "required": [ "message" ]
    },

    "resultProvenance": {
      "description": "Contains information about how and when a result was detected.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "firstDetectionTimeUtc": {
          "description": "The Coordinated Universal Time (UTC) date and time at which the result w
as first detected. See \"Date/time properties\" in the SARIF spec for the required format.",
          "type": "string",
          "format": "date-time"
        },

        "lastDetectionTimeUtc": {
          "description": "The Coordinated Universal Time (UTC) date and time at which the result w
as most recently detected. See \"Date/time properties\" in the SARIF spec for the required format.
",
          "type": "string",
          "format": "date-time"
```

```
        },

        "firstDetectionRunGuid": {
          "description": "A GUID-valued string equal to the automationDetails.guid property of the
 run in which the result was first detected.",
          "type": "string",
          "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
-9a-fA-F]{12}$"
        },

        "lastDetectionRunGuid": {
          "description": "A GUID-valued string equal to the automationDetails.guid property of the
 run in which the result was most recently detected.",
          "type": "string",
          "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
-9a-fA-F]{12}$"
        },

        "invocationIndex": {
          "description": "The index within the run.invocations array of the invocation object whic
h describes the tool invocation that detected the result.",
          "type": "integer",
          "default": -1,
          "minimum": -1
        },

        "conversionSources": {
          "description": "An array of physicalLocation objects which specify the portions of an an
alysis tool's output that a converter transformed into the result.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/physicalLocation"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the result.",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "run": {
      "description": "Describes a single run of an analysis tool, and contains the reported output
 of that run.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "tool": {
          "description": "Information about the tool or tool pipeline that generated the results i
n this run. A run can only contain results produced by a single tool or tool pipeline. A run can a
ggregate results from multiple log files, as long as context around the tool run (tool command-lin
e arguments and the like) is identical for all aggregated files.",
          "$ref": "#/definitions/tool"
        },

        "invocations": {
          "description": "Describes the invocation of the analysis tool.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": false,
```

```
          "default": [],
          "items": {
            "$ref": "#/definitions/invocation"
          }
        },

        "conversion": {
          "description": "A conversion object that describes how a converter transformed an analys
is tool's native reporting format into the SARIF format.",
          "$ref": "#/definitions/conversion"
        },

        "language": {
          "description": "The language of the messages emitted into the log file during this run (
expressed as an ISO 639-1 two-letter lowercase culture code) and an optional region (expressed as
an ISO 3166-1 two-letter uppercase subculture code associated with a country or region). The casin
g is recommended but not required (in order for this data to conform to RFC5646).",
          "type": "string",
          "default": "en-US",
          "pattern": "^[a-zA-Z]{2}(-[a-zA-Z]{2})?$"
        },

        "versionControlProvenance": {
          "description": "Specifies the revision in version control of the artifacts that were sca
nned.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/versionControlDetails"
          }
        },

        "originalUriBaseIds": {
          "description": "The artifact location specified by each uriBaseId symbol on the machine
where the tool originally ran.",
          "type": "object",
          "additionalProperties": {
            "$ref": "#/definitions/artifactLocation"
          }
        },

        "artifacts": {
          "description": "An array of artifact objects relevant to the run.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "items": {
            "$ref": "#/definitions/artifact"
          }
        },

        "logicalLocations": {
          "description": "An array of logical locations such as namespaces, types or functions.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/logicalLocation"
          }
        },

        "graphs": {
```

```
2415          "description": "An array of zero or more unique graph objects associated with the run.",
2416          "type": "array",
2417          "minItems": 0,
2418          "uniqueItems": true,
2419          "default": [],
2420          "items": {
2421            "$ref": "#/definitions/graph"
2422          }
2423        },
2424
2425        "results": {
2426          "description": "The set of results contained in an SARIF log. The results array can be o
      mitted when a run is solely exporting rules metadata. It must be present (but may be empty) if a l
      og file represents an actual scan.",
2427          "type": "array",
2428          "minItems": 0,
2429          "uniqueItems": false,
2430          "items": {
2431            "$ref": "#/definitions/result"
2432          }
2433        },
2434
2435        "automationDetails": {
2436          "description": "Automation details that describe this run.",
2437          "$ref": "#/definitions/runAutomationDetails"
2438        },
2439
2440        "runAggregates": {
2441          "description": "Automation details that describe the aggregate of runs to which this run
       belongs.",
2442          "type": "array",
2443          "minItems": 0,
2444          "uniqueItems": true,
2445          "default": [],
2446          "items": {
2447            "$ref": "#/definitions/runAutomationDetails"
2448          }
2449        },
2450
2451        "baselineGuid": {
2452          "description": "The 'guid' property of a previous SARIF 'run' that comprises the baselin
      e that was used to compute result 'baselineState' properties for the run.",
2453          "type": "string",
2454          "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
      -9a-fA-F]{12}$"
2455        },
2456
2457        "redactionTokens": {
2458          "description": "An array of strings used to replace sensitive information in a redaction
      -aware property.",
2459          "type": "array",
2460          "minItems": 0,
2461          "uniqueItems": true,
2462          "default": [],
2463          "items": {
2464            "type": "string"
2465          }
2466        },
2467
2468        "defaultEncoding": {
2469          "description": "Specifies the default encoding for any artifact object that refers to a
      text file.",
2470          "type": "string"
2471        },
2472
```

```
2473          "defaultSourceLanguage": {
2474            "description": "Specifies the default source language for any artifact object that refer
      s to a text file that contains source code.",
2475            "type": "string"
2476          },
2477
2478          "newlineSequences": {
2479            "description": "An ordered list of character sequences that were treated as line breaks
      when computing region information for the run.",
2480            "type": "array",
2481            "minItems": 1,
2482            "uniqueItems": true,
2483            "default": [ "\r\n", "\n" ],
2484            "items": {
2485              "type": "string"
2486            }
2487          },
2488
2489          "columnKind": {
2490            "description": "Specifies the unit in which the tool measures columns.",
2491            "enum": [ "utf16CodeUnits", "unicodeCodePoints" ],
2492            "type": "string"
2493          },
2494
2495          "externalPropertyFileReferences": {
2496            "description": "References to external property files that should be inlined with the co
      ntent of a root log file.",
2497            "$ref": "#/definitions/externalPropertyFileReferences"
2498          },
2499
2500          "threadFlowLocations": {
2501            "description": "An array of threadFlowLocation objects cached at run level.",
2502            "type": "array",
2503            "minItems": 0,
2504            "uniqueItems": true,
2505            "default": [],
2506            "items": {
2507              "$ref": "#/definitions/threadFlowLocation"
2508            }
2509          },
2510
2511          "taxonomies": {
2512            "description": "An array of toolComponent objects relevant to a taxonomy in which result
      s are categorized.",
2513            "type": "array",
2514            "minItems": 0,
2515            "uniqueItems": true,
2516            "default": [],
2517            "items": {
2518              "$ref": "#/definitions/toolComponent"
2519            }
2520          },
2521
2522          "addresses": {
2523            "description": "Addresses associated with this run instance, if any.",
2524            "type": "array",
2525            "minItems": 0,
2526            "uniqueItems": false,
2527            "default": [],
2528            "items": {
2529              "$ref": "#/definitions/address"
2530            }
2531          },
2532
2533          "translations": {
```

```
2534          "description": "The set of available translations of the localized data provided by the
      tool.",
2535          "type": "array",
2536          "minItems": 0,
2537          "uniqueItems": true,
2538          "default": [],
2539          "items": {
2540            "$ref": "#/definitions/toolComponent"
2541          }
2542        },
2543
2544        "policies": {
2545          "description": "Contains configurations that may potentially override both reportingDesc
      riptor.defaultConfiguration (the tool's default severities) and invocation.configurationOverrides
      (severities established at run-time from the command line).",
2546          "type": "array",
2547          "minItems": 0,
2548          "uniqueItems": true,
2549          "default": [],
2550          "items": {
2551            "$ref": "#/definitions/toolComponent"
2552          }
2553        },
2554
2555        "webRequests": {
2556          "description": "An array of request objects cached at run level.",
2557          "type": "array",
2558          "minItems": 0,
2559          "uniqueItems": true,
2560          "default": [],
2561          "items": {
2562            "$ref": "#/definitions/webRequest"
2563          }
2564        },
2565
2566        "webResponses": {
2567          "description": "An array of response objects cached at run level.",
2568          "type": "array",
2569          "minItems": 0,
2570          "uniqueItems": true,
2571          "default": [],
2572          "items": {
2573            "$ref": "#/definitions/webResponse"
2574          }
2575        },
2576
2577        "specialLocations": {
2578          "description": "A specialLocations object that defines locations of special significance
       to SARIF consumers.",
2579          "$ref": "#/definitions/specialLocations"
2580        },
2581
2582        "properties": {
2583          "description": "Key/value pairs that provide additional information about the run.",
2584          "$ref": "#/definitions/propertyBag"
2585        }
2586      },
2587
2588      "required": [ "tool" ]
2589    },
2590
2591    "runAutomationDetails": {
2592      "description": "Information that describes a run's identity and role within an engineering s
      ystem process.",
2593      "additionalProperties": false,
```

```
      "type": "object",
      "properties": {

        "description": {
          "description": "A description of the identity and role played within the engineering system by this object's containing run object.",
          "$ref": "#/definitions/message"
        },

        "id": {
          "description": "A hierarchical string that uniquely identifies this object's containing run object.",
          "type": "string"
        },

        "guid": {
          "description": "A stable, unique identifier for this object's containing run object in the form of a GUID.",
          "type": "string",
          "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0-9a-fA-F]{12}$"
        },

        "correlationGuid": {
          "description": "A stable, unique identifier for the equivalence class of runs to which this object's containing run object belongs in the form of a GUID.",
          "type": "string",
          "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0-9a-fA-F]{12}$"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the run automation details.",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "specialLocations": {
      "description": "Defines locations of special significance to SARIF consumers.",
      "type": "object",
      "additionalProperties": false,
      "properties": {

        "displayBase": {
          "description": "Provides a suggestion to SARIF consumers to display file paths relative to the specified location.",
          "$ref": "#/definitions/artifactLocation"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the special locations.",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "stack": {
      "description": "A call stack that is relevant to a result.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

```

```
        "message": {
          "description": "A message relevant to this call stack.",
          "$ref": "#/definitions/message"
        },

        "frames": {
          "description": "An array of stack frames that represents a sequence of calls, rendered in reverse chronological order, that comprise the call stack.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": false,
          "items": {
            "$ref": "#/definitions/stackFrame"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the stack.",
          "$ref": "#/definitions/propertyBag"
        }
      },
      "required": [ "frames" ]
    },

    "stackFrame": {
      "description": "A function call within a stack trace.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "location": {
          "description": "The location to which this stack frame refers.",
          "$ref": "#/definitions/location"
        },

        "module": {
          "description": "The name of the module that contains the code of this stack frame.",
          "type": "string"
        },

        "threadId": {
          "description": "The thread identifier of the stack frame.",
          "type": "integer"
        },

        "parameters": {
          "description": "The parameters of the call that is executing.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": false,
          "default": [],
          "items": {
            "type": "string",
            "default": []
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the stack frame.",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

```

```
    "suppression": {
      "description": "A suppression that is relevant to a result.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "guid": {
          "description": "A stable, unique identifier for the suprression in the form of a GUID.",
          "type": "string",
          "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0-9a-fA-F]{12}$"
        },

        "kind": {
          "description": "A string that indicates where the suppression is persisted.",
          "enum": [
            "inSource",
            "external"
          ],
          "type": "string"
        },

        "status": {
          "description": "A string that indicates the review status of the suppression.",
          "enum": [
            "accepted",
            "underReview",
            "rejected"
          ],
          "type": "string"
        },

        "justification": {
          "description": "A string representing the justification for the suppression.",
          "type": "string"
        },

        "location": {
          "description": "Identifies the location associated with the suppression.",
          "$ref": "#/definitions/location"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the suppression.",
          "$ref": "#/definitions/propertyBag"
        }
      },
      "required": [ "kind" ]
    },

    "threadFlow": {
      "description": "Describes a sequence of code locations that specify a path through a single thread of execution such as an operating system or fiber.",
      "type": "object",
      "additionalProperties": false,
      "properties": {

        "id": {
          "description": "An string that uniquely identifies the threadFlow within the codeFlow in which it occurs.",
          "type": "string"
        },

        "message": {
```

```
2774            "description": "A message relevant to the thread flow.",
2775            "$ref": "#/definitions/message"
2776          },
2777
2778
2779          "initialState": {
2780            "description": "Values of relevant expressions at the start of the thread flow that may
      change during thread flow execution.",
2781            "type": "object",
2782            "additionalProperties": {
2783              "$ref": "#/definitions/multiformatMessageString"
2784            }
2785          },
2786
2787          "immutableState": {
2788            "description": "Values of relevant expressions at the start of the thread flow that rema
      in constant.",
2789            "type": "object",
2790            "additionalProperties": {
2791              "$ref": "#/definitions/multiformatMessageString"
2792            }
2793          },
2794
2795          "locations": {
2796            "description": "A temporally ordered array of 'threadFlowLocation' objects, each of whic
      h describes a location visited by the tool while producing the result.",
2797            "type": "array",
2798            "minItems": 1,
2799            "uniqueItems": false,
2800            "items": {
2801              "$ref": "#/definitions/threadFlowLocation"
2802            }
2803          },
2804
2805          "properties": {
2806            "description": "Key/value pairs that provide additional information about the thread flo
      w.",
2807            "$ref": "#/definitions/propertyBag"
2808          }
2809        },
2810
2811        "required": [ "locations" ]
2812      },
2813
2814      "threadFlowLocation": {
2815        "description": "A location visited by an analysis tool while simulating or monitoring the ex
      ecution of a program.",
2816        "additionalProperties": false,
2817        "type": "object",
2818        "properties": {
2819
2820          "index": {
2821            "description": "The index within the run threadFlowLocations array.",
2822            "type": "integer",
2823            "default": -1,
2824            "minimum": -1
2825          },
2826
2827          "location": {
2828            "description": "The code location.",
2829            "$ref": "#/definitions/location"
2830          },
2831
2832          "stack": {
2833            "description": "The call stack leading to this location.",
```

```
2834            "$ref": "#/definitions/stack"
2835          },
2836
2837          "kinds": {
2838            "description": "A set of distinct strings that categorize the thread flow location. Well
      -known kinds include 'acquire', 'release', 'enter', 'exit', 'call', 'return', 'branch', 'implicit'
      , 'false', 'true', 'caution', 'danger', 'unknown', 'unreachable', 'taint', 'function', 'handler',
      'lock', 'memory', 'resource', 'scope' and 'value'.",
2839            "type": "array",
2840            "minItems": 0,
2841            "uniqueItems": true,
2842            "default": [],
2843            "items": {
2844              "type": "string"
2845            }
2846          },
2847
2848          "taxa": {
2849            "description": "An array of references to rule or taxonomy reporting descriptors that ar
      e applicable to the thread flow location.",
2850            "type": "array",
2851            "default": [],
2852            "minItems": 0,
2853            "uniqueItems": true,
2854            "items": {
2855              "$ref": "#/definitions/reportingDescriptorReference"
2856            }
2857          },
2858
2859          "module": {
2860            "description": "The name of the module that contains the code that is executing.",
2861            "type": "string"
2862          },
2863
2864          "state": {
2865            "description": "A dictionary, each of whose keys specifies a variable or expression, the
       associated value of which represents the variable or expression value. For an annotation of kind
      'continuation', for example, this dictionary might hold the current assumed values of a set of glo
      bal variables.",
2866            "type": "object",
2867            "additionalProperties": {
2868              "$ref": "#/definitions/multiformatMessageString"
2869            }
2870          },
2871
2872          "nestingLevel": {
2873            "description": "An integer representing a containment hierarchy within the thread flow."
      ,
2874            "type": "integer",
2875            "minimum": 0
2876          },
2877
2878          "executionOrder": {
2879            "description": "An integer representing the temporal order in which execution reached th
      is location.",
2880            "type": "integer",
2881            "default": -1,
2882            "minimum": -1
2883          },
2884
2885          "executionTimeUtc": {
2886            "description": "The Coordinated Universal Time (UTC) date and time at which this locatio
      n was executed.",
2887            "type": "string",
2888            "format": "date-time"
```

```
        },

        "importance": {
          "description": "Specifies the importance of this location in understanding the code flow in which it occurs. The order from most to least important is \"essential\", \"important\", \"unimportant\". Default: \"important\".",
          "enum": [ "important", "essential", "unimportant" ],
          "default": "important",
          "type": "string"
        },

        "webRequest": {
          "description": "A web request associated with this thread flow location.",
          "$ref": "#/definitions/webRequest"
        },

        "webResponse": {
          "description": "A web response associated with this thread flow location.",
          "$ref": "#/definitions/webResponse"
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the threadflow location.",
          "$ref": "#/definitions/propertyBag"
        }
      }
    },

    "tool": {
      "description": "The analysis tool that was run.",
      "additionalProperties": false,
      "type": "object",
      "properties": {

        "driver": {
          "description": "The analysis tool that was run.",
          "$ref": "#/definitions/toolComponent"
        },

        "extensions": {
          "description": "Tool extensions that contributed to or reconfigured the analysis tool that was run.",
          "type": "array",
          "minItems": 0,
          "uniqueItems": true,
          "default": [],
          "items": {
            "$ref": "#/definitions/toolComponent"
          }
        },

        "properties": {
          "description": "Key/value pairs that provide additional information about the tool.",
          "$ref": "#/definitions/propertyBag"
        }
      },

      "required": [ "driver" ]
    },

    "toolComponent": {
      "description": "A component, such as a plug-in or the driver, of the analysis tool that was run.",
      "additionalProperties": false,
```

```
2949        "type": "object",
2950        "properties": {
2951
2952          "guid": {
2953            "description": "A unique identifier for the tool component in the form of a GUID.",
2954            "type": "string",
2955            "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
      -9a-fA-F]{12}$"
2956          },
2957
2958          "name": {
2959            "description": "The name of the tool component.",
2960            "type": "string"
2961          },
2962
2963          "organization": {
2964            "description": "The organization or company that produced the tool component.",
2965            "type": "string"
2966          },
2967
2968          "product": {
2969            "description": "A product suite to which the tool component belongs.",
2970            "type": "string"
2971          },
2972
2973          "productSuite": {
2974            "description": "A localizable string containing the name of the suite of products to whi
      ch the tool component belongs.",
2975            "type": "string"
2976          },
2977
2978          "shortDescription": {
2979            "description": "A brief description of the tool component.",
2980            "$ref": "#/definitions/multiformatMessageString"
2981          },
2982
2983          "fullDescription": {
2984            "description": "A comprehensive description of the tool component.",
2985            "$ref": "#/definitions/multiformatMessageString"
2986          },
2987
2988          "fullName": {
2989            "description": "The name of the tool component along with its version and any other usef
      ul identifying information, such as its locale.",
2990            "type": "string"
2991          },
2992
2993          "version": {
2994            "description": "The tool component version, in whatever format the component natively pr
      ovides.",
2995            "type": "string"
2996          },
2997
2998          "semanticVersion": {
2999            "description": "The tool component version in the format specified by Semantic Versionin
      g 2.0.",
3000            "type": "string"
3001          },
3002
3003          "dottedQuadFileVersion": {
3004            "description": "The binary version of the tool component's primary executable file expre
      ssed as four non-negative integers separated by a period (for operating systems that express file
      versions in this way).",
3005            "type": "string",
3006            "pattern": "[0-9]+(\\.[0-9]+){3}"
```

```
3007         },
3008
3009         "releaseDateUtc": {
3010           "description": "A string specifying the UTC date (and optionally, the time) of the compo
     nent's release.",
3011           "type": "string"
3012         },
3013
3014         "downloadUri": {
3015           "description": "The absolute URI from which the tool component can be downloaded.",
3016           "type": "string",
3017           "format": "uri"
3018         },
3019
3020         "informationUri": {
3021           "description": "The absolute URI at which information about this version of the tool com
     ponent can be found.",
3022           "type": "string",
3023           "format": "uri"
3024         },
3025
3026         "globalMessageStrings": {
3027           "description": "A dictionary, each of whose keys is a resource identifier and each of wh
     ose values is a multiformatMessageString object, which holds message strings in plain text and (op
     tionally) Markdown format. The strings can include placeholders, which can be used to construct a
     message in combination with an arbitrary number of additional string arguments.",
3028           "type": "object",
3029           "additionalProperties": {
3030             "$ref": "#/definitions/multiformatMessageString"
3031           }
3032         },
3033
3034         "notifications": {
3035           "description": "An array of reportingDescriptor objects relevant to the notifications re
     lated to the configuration and runtime execution of the tool component.",
3036           "type": "array",
3037           "minItems": 0,
3038           "uniqueItems": true,
3039           "default": [],
3040           "items": {
3041             "$ref": "#/definitions/reportingDescriptor"
3042           }
3043         },
3044
3045         "rules": {
3046           "description": "An array of reportingDescriptor objects relevant to the analysis perform
     ed by the tool component.",
3047           "type": "array",
3048           "minItems": 0,
3049           "uniqueItems": true,
3050           "default": [],
3051           "items": {
3052             "$ref": "#/definitions/reportingDescriptor"
3053           }
3054         },
3055
3056         "taxa": {
3057           "description": "An array of reportingDescriptor objects relevant to the definitions of b
     oth standalone and tool-defined taxonomies.",
3058           "type": "array",
3059           "minItems": 0,
3060           "uniqueItems": true,
3061           "default": [],
3062           "items": {
3063             "$ref": "#/definitions/reportingDescriptor"
```

```
3064           }
3065         },
3066
3067         "locations": {
3068           "description": "An array of the artifactLocation objects associated with the tool compon
     ent.",
3069           "type": "array",
3070           "minItems": 0,
3071           "default": [],
3072           "items": {
3073             "$ref": "#/definitions/artifactLocation"
3074           }
3075         },
3076
3077         "language": {
3078           "description": "The language of the messages emitted into the log file during this run (
     expressed as an ISO 639-1 two-letter lowercase language code) and an optional region (expressed as
      an ISO 3166-1 two-letter uppercase subculture code associated with a country or region). The casi
     ng is recommended but not required (in order for this data to conform to RFC5646).",
3079           "type": "string",
3080           "default": "en-US",
3081           "pattern": "^[a-zA-Z]{2}(-[a-zA-Z]{2})?$"
3082         },
3083
3084         "contents": {
3085           "description": "The kinds of data contained in this object.",
3086           "type": "array",
3087           "uniqueItems": true,
3088           "default": [ "localizedData", "nonLocalizedData" ],
3089           "items": {
3090             "enum": [
3091               "localizedData",
3092               "nonLocalizedData"
3093             ],
3094             "type": "string"
3095           }
3096         },
3097
3098         "isComprehensive": {
3099           "description": "Specifies whether this object contains a complete definition of the loca
     lizable and/or non-localizable data for this component, as opposed to including only data that is
     relevant to the results persisted to this log file.",
3100           "type": "boolean",
3101           "default": false
3102         },
3103
3104         "localizedDataSemanticVersion": {
3105           "description": "The semantic version of the localized strings defined in this component;
      maintained by components that provide translations.",
3106           "type": "string"
3107         },
3108
3109         "minimumRequiredLocalizedDataSemanticVersion": {
3110           "description": "The minimum value of localizedDataSemanticVersion required in translatio
     ns consumed by this component; used by components that consume translations.",
3111           "type": "string"
3112         },
3113
3114         "associatedComponent": {
3115           "description": "The component which is strongly associated with this component. For a tr
     anslation, this refers to the component which has been translated. For an extension, this is the d
     river that provides the extension's plugin model.",
3116           "$ref": "#/definitions/toolComponentReference"
3117         },
3118
```

```
3119          "translationMetadata": {
3120            "description": "Translation metadata, required for a translation, not populated by other
       component types.",
3121            "$ref": "#/definitions/translationMetadata"
3122          },
3123
3124          "supportedTaxonomies": {
3125            "description": "An array of toolComponentReference objects to declare the taxonomies sup
      ported by the tool component.",
3126            "type": "array",
3127            "minItems": 0,
3128            "uniqueItems": true,
3129            "default": [],
3130            "items": {
3131              "$ref": "#/definitions/toolComponentReference"
3132            }
3133          },
3134
3135          "properties": {
3136            "description": "Key/value pairs that provide additional information about the tool compo
      nent.",
3137            "$ref": "#/definitions/propertyBag"
3138          }
3139        },
3140
3141        "required": [ "name" ]
3142      },
3143
3144      "toolComponentReference": {
3145        "description": "Identifies a particular toolComponent object, either the driver or an extens
      ion.",
3146        "type": "object",
3147        "additionalProperties": false,
3148        "properties": {
3149
3150          "name": {
3151            "description": "The 'name' property of the referenced toolComponent.",
3152            "type": "string"
3153          },
3154
3155          "index": {
3156            "description": "An index into the referenced toolComponent in tool.extensions.",
3157            "type": "integer",
3158            "default": -1,
3159            "minimum": -1
3160          },
3161
3162          "guid": {
3163            "description": "The 'guid' property of the referenced toolComponent.",
3164            "type": "string",
3165            "pattern": "^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0
      -9a-fA-F]{12}$"
3166          },
3167
3168          "properties": {
3169            "description": "Key/value pairs that provide additional information about the toolCompon
      entReference.",
3170            "$ref": "#/definitions/propertyBag"
3171          }
3172        }
3173      },
3174
3175      "translationMetadata": {
3176        "description": "Provides additional metadata related to translation.",
3177        "type": "object",
```

```
3178       "additionalProperties": false,
3179       "properties": {
3180
3181         "name": {
3182           "description": "The name associated with the translation metadata.",
3183           "type": "string"
3184         },
3185
3186         "fullName": {
3187           "description": "The full name associated with the translation metadata.",
3188           "type": "string"
3189         },
3190
3191         "shortDescription": {
3192           "description": "A brief description of the translation metadata.",
3193           "$ref": "#/definitions/multiformatMessageString"
3194         },
3195
3196         "fullDescription": {
3197           "description": "A comprehensive description of the translation metadata.",
3198           "$ref": "#/definitions/multiformatMessageString"
3199         },
3200
3201         "downloadUri": {
3202           "description": "The absolute URI from which the translation metadata can be downloaded."
     ,
3203           "type": "string",
3204           "format": "uri"
3205         },
3206
3207         "informationUri": {
3208           "description": "The absolute URI from which information related to the translation metad
     ata can be downloaded.",
3209           "type": "string",
3210           "format": "uri"
3211         },
3212
3213         "properties": {
3214           "description": "Key/value pairs that provide additional information about the translatio
     n metadata.",
3215           "$ref": "#/definitions/propertyBag"
3216         }
3217       },
3218       "required": [ "name" ]
3219     },
3220
3221     "versionControlDetails": {
3222       "description": "Specifies the information necessary to retrieve a desired revision from a ve
     rsion control system.",
3223       "type": "object",
3224       "additionalProperties": false,
3225       "properties": {
3226
3227         "repositoryUri": {
3228           "description": "The absolute URI of the repository.",
3229           "type": "string",
3230           "format": "uri"
3231         },
3232
3233         "revisionId": {
3234           "description": "A string that uniquely and permanently identifies the revision within th
     e repository.",
3235           "type": "string"
3236         },
3237
```

```
3238          "branch": {
3239            "description": "The name of a branch containing the revision.",
3240            "type": "string"
3241          },
3242
3243          "revisionTag": {
3244            "description": "A tag that has been applied to the revision.",
3245            "type": "string"
3246          },
3247
3248          "asOfTimeUtc": {
3249            "description": "A Coordinated Universal Time (UTC) date and time that can be used to synchronize an enlistment to the state of the repository at that time.",
3250            "type": "string",
3251            "format": "date-time"
3252          },
3253
3254          "mappedTo": {
3255            "description": "The location in the local file system to which the root of the repository was mapped at the time of the analysis.",
3256            "$ref": "#/definitions/artifactLocation"
3257          },
3258
3259          "properties": {
3260            "description": "Key/value pairs that provide additional information about the version control details.",
3261            "$ref": "#/definitions/propertyBag"
3262          }
3263        },
3264
3265        "required": [ "repositoryUri" ]
3266      },
3267
3268      "webRequest": {
3269        "description": "Describes an HTTP request.",
3270        "type": "object",
3271        "additionalProperties": false,
3272        "properties": {
3273
3274          "index": {
3275            "description": "The index within the run.webRequests array of the request object associated with this result.",
3276            "type": "integer",
3277            "default": -1,
3278            "minimum": -1
3279
3280          },
3281
3282          "protocol": {
3283            "description": "The request protocol. Example: 'http'.",
3284            "type": "string"
3285          },
3286
3287          "version": {
3288            "description": "The request version. Example: '1.1'.",
3289            "type": "string"
3290          },
3291
3292          "target": {
3293            "description": "The target of the request.",
3294            "type": "string"
3295          },
3296
3297          "method": {
3298            "description": "The HTTP method. Well-known values are 'GET', 'PUT', 'POST', 'DELETE', '
```

```
      PATCH', 'HEAD', 'OPTIONS', 'TRACE', 'CONNECT'.",
3299            "type": "string"
3300          },
3301
3302          "headers": {
3303            "description": "The request headers.",
3304            "type": "object",
3305            "additionalProperties": {
3306              "type": "string"
3307            }
3308          },
3309
3310          "parameters": {
3311            "description": "The request parameters.",
3312            "type": "object",
3313            "additionalProperties": {
3314              "type": "string"
3315            }
3316          },
3317
3318          "body": {
3319            "description": "The body of the request.",
3320            "$ref": "#/definitions/artifactContent"
3321          },
3322
3323          "properties": {
3324            "description": "Key/value pairs that provide additional information about the request.",
3325            "$ref": "#/definitions/propertyBag"
3326          }
3327        }
3328      },
3329
3330      "webResponse": {
3331        "description": "Describes the response to an HTTP request.",
3332        "type": "object",
3333        "additionalProperties": false,
3334        "properties": {
3335
3336          "index": {
3337            "description": "The index within the run.webResponses array of the response object assoc
      iated with this result.",
3338            "type": "integer",
3339            "default": -1,
3340            "minimum": -1
3341          },
3342
3343          "protocol": {
3344            "description": "The response protocol. Example: 'http'.",
3345            "type": "string"
3346          },
3347
3348          "version": {
3349            "description": "The response version. Example: '1.1'.",
3350            "type": "string"
3351          },
3352
3353          "statusCode": {
3354            "description": "The response status code. Example: 451.",
3355            "type": "integer"
3356          },
3357
3358          "reasonPhrase": {
3359            "description": "The response reason. Example: 'Not found'.",
3360            "type": "string"
3361          },
```

```
3362
3363          "headers": {
3364            "description": "The response headers.",
3365            "type": "object",
3366            "additionalProperties": {
3367              "type": "string"
3368            }
3369          },
3370
3371          "body": {
3372            "description": "The body of the response.",
3373            "$ref": "#/definitions/artifactContent"
3374          },
3375
3376          "noResponseReceived": {
3377            "description": "Specifies whether a response was received from the server.",
3378            "type": "boolean",
3379            "default": false
3380          },
3381
3382          "properties": {
3383            "description": "Key/value pairs that provide additional information about the response."
    ,
3384            "$ref": "#/definitions/propertyBag"
3385          }
3386        }
3387      }
3388    }
3389  }
3390
```

## 07-python-sast-scanner/scripts/make_charts.py

```
"""Charts from the JSON reports written by run_demo.sh (outputs/*.png)."""
import json
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

OUT = Path("outputs")
COLORS = {"HIGH": "#c62828", "MEDIUM": "#ef6c00", "LOW": "#607d8b"}


def load(p):
    return json.loads(Path(p).read_text())


def rule_severity_bars(report, title, path):
    findings = [f for f in report["findings"] if not f["suppressed"]] or report["findings"]
    rules = sorted({f["rule_id"] + " " + f["rule_name"] for f in report["findings"]})
    fig, ax = plt.subplots(figsize=(8, 4.2))
    left = [0] * len(rules)
    for sev in ("HIGH", "MEDIUM", "LOW"):
        vals = [sum(1 for f in findings if f["severity"] == sev and f"{f['rule_id']} {f['rule_name
']}" == r)
                for r in rules]
        ax.barh(rules, vals, left=left, color=COLORS[sev], label=sev)
        left = [a + b for a, b in zip(left, vals)]
    for i, v in enumerate(left):
        ax.text(v + 0.1, i, str(v), va="center", fontsize=9)
    ax.invert_yaxis()
    ax.set_xlabel("findings")
    ax.set_title(title)
    ax.legend(loc="lower right", fontsize=8)
    fig.tight_layout()
    fig.savefig(path, dpi=120)
    plt.close(fig)


def main():
    rule_severity_bars(load(OUT / "reports/vulnerable.json"),
                       "Deliberately vulnerable sample: findings by rule and severity",
                       OUT / "chart_vulnerable_by_rule.png")
    rule_severity_bars(load(OUT / "real/requests.json"),
                       "psf/requests v2.32.3: raw findings before triage",
                       OUT / "chart_requests_by_rule.png")
    ev = load(OUT / "evaluation.json")
    rules = sorted(ev["per_rule"])
    fig, ax = plt.subplots(figsize=(8, 3.6))
    ax.bar(rules, [ev["per_rule"][r]["expected"] for r in rules], color="#cfd8dc", label="planted"
)
    ax.bar(rules, [ev["per_rule"][r]["detected"] for r in rules], width=0.5, color="#2e7d32", labe
l="detected")
    ax.set_title(f"Sample corpus: {ev['true_positives']}/{ev['expected']} planted issues found, "
                 f"{len(ev['false_positives_benign'])} false positives on benign code")
    ax.legend(fontsize=8)
    fig.tight_layout()
    fig.savefig(OUT / "chart_corpus_recall.png", dpi=120)
    plt.close(fig)
    print("charts written: chart_vulnerable_by_rule.png, chart_requests_by_rule.png, chart_corpus_
recall.png")


if __name__ == "__main__":
    main()

```

```
"""Collect headline numbers from outputs/ into outputs/results.json."""
import json
import re
from pathlib import Path

O = Path("outputs")
load = lambda p: json.loads((O / p).read_text())
ev = load("evaluation.json")
vul, ben = load("reports/vulnerable.json")["summary"], load("reports/benign.json")["summary"]
sp, rq, rqt = (load(f"real/{n}.json") for n in ("sampleproject", "requests", "requests-triaged"))
m = re.search(r"(\d+) passed(?:, (\d+) failed)?(?:, (\d+) skipped)?", (O / "pytest_output.txt").read_text())
passed, failed, skipped = (int(x) if x else 0 for x in m.groups())
rs = rq["summary"]
results = {
    "title": "pysast: AST-based Python SAST scanner",
    "pitch": "Parses Python into a syntax tree (never runs it), flags 8 CWE-mapped risky patterns, "
             "and writes JSON, SARIF 2.1.0 and HTML reports with suppression support.",
    "tests": {"passed": passed, "failed": failed, "skipped": skipped},
    "headline": [
        f"{ev['true_positives']}/{ev['expected']} planted issues detected across all 8 rules, "
        f"0 false positives on benign look-alike code (own corpus, not an independent benchmark)",
        f"psf/requests v2.32.3: {rs['files_scanned']} files / {rs['lines_scanned']:,} lines scanned in "
        f"{rs['duration_s']} s, {rs['total']} findings, 0 exploitable after manual triage",
        f"pypa/sampleproject @ {sp['metadata']['commit'][:12]}: {sp['summary']['total']} findings",
        "SARIF output validates against the official OASIS 2.1.0 JSON schema",
        f"{passed} pytest tests pass, incl. a test proving scanned code is never executed",
    ],
    "metrics": {
        "rules": 8, "corpus_planted": ev["expected"], "corpus_detected": ev["true_positives"],
        "corpus_recall": ev["recall"], "corpus_precision": ev["precision"],
        "benign_false_positives": len(ev["false_positives_benign"]),
        "vulnerable_files": vul["files_scanned"], "vulnerable_lines": vul["lines_scanned"],
        "requests_files": rs["files_scanned"], "requests_lines": rs["lines_scanned"],
        "requests_scan_seconds": rs["duration_s"], "requests_findings": rs["total"],
        "requests_high": rs["by_severity"]["HIGH"], "requests_medium": rs["by_severity"]["MEDIUM"],
        "requests_low": rs["by_severity"]["LOW"], "requests_exploitable_after_triage": 0,
        "requests_suppressed_with_allowlist": rqt["summary"]["suppressed"],
        "sampleproject_files": sp["summary"]["files_scanned"], "sampleproject_findings": sp["summary"]["total"],
    },
    "data_sources": [
        "samples/vulnerable and samples/benign: written for this project; all credentials are fake strings",
        f"pypa/sampleproject commit {sp['metadata']['commit']} via https://github.com/pypa/sampleproject (zip archive)",
        f"psf/requests tag v2.32.3 (commit {rq['metadata']['commit']}) via https://github.com/psf/requests (zip archive)",
        "SARIF 2.1.0 schema: https://raw.githubusercontent.com/oasis-tcs/sarif-spec/main/sarif-2.1/schema/sarif-schema-2.1.0.json",
    ],
    "honesty_notes": [
        "Detection is syntactic only; no data-flow/taint proof that untrusted input reaches a sink.",
        "The 100% recall / 0 FP figure is on a corpus written together with the rules, not an independent benchmark.",
        "The requests triage (0 exploitable of 14) is a manual judgment recorded in samples/allowlists/requests-triage.yml.",
        "Known misses are pinned by tests: SQL built in a helper function; secrets under innocuous names.",
        "JavaScript scanning (optional in the original idea) was not built.",
```

**07-python-sast-scanner/scripts/make_results.py (continued)**

```
    ],
}
(O / "results.json").write_text(json.dumps(results, indent=2))
print(json.dumps(results["headline"], indent=2))

```

## 07-python-sast-scanner/scripts/run_demo.sh

```
#!/usr/bin/env bash
# End-to-end demo: rules -> corpus evaluation -> vulnerable sample scan ->
# fetch + scan two pinned real public repos -> triaged rescan -> charts.
# Usage (from the project root):  bash scripts/run_demo.sh [python]
set -euo pipefail
PY="${1:-python}"
mkdir -p outputs/reports outputs/real data/real

echo "== pysast rules"; $PY cli.py rules

echo; echo "== Ground-truth evaluation on samples/"
$PY cli.py evaluate --out outputs/evaluation.json | head -16

echo; echo "== Scan of the deliberately vulnerable sample repo"
$PY cli.py scan samples/vulnerable --name "samples/vulnerable" --show-suppressed \
    --out outputs/reports/vulnerable || true

echo; echo "== Scan of the benign look-alike corpus"
$PY cli.py scan samples/benign --name "samples/benign" --show-suppressed --out outputs/reports/ben
ign

SP_SHA=621e4974ca25ce531773def586ba3ed8e736b3fc
echo; echo "== Fetch pypa/sampleproject @ $SP_SHA (download + extract only; nothing is run)"
rm -rf data/real/sampleproject
$PY cli.py fetch-github pypa/sampleproject --ref "$SP_SHA" --dest data/real/sampleproject
$PY cli.py scan "data/real/sampleproject/sampleproject-$SP_SHA" --name "pypa/sampleproject" \
    --meta data/real/sampleproject/fetch_metadata.json --out outputs/real/sampleproject

echo; echo "== Fetch psf/requests v2.32.3"
rm -rf data/real/requests
$PY cli.py fetch-github psf/requests --ref v2.32.3 --dest data/real/requests > /dev/null
$PY -c "import json;m=json.load(open('data/real/requests/fetch_metadata.json'));print({k:m[k] for
k in ('repo','ref','commit','files_extracted','archive_sha256')})"
$PY cli.py scan data/real/requests/requests-2.32.3 --name "psf/requests v2.32.3" \
    --meta data/real/requests/fetch_metadata.json --out outputs/real/requests

echo; echo "== Same scan with the manual-triage allowlist"
$PY cli.py scan data/real/requests/requests-2.32.3 --name "psf/requests v2.32.3 (triaged)" \
    --meta data/real/requests/fetch_metadata.json --allowlist samples/allowlists/requests-triage.y
ml \
    --out outputs/real/requests-triaged --fail-on HIGH && echo "CI gate (fail-on HIGH): PASS"

$PY scripts/make_charts.py

```